UNIVERSITY OF GUJRAT
A WORLD CLASS UNIVERSITY

Department of Software Engineering

University of Gujrat

BotFizz - AI Chatbot Builder Platform

Session: BSSE 22

Project Advisor: Dr. Hafsa Sharif Dar

Submitted By

M. Hanzala Zaheer **Reg#** 22014198-060

M. Waleed Shams **Reg#** 22014198-079

STATEMENT OF SUBMISSION

This is certified that **M. Hanzala Zaheer** Roll No. **22014198-060** and **M. Waleed Shams** Roll No. **22014198-079** has successfully completed the final year project named as **BotFizz - AI Chatbot Builder Platform** at the Department of Software Engineering, University of Gujrat, to fulfill the requirement of the degree of **BS in Software Engineering.**

Acknowledgement

We extend our sincere gratitude to **Dr. Hafsa Sharif Dar**, our Project Advisor, for her invaluable guidance, continuous support, and insightful feedback throughout this project. Her expertise and encouragement were instrumental in shaping this work. We are deeply thankful to **Mr. Muhammad Tauqeer ul Amin**, Chairman of the Department of Software Engineering, University of Gujrat, for providing excellent academic resources and a conducive learning environment. Our heartfelt appreciation goes to the faculty members of the Department of Software Engineering for their knowledge and support. We also thank our friends and family members for their unwavering encouragement, patience, and silent support throughout this academic journey.

Date: 01-06-2026

© University of Gujrat

Abstract

BotFizz is a web-based AI chatbot builder platform designed to democratize access to intelligent conversational automation for small and medium businesses. The system enables users without programming expertise to create, train, and deploy AI-powered chatbots across multiple messaging platforms including WhatsApp, Instagram, and websites. By providing a visual drag-and-drop flow builder, integration with external AI services for natural language understanding, and real-time analytics, BotFizz eliminates the technical and financial barriers typically associated with chatbot development. This project implements a functional prototype demonstrating core features: user authentication, visual chatbot design, AI intent recognition, multiplatform integration, and conversation monitoring. The solution is built using React.js for the frontend, Node.js with Express for the backend, and PostgreSQL for data storage, following modern software engineering practices to ensure scalability, maintainability, and security.

© University of Gujrat

Table of Contents

CHAPTER 1: PROJECT FEASIBILITY REPORT 7
1.1 Introduction 8
1.2 Product Feasibility Report 8
1.2.1 Technical Feasibility 8
1.2.2 Operational Feasibility 9
1.2.3 Economic Feasibility 9
1.2.4 Schedule Feasibility 9
1.2.5 Specification Feasibility 10
1.2.6 Information Feasibility 10
1.2.7 Motivational Feasibility 10
1.2.8 Legal & Ethical Feasibility 10
1.3 Project Scope 10
1.4 Project Costing 11
1.4.1 Project Cost Estimation By Function Point Analysis 11
1.4.3 Activity Based Costing 15
1.5 Task Dependency Table 16
1.6 CPM - Critical Path Method 17
1. Specify the Individual Activities 17
2. Determine the Sequence of the Activities 17
5. Update CPM Diagram 20
1.7 Gantt chart 21
1.8 Introduction to Team member and their skill set 22
1.9 Task and Member Assignment Table 23
1.10 Tools and Technology with Reasoning 25
1.11 Vision Document 26
1. Introduction 26
2. Vision Statement 26
3. Objectives 26
4. Target Users 26
5. Key Features 26
6. Success Criteria 27
7. Assumptions 27
8. Risks 27
9. Constraints 27
1.12 Risk List 28
1.13 Product Features / Product Decomposition 28
CHAPTER 2: SOFTWARE REQUIREMENT SPECIFICATION (FOR OBJECT ORIENTED APPROACH) 30
2.1 INTRODUCTION: 31
2.1.1 Systems Specifications 32
2.1.2 Identifying External Entities 37
2.1.3 Context Level Data Flow Diagram: 38
2.1.4 Capture "shall" Statements: 38
2.1.8 High Level Usecase Diagram: 42
2.1.9 Analysis Level Usecase Diagram: 43
CHAPTER 3: DESIGN DOCUMENT (FOR OBJECT ORIENTED APPROACH) 49
3.2 Domain Model 50
Domain Model Diagram 51
3.3 System Sequence Diagram 52
3.4 Sequence Diagram 53
3.5 COLLABORATION DIAGRAM 55
3.6 Operation Contracts 56
Operation Contract 1: registerUser 56
Operation Contract 2: login 56
Operation Contract 3: createChatbot 56

© University of Gujrat

Operation Contract 4: addNode 57
Operation Contract 5: connectNodes 57
Operation Contract 6: enableAI 57
Operation Contract 7: uploadTrainingData 57
Operation Contract 8: connectWhatsApp 58
Operation Contract 9: connectInstagram 58
Operation Contract 10: deployBot 58
Operation Contract 11: processMessage 59
Operation Contract 12: getAnalytics 59
3.7 Design Class Diagram 60
3.8 State Chart Diagram 61
3.9 Data Model 63

CHAPTER 4: USER INTERFACE DESIGN 64

4.1 INTRODUCTION 65
4.3 STORY BOARDS 66
4.4 NAVIGATIONAL MAPS: 77
4.5 TRACE-ABILITY MATRIX 77

CHAPTER 5: SOFTWARE TESTING 84

5.1. INTRODUCTION 85
5.2. TEST PLAN 85
5.2.1. Purpose 85
5.2.2. Outline 85
5.3. TEST DESIGN SPECIFICATION 90
5.3.1. Purpose 90
5.3.2. Outline 90
5.4. TEST CASE SPECIFICATION 92
5.4.1. Purpose 92
5.4.2. Outline 92
5.4.2.1. Test Cases TDS-01: User Authentication 93
5.4.2.2. Test Cases TDS-02: Chatbot Flow Builder 97
5.4.2.3. Test Cases — TDS-03: AI and Integration 101
5.4.2.4. Test Cases — TDS-04: Deployment and Analytics 106
5.5. TEST PROCEDURE SPECIFICATION 110
5.5.1. Purpose 110
5.5.2. Outline 110
5.6. TEST ITEM TRANSMITTAL REPORT 111
5.6.1. Purpose 111
5.6.2. Outline 112
5.7. TEST LOG 113
5.7.1. Purpose 113
5.7.2. Outline - Activity and Event Entries 113
5.8. TEST INCIDENT REPORT 115
5.8.1. Purpose 115
5.8.2. Outline 116
5.9. TEST SUMMARY REPORT 117
5.9.1. Purpose 117
5.9.2. Outline 117

CHAPTER 6: USER TECHNICAL MANUAL 120

© University of Gujrat

Chapter 1: Project Feasibility Report

1.1 Introduction

Chatbots are increasingly used by businesses to automate customer support and lead handling across messaging platforms. However, most small and medium businesses cannot build or maintain AI chatbots due to technical complexity, high cost, and the difficulty of integrating with channels like WhatsApp, Instagram, and websites.

Botfizz is a web-based AI chatbot builder platform that allows users to design, train, and deploy chatbots without programming. It provides a visual flow builder, AI intent recognition, and integration with WhatsApp, Instagram, and an embeddable website widget through standard APIs. The goal is to make multi-channel chatbot deployment simple, fast, and affordable.

This project will deliver a functional prototype of Botfizz with the following objectives:

- To develop a visual web interface for building chatbot flows
- To integrate AI for intent understanding using an external model
- To implement connectors for WhatsApp, Instagram, and an embeddable website widget
- To provide a basic dashboard for monitoring and managing conversations

1.2 Product Feasibility Report

A feasibility study evaluates whether the proposed system can be realistically developed and deployed with the available resources, time, technology, and constraints. The objective is to determine whether the Botfizz platform is viable and worthwhile for development as a Final Year Project with potential for commercial use.

There are several types of feasibilities to consider:

- Technical
- Operational
- Economics
- Schedule
- Specification
- Information
- Motivational
- Legal and Ethical

1.2.1 Technical Feasibility

The technical feasibility of the Botfizz platform is high because the required tools, frameworks, and integration APIs are mature, accessible, and compatible with the development team's expertise.

The system will be developed using the following technology stack:

- **Frontend:** React.js for component-based UI and real-time interface rendering
- **Backend:** Node.js (Express) for REST API services and business logic
- **Database:** PostgreSQL for reliable structured data storage
- **AI Integration:** External AI/NLP APIs (e.g., OpenAI) for intent recognition
- **Messaging Integrations:** WhatsApp Business Cloud API, Instagram Messaging API & Embeddable website widget via standard script injection
- **Hosting Infrastructure:** VPS server (Digital ocean) instead of managed cloud
- **Development Tools:** GitHub, Postman, Docker (where needed)

© University of Gujrat

These technologies are widely used in industry and have extensive documentation, tutorials, and community support.

1.2.2 Operational Feasibility

Operational feasibility determines whether the solution will function effectively for its intended users. The platform is designed for small and medium businesses that lack technical expertise but require automated customer engagement. Botfizz provides a nocode interface to build chatbot flows, monitor conversations, and deploy AI bots without programming knowledge. Since users interact through a graphical dashboard and standard messaging channels, the system is practical and acceptable in real operational environments. The development team also possesses the required skills to operate and maintain the platform during the project lifecycle.

1.2.3 Economic Feasibility

Economic feasibility evaluates whether the financial requirements of the project are justified relative to its expected utility and long-term value. Since Botfizz is initially developed as an academic prototype and later intended for deployment, both predeployment and deployment-phase costs are considered.

For post-deployment, the expected costs are as follows:

Cost Component	Description	Estimated Cost
VPS Hosting (1 Year)	Required for production hosting of backend, frontend &database	40,000 PKR/year
Domain + SSL Botfizz.com (**already own**)	For public accessibility and secure communication	5,000 PKR/year
AI Intent Recognition API & Other Tools	Usage cost depends on customer load	35,000PKR/year (allocated budget)

(Estimated Deployment Cost Table)

Estimated Total Deployment Cost ≈ 80,000 PKR

These expenses become applicable only after deployment. Before that, development will rely on localhost and free-tier services to eliminate cost burden during the academic phase.

From a commercialization perspective, the system can generate recurring revenue by offering chatbot deployment services to SMEs on a subscription or setup basis. With even a small number of clients, the yearly revenue potential can exceed the deployment cost, making the solution financially viable.

Hence, the Botfizz platform is economically feasible, as its projected benefits and return on investment justify the estimated financial requirements.

1.2.4 Schedule Feasibility

The development activities are planned to be completed within the academic semester timeline through systematic phase-wise implementation: requirements, design, implementation, integration, testing, and deployment. Team capacity, complexity level, and technology readiness align with the project schedule. Critical milestones can be

achieved using the CPM and Gantt-based time allocation ensured in project planning. Thus, the project is feasible to complete within the allocated duration.

1.2.5 Specification Feasibility

The system requirements of Botfizz are clearly defined and within the scope of available technology and academic constraints. The primary specifications — chatbot builder interface, AI intent recognition, API integrations, and conversation dashboard — are measurable, implementable, and not dependent on unaffordable enterprise services. Advanced enterprise features such as CRM, mobile apps, and SMS are excluded to maintain feasibility within scope boundaries.

1.2.6 Information Feasibility

All required information for development is available through official documentation of WhatsApp Cloud API, Instagram API, OpenAI API, and standard web development frameworks. Industry literature and case studies on chatbot builders further support knowledge availability. Therefore, there is no dependency on unavailable or restricted information sources.

1.2.7 Motivational Feasibility

The project team is strongly motivated since the system has both academic significance and real commercialization potential. The usability of the platform for real clients increases commitment and encourages consistent development. The dual outcome of academic evaluation and business adoption further enhances motivational feasibility.

1.2.8 Legal & Ethical Feasibility

Development of Botfizz adheres to legal and ethical guidelines by using licensed APIs, respecting platform Terms of Service, and ensuring no unauthorized data misuse. User data confidentiality and consent are preserved, and no copyright or intellectual property infringement is involved. Therefore, the system is legally and ethically feasible.

1.3 Project Scope

The scope of the Botfizz project includes the development of a web-based AI chatbot builder platform that enables users to create and deploy chatbots without programming. The system will provide a visual flow builder, AI intent understanding via external APIs, and integration with WhatsApp, Instagram, and a website widget. A basic dashboard for conversation monitoring and bot management will also be delivered. The scope excludes mobile applications, SMS integration, custom AI model training, automated billing, and enterprise-level CRM support, which may be considered in future phases.

© University of Gujrat

1.4 Project Costing

1.4.1 Project Cost Estimation By Function Point Analysis

Function Point Analysis provides an objective measure of software functionality based on user-visible features. For BotFizz, we analyze the platform's inputs, outputs, inquiries, files, and interfaces to calculate development effort and cost.

Information domain values are defined as follows:

Number of user inputs:

- **User Authentication**
  1 Registration (Email, Password, Full Name, Organization)
  2 Login (Email, Password)
  3 Password Reset (Email, New Password)
  4 Profile Update (Name, Email, Organization Details)
- **Chatbot Builder**
  5 Add Conversation Node
  6 Add Response Message
  7 Add Conditional Logic
  8 Configure AI Settings
  9 Upload Training Data
- **Integration Settings**
  10 WhatsApp API Configuration (Business Account ID, API Key)
  11 Instagram API Configuration (Page ID, Access Token)
  12 Website Widget Customization (Color Theme, Position, Welcome Message)
- **Dashboard Actions**
  13 Create New Chatbot
  14 Edit Chatbot Configuration
  15 Activate/Deactivate Chatbot
  16 Filter Analytics Data

Number of user outputs:

1. **Dashboard Views**
   - Chatbot List Display
   - Analytics Overview (Conversation Count, Response Rate, Active Users)
   - Conversation History Logs
2. **Builder Interface**
   - Visual Flow Canvas Display
   - Node Property Panel
   - Connection Validation Messages
3. **Deployment Outputs**
   - WhatsApp Integration Status
   - Instagram Integration Status
   - Website Widget Embed Code
   - QR Code for WhatsApp

4. **Reports**
    - Daily Conversation Summary
    - User Engagement Metrics
    - Response Accuracy Report
    - Error Logs and Alerts

Number of user inquiries:

1. **Search Functions**
    - Search Chatbot by Name
    - Search Conversation History
    - Filter by Date Range
    - Filter by Platform
2. **Status Checks**
    - Check Integration Status
    - Verify AI Training Status
    - Test Chatbot Response
    - Preview Chatbot Behavior
3. **Analytics Queries**
    - View Conversation Details
    - Check User Statistics
    - Monitor Response Times

Number of files:

1. User Database (User Accounts, Authentication Data)
2. Chatbot Configuration Database (Flow Designs, Response Templates)
3. Conversation History Database (Message Logs, User Interactions)
4. Training Data Storage (Uploaded Documents, Knowledge Base)
5. Analytics Data Store (Metrics, Performance Data)
6. Integration Credentials Store (API Keys, Tokens)
7. Session Management (Active Sessions, Cache)2

Number of external interfaces:

1. AI API (Natural Language Processing Service)
2. WhatsApp Business API (Meta Platform Integration)
3. Instagram Messaging API (Meta Platform Integration)
4. Authentication Service (JWT Token Validation)

© University of Gujrat

Unadjusted Functional Point Table:

Table 1.1: Unadjusted Functional Point

Type of Components	**Complexity of Components**			
	Low	**Average**	**High**	**Total**
External Input	0 × 3 = 0	16 × 4 = 64	0 × 6 = 0	64
External Output	0 × 4 = 0	14 × 5 = 70	0 × 7 = 0	70
External Inquiries	0 × 3 = 0	11 × 4 = 44	0 × 6 = 0	44
Internal Logical Files	0 × 7 = 0	7 × 10 = 70	0 × 15 = 0	70
External Interface Files	0 × 5 = 0	0 × 7 = 0	4 × 10 = 40	40
Total				**288**

Value Adjustment Factor Table:

Table 1.2: Value Adjustment Factor.

Sr. No	**Value Adjustment Factor (VAF)**	**Value**
1	Data communications	5
2	Distributed data processing	4
3	Performance	4
4	Heavily used configuration	3
5	Transaction rate	4
6	On-Line data entry	5
7	End-user efficiency	5
8	On-Line update	5
9	Complex processing	4
10	Reusability	3
11	Installation ease	4
12	Operational ease	4
13	Multiple sites	5
14	Facilitate change	4
Total		**59**

Calculations:

Total Unadjusted Function Points (UFP) = **288**

Product Complexity Adjustment (PC) = 0.65 + (0.01 × 59) = 0.65 + 0.59 = **1.24** Total Adjusted Function Points (FP) = UFP × PC = 288 × 1.24 = **357.12**

Language Factor (LF) for JavaScript (React/Node.js) = **50 LOC** (Lines of Code per Function Point)

Source Lines of Code (SLOC) = FP × LF = 357.12 × 50 = **17,856 LOC**

For our Project:

Average Productivity = 3 × 8 = **24 FP/PM** (Function Points per Person-Month)

Labor Rate = **PKR 50,000/Month**

Total Estimated Effort = FP est. / Productivity
Total Estimated Effort = 357.12 / 24 = **14.88 Person-Months**

Cost / FP = Labor Rate / Productivity Parameter
Cost / FP = 50,000 / 24 = **PKR 2,083.33 per FP**

Total Project Cost = 357.12 × 2,083.33 = PKR 743,998.80

1.4.2 Project Cost Estimation by using COCOMO'81 (Constructive Cost Model)

Basic COCOMO

Type	**Effort**	**Schedule**
Organic	PM = 2.4 (KLOC)1.05	TD = 2.5 (PM)0.38
Semi-Detached	PM = 3.0 (KLOC)1.12	TD = 2.5 (PM)0.35
Embedded	PM = 2.4 (KLOC)1.20	TD = 2.5 (PM)0.32

PM = Person-Month (Effort)

KLOC = Line of Code, In Thousand

TD = Number of Months Estimated for Software Development (Duration) Our Project Falls under Semi-Detached:

Now we will put values in the given formulas:

Modes of COCOMO Model:

For BotFizz, we apply the **Semi-Detached** model as the project involves moderate complexity with some integration challenges and third-party API dependencies, but the team has reasonable experience with the technology stack.

Basic COCOMO - Semi-Detached Type:
Effort:
PM = 3.0 (KLOC)1.12

KLOC = 17.856 (from Function Point Analysis)

© University of Gujrat

PM = 3.0 (17.856)1.12
PM = 3.0 * 25.24 **PM = 75.72**
Person-Months

Schedule:
TD = 2.5 (PM)0.35
TD = 2.5 (75.72)0.35
TD = 2.5 * 4.55 **TD =**
11.37 Months

People Required:
People Required = Effort / Duration
People Required = 75.72 / 11.37 **People**
Required = 6.66 ≈ 7 Developers

Calculate Productivity:
Productivity = FP / Effort
Productivity = 357.12 / 75.72
Productivity = 4.7165 (FP per Person-Month)

Note: Since our actual team consists of 2 developers working over an extended timeline (200 days ≈ 6.67 months), the project will require proportionally more time with the smaller team size. The COCOMO calculation indicates the effort required; with 2 developers, we adjust the timeline accordingly while maintaining quality standards.

1.4.3 Activity Based Costing

Table 1.3: Activity Based Costing

S/No	Activity Name	Duration	Cost (PKR)
1	Requirements Analysis & Design	5 Weeks	30,000
2	User Authentication Module Development	2 Weeks	15,000
3	Database Design & Implementation	2 Weeks	15,000
4	Drag-and-Drop Builder Frontend	4 Weeks	35,000
5	AI API Integration	3 Weeks	30,000
6	WhatsApp Integration	2 Weeks	20,000
7	Instagram Integration	2 Weeks	20,000
8	Website Widget Development	2 Weeks	18,000
9	Dashboard & Analytics Development	3 Weeks	25,000
10	System Integration Testing	2 Weeks	15,000
11	User Acceptance Testing	1 Week	8,000
12	Deployment & Configuration	1 Week	10,000
13	Documentation & Training Materials	1 Week	9,000
Total	**30 Weeks (≈ 200 Days)**	**PKR 250,000**	

© University of Gujrat

Post-Deployment Operational Costs (Annual):

Table 1.4: Post-Deployment Operational Costs

Item	**Cost (PKR)**
Server Hosting (Ubuntu VPS)	40,000
Domain & SSL Certificates	5,000
API Usage (AI, WhatsApp, Instagram)	20,000
Maintenance & Updates	5,000
Backup & Security Services	5,000
Monitoring Tools	2500
Contingency Reserve	2500
Total Annual Cost	**PKR 80,000**

1.5 Task Dependency Table

Table 1.5: Task Dependency Table

Task	**Task Name**	**Dependency**
A	Requirements Analysis	None
B	System Architecture Design	A
C	Database Schema Design	A, B
D	UI/UX Design & Prototyping	A, B
E	User Authentication Module	C, D
F	Chatbot Builder Frontend	D, E
G	Backend API Development	C, E
H	AI API Integration	G
I	Drag-and-Drop Logic Implementation	F, G
J	WhatsApp Integration	G, H
K	Instagram Integration	G, H
L	Website Widget Development	F, G
M	Dashboard & Analytics	G, I
N	System Integration Testing	J, K, L, M
O	User Acceptance Testing	N
P	Deployment & Configuration	O
Q	Documentation	P

© University of Gujrat

1.6 CPM - Critical Path Method

1. Specify the Individual Activities

Table 1.6: Project Activities

Task	Task Name
A	Requirements Analysis
B	System Architecture Design
C	Database Schema Design
D	UI/UX Design & Prototyping
E	User Authentication Module
F	Chatbot Builder Frontend
G	Backend API Development
H	AI API Integration
I	Drag-and-Drop Logic Implementation
J	WhatsApp Integration
K	Instagram Integration
L	Website Widget Development
M	Dashboard & Analytics
N	System Integration Testing
O	User Acceptance Testing
P	Deployment & Configuration
Q	Documentation

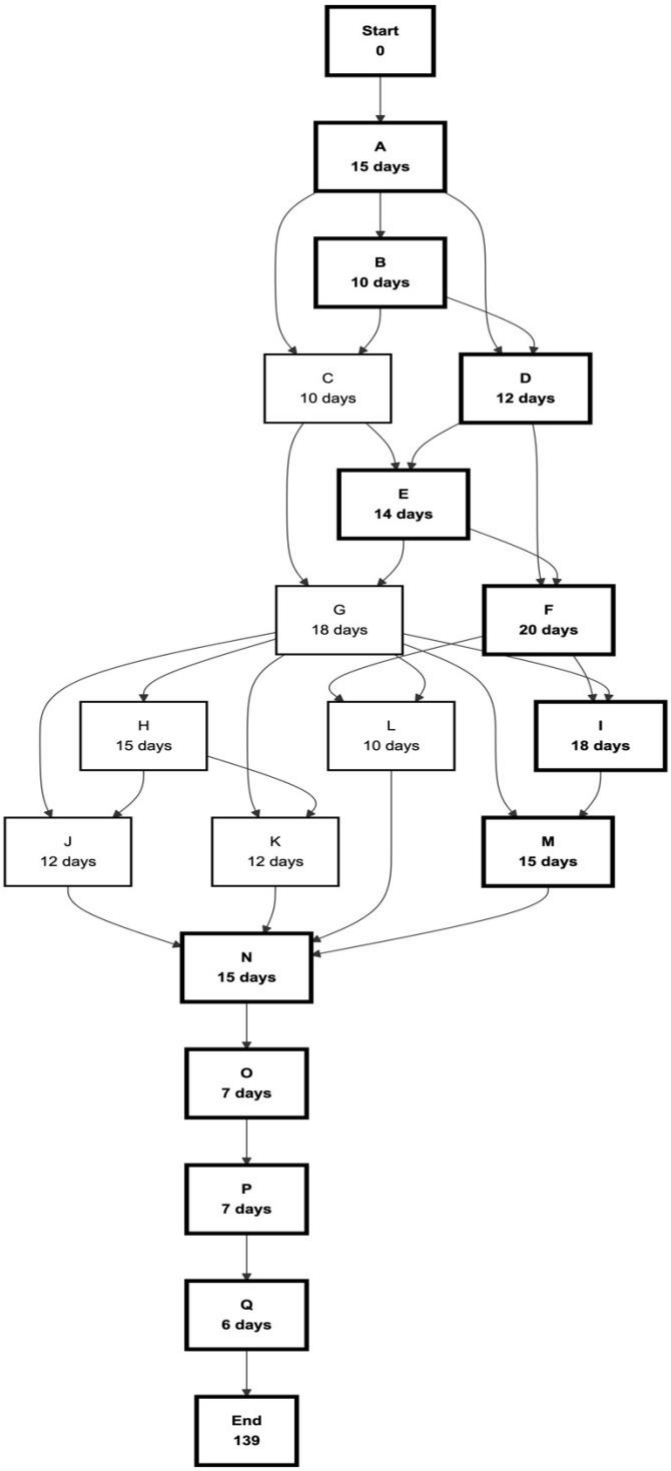
2. Determine the Sequence of the Activities

Table 1.7: Sequence of Activities

Activity	Predecessor	Duration (Days)
A	None	15
B	A	10
C	A, B	10
D	A, B	12
E	C, D	14
F	D, E	20
G	C, E	18
H	G	15
I	F, G	18
J	G, H	12
K	G, H	12
L	F, G	10
M	G, I	15
N	J, K, L, M	15
O	N	7
P	O	7
Q	P	6

3. Draw the Network Diagram.

Figure 1.1: Network Diagram



© University of Gujrat

4. Estimate Activity Completion Time

Table 1.8: Activity Completion Time

Activity	Duration (Days)	ES	EF	LS	LF	TS	FS
A	15	0	15	0	15	0	0
B	10	15	25	15	25	0	0
C	10	25	35	25	35	0	0
D	12	25	37	26	38	1	1
E	14	37	51	38	52	1	1
F	20	51	71	52	72	1	1
G	18	51	69	52	70	1	0
H	15	69	84	70	85	1	1
I	18	71	89	72	90	1	1
J	12	84	96	85	97	1	1
K	12	84	96	133	145	49	49
L	10	71	81	135	145	64	64
M	15	89	104	130	145	41	41
N	15	104	119	145	160	41	41
O	7	119	126	160	167	41	41
P	7	126	133	167	174	41	41
Q	6	133	139	174	180	41	41

Note: ES = Earliest Start, EF = Earliest Finish, LS = Latest Start, LF = Latest Finish, TS = Total Slack, FS = Free Slack

5. Identify the Critical Path

The critical path is the longest sequence of dependent tasks that determines the minimum total project duration. Any delay on a task along this path will delay the entire project.

Path Analysis:

The critical path is determined by whichever of these four predecessors takes the *longest* to finish. The critical path is determined through forward and backward pass calculations. Based on the network analysis:

Path 1: A → B → D → E → F → I → M → N → O → P → Q
Duration: 15 + 10 + 12 + 14 + 20 + 18 + 15 + 15 + 7 + 7 + 6 = 139 days

Path 2: A → B → C → E → G → H → J → N → O → P → Q
Duration: 15 + 10 + 10 + 14 + 18 + 15 + 12 + 15 + 7 + 7 + 6 = 129 days
Path 3: A → B → C → E → G → H → K → N → O → P → Q
Duration: 15 + 10 + 10 + 14 + 18 + 15 + 12 + 15 + 7 + 7 + 6 = 129 days

© University of Gujrat

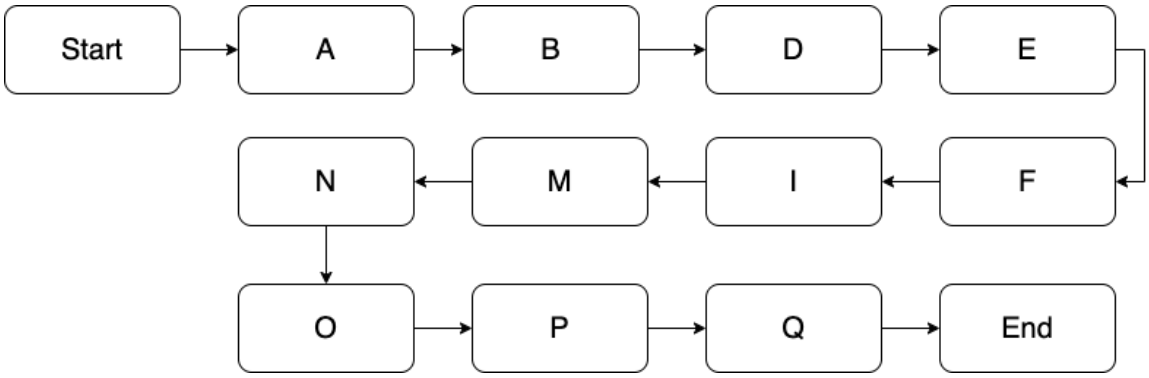
Path 4: A → B → D → E → F → L → N → O → P → Q
Duration: 15 + 10 + 12 + 14 + 20 + 10 + 15 + 7 + 7 + 6 = 116 days

Critical Path: Path 1 (139 days) is the longest and determines the minimum project duration.

Total Project Duration = 139 Days

5. Update CPM Diagram

Figure 1.2: Update CPM Diagram



The critical path is:

A → B → D → E → F → I → M → N → O → P → Q

(Path 1)

© University of Gujrat

1.7 Gantt chart

Table 1.9: Gantt Chart for BotFizz Project

Sr	Activity	Days	Week 1-4	Week 5-8	Week 9-12	Week 13-16	Week 17-20	Week 21-24	Week 25-28
A	Requirements Analysis	15							
B	System Architecture	10							
C	Database Schema	10							
D	UI/UX Design	12							
E	Authentication Module	14							
F	Builder Frontend	20							
G	Backend API	18							
H	AI Integration	15							
I	Drag-Drop Logic	18							
J	WhatsApp Integration	12							
K	Instagram Integration	12							
L	Widget Development	10							
M	Dashboard Analytics	15							
N	Integration Testing	15							
O	UAT	7							
P	Deployment	7							
Q	Documentation	6							

Timeline: Days 1-200 (Approximately 28 weeks ≈ 7 months)

© University of Gujrat

1.8 Introduction to Team member and their skill set

Table 1.10: Team Members, Skills set & Tasks

Name	**Registration #**	**Skill Set**	**Responsibilities**
M. Hanzala Zaheer	22014198-060	Backend Development (Node.js), Database Design (PostgreSQL), API Integration, Deployment	System architecture design, PostgreSQL database implementation, Node.js backend API development, WhatsApp/Instagram API integration, authentication implementation, server deployment, backend testing
M. Waleed Shams	22014198-079	Frontend Development (React.js), UI/UX Design, API Integration	Requirements gathering, UI/UX design, React.js development for chatbot builder and dashboard, React Flow drag-anddrop implementation, frontend testing, documentation

1.9 Task and Member Assignment Table

Table 1.11: Task Durations and Dependencies

Activity	Predecessor	Duration (Days)
A	None	15
B	A	10
C	A, B	10
D	A, B	12
E	C, D	14
F	D, E	20
G	C, E	18
H	G	15
I	F, G	18
J	G, H	12
K	G, H	12
L	F, G	10
M	G, I	15
N	J, K, L, M	15
O	N	7
P	O	7
Q	P	6

© University of Gujrat

M1 = M. Hanzala Zaheer (Backend Specialist)
M2 = M. Waleed Shams (Frontend Specialist)

Table 1.12: Task Assignment Table

Activity	**Assigned Members**	**Duration**
A: Requirements Analysis	M1, M2	15 Days
B: System Architecture Design	M1, M2	10 Days
C: Database Schema Design	M1	10 Days
D: UI/UX Design & Prototyping	M2	12 Days
E: User Authentication Module	M1	14 Days
F: Chatbot Builder Frontend	M2	20 Days
G: Backend API Development	M1	18 Days
H: AI API Integration	M1	15 Days
I: Drag-and-Drop Logic	M2	18 Days
J: WhatsApp Integration	M1	12 Days
K: Instagram Integration	M1	12 Days
L: Website Widget	M2	10 Days
M: Dashboard & Analytics	M1, M2	15 Days
N: Integration Testing	M1, M2	15 Days
O: User Acceptance Testing	M1, M2	7 Days
P: Deployment	M1	7 Days
Q: Documentation	M1, M2	6 Days

1.10 Tools and Technology with Reasoning

Category	Tool/Technology	Reasoning
Frontend	React.js	Component-based architecture for building dynamic, responsive user interfaces
	React Flow	Specialized library for node-based editors and drag-and-drop functionality
	Tailwind CSS	Utility-first CSS framework for rapid UI development
Backend	Node.js	Event-driven runtime for scalable serverside applications
	Express.js	Minimalist framework for building RESTful APIs
	PostgreSQL	Relational database with ACID compliance and scalability
AI Integration	AI API	Pre-trained language models for natural language processing
Third-Party APIs	WhatsApp Business API	Official API for WhatsApp chatbot integration
	Instagram Messaging API	Instagram platform integration for automated messaging
Development Tools	VS Code	Lightweight code editor with extensive extensions
	Git & GitHub	Version control and collaborative development
	Postman	API development and testing tool
Design Tools	Figma	UI/UX design and prototyping
	MS Planner	Project timeline and task management
	Draw.io	System architecture and database diagrams
Deployment	Ubuntu Server 22.04	Production server operating system
	Nginx	Web server and reverse proxy
	PM2	Process manager for Node.js applications

© University of Gujrat

1.11 Vision Document

Vision Document for BotFizz - AI Chatbot Builder Platform

1. Introduction

BotFizz is an AI-powered chatbot builder platform designed to democratize access to intelligent conversational automation, enabling businesses to create sophisticated chatbots without programming knowledge.

2. Vision Statement

BotFizz will revolutionize chatbot creation by providing a user-friendly platform where non-technical users can design, train, and deploy AI-powered chatbots across WhatsApp, Instagram, and websites, eliminating technical barriers and enabling businesses of all sizes to leverage artificial intelligence for customer engagement.

3. Objectives

- **Democratize Chatbot Creation**: Enable non-technical users to build sophisticated chatbots through visual drag-and-drop interfaces
- **Multi-Channel Deployment**: Provide seamless integration with WhatsApp, Instagram, and website platforms
- **Intelligent Conversations**: Integrate AI for natural language understanding and context-aware responses
- **Custom AI Training**: Allow users to upload business-specific content for domain-specialized chatbots
- **Comprehensive Analytics**: Deliver dashboard insights on conversation metrics and performance
- **User-Friendly Experience**: Minimize learning curve through intuitive interface design
- **Scalable Architecture**: Build on robust technology supporting concurrent users and high-volume interactions
- **Cost-Effective Solution**: Provide affordable alternative to custom chatbot development

4. Target Users

- Small to medium businesses seeking customer engagement automation
- Marketers managing social media customer interactions
- Customer support teams requiring 24/7 response capabilities
- E-commerce businesses automating order inquiries
- Content creators engaging with audiences
- Educational institutions providing automated information services

5. Key Features

- Visual drag-and-drop chatbot builder
- AI-powered natural language processing
- User data upload for AI training
- WhatsApp Business integration
- Instagram Messaging integration
- Embeddable website widgets
- User authentication and multi-user support

© University of Gujrat

- Real-time analytics dashboard
- Conversation history and logs
- Multi-language support capabilities

6. Success Criteria

- Functional chatbot builder with intuitive drag-and-drop interface
- Successful deployment on at least two platforms (WhatsApp/Instagram/Website)
- AI-powered conversation capabilities with contextual understanding
- User ability to train chatbots with custom data
- Dashboard displaying real-time analytics and metrics
- Positive user feedback on ease of use and functionality
- Stable performance under concurrent user load
- Secure data handling and authentication

7. Assumptions

- Stable internet connectivity for cloud-based operations
- API availability from third-party providers (AI, WhatsApp, Instagram)
- Users have basic computer literacy and familiarity with web applications
- Businesses have valid accounts on platforms they wish to integrate
- Server infrastructure remains available and operational

8. Risks

- Third-party API rate limits affecting chatbot performance
- Integration challenges with Meta platform policies and requirements
- AI API costs scaling with increased usage
- Competition from established chatbot platforms
- Technical complexity in maintaining multiple platform integrations

9. Constraints

- Development timeline limited to 200 days
- Two-member development team capacity
- Budget constraints for API usage during development
- Third-party platform policy compliance requirements
- Academic project scope and resource limitation

© University of Gujrat

1.12 Risk List

Table 1.13: Project Risks

Risk Category	Risk Description	Impact	Mitigation Strategy
Technical	Third-party API rate limits or service downtime	High	Implement caching, queue systems, and fallback mechanisms
Technical	Integration complexity with WhatsApp/Instagram APIs	Medium	Thorough API documentation review and early prototyping
Technical	AI API costs exceeding budget expectations	Medium	Monitor usage closely, implement usage caps and optimization
Schedule	Development delays due to unforeseen technical challenges	Medium	Buffer time included in timeline, agile approach for adjustments
Operational	Platform policy changes by Meta affecting integrations	High	Regular monitoring of API updates, flexible architecture design
Security	Data breach or unauthorized access to user information	High	Implement encryption, secure authentication, regular security audits
User Adoption	Users finding interface complex or confusing	Medium	User testing, iterative design improvements, comprehensive tutorials
Resource	Team member availability or technical skill gaps	Medium	Cross-training, documentation, task distribution flexibility

1.13 Product Features / Product Decomposition

Core Features:

1. **User Authentication System**
    - Secure registration and login
    - Password recovery mechanism
    - Session management
2. **Chatbot Builder Interface**
    - Drag-and-drop conversation flow design
    - Node-based editor for responses
    - Conditional logic configuration

3. **AI Integration Module**
    - Natural language processing
    - Intelligent response generation
    - Custom data upload for training
4. **Multi-Channel Deployment**
    - WhatsApp Business integration
    - Instagram Messaging integration
    - Website widget embedding
5. **Management Dashboard**
    - Chatbot list and status overview
    - Real-time analytics and metrics
    - Conversation history logs

© University of Gujrat

Chapter 2: Software Requirement Specification (For Object Oriented Approach)

2.1 Introduction:

The requirements engineering process provides mechanisms to understand what the customer wants, analyze needs, assess feasibility, negotiate solutions, specify these solutions clearly, validate the specifications, and manage the requirements as they evolve into an operational system. This process ensures the development of high-quality systems that satisfy user needs through structured capturing and representation of requirements. (i.e. are of good quality).

- Requirements elicitation
- Requirements analysis and negotiation
- Requirements specification
- System modeling
- Requirements validation
- Requirements management

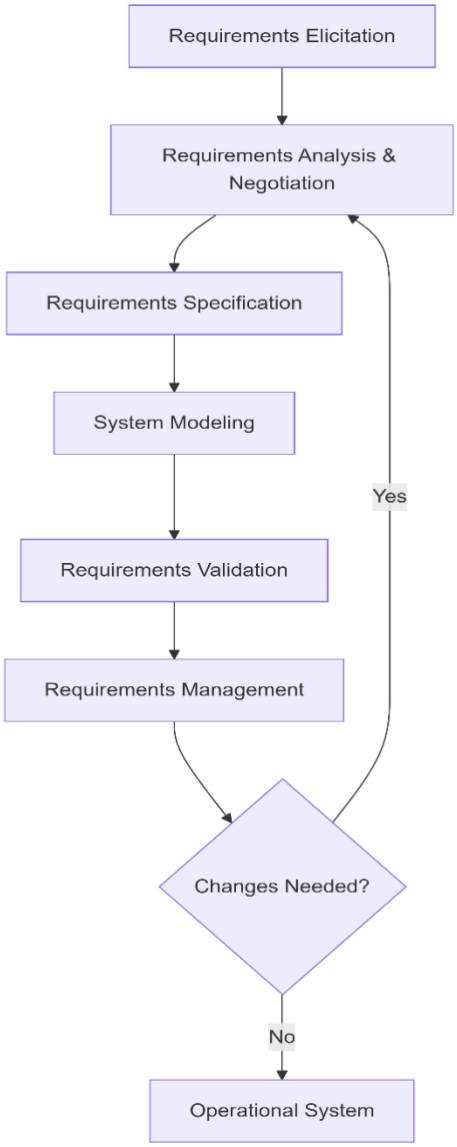


Figure 2.1: System Specification

© University of Gujrat

2.1.1 Systems Specifications

The following are the clauses that must be included while describing the system specifications.

Introduction

BotFizz is a web-based AI chatbot builder platform designed to democratize access to intelligent conversational automation. The platform enables businesses to create sophisticated chatbots without programming knowledge, providing a visual flow builder, AI intent recognition, and integration with WhatsApp, Instagram, and website widgets. The goal is to make multi-channel chatbot deployment simple, fast, and affordable for small and medium businesses.

Existing System

Current chatbot platforms such as ManyChat, Chatfuel, and Dialogflow have significant limitations for small and medium-sized businesses. ManyChat is limited to Messenger, Chatfuel offers rigid templates, and Dialogflow requires technical setup. Overall, most lack true multi-channel integration, easy no-code customization, and affordability.

© University of Gujrat

Organizational Chart

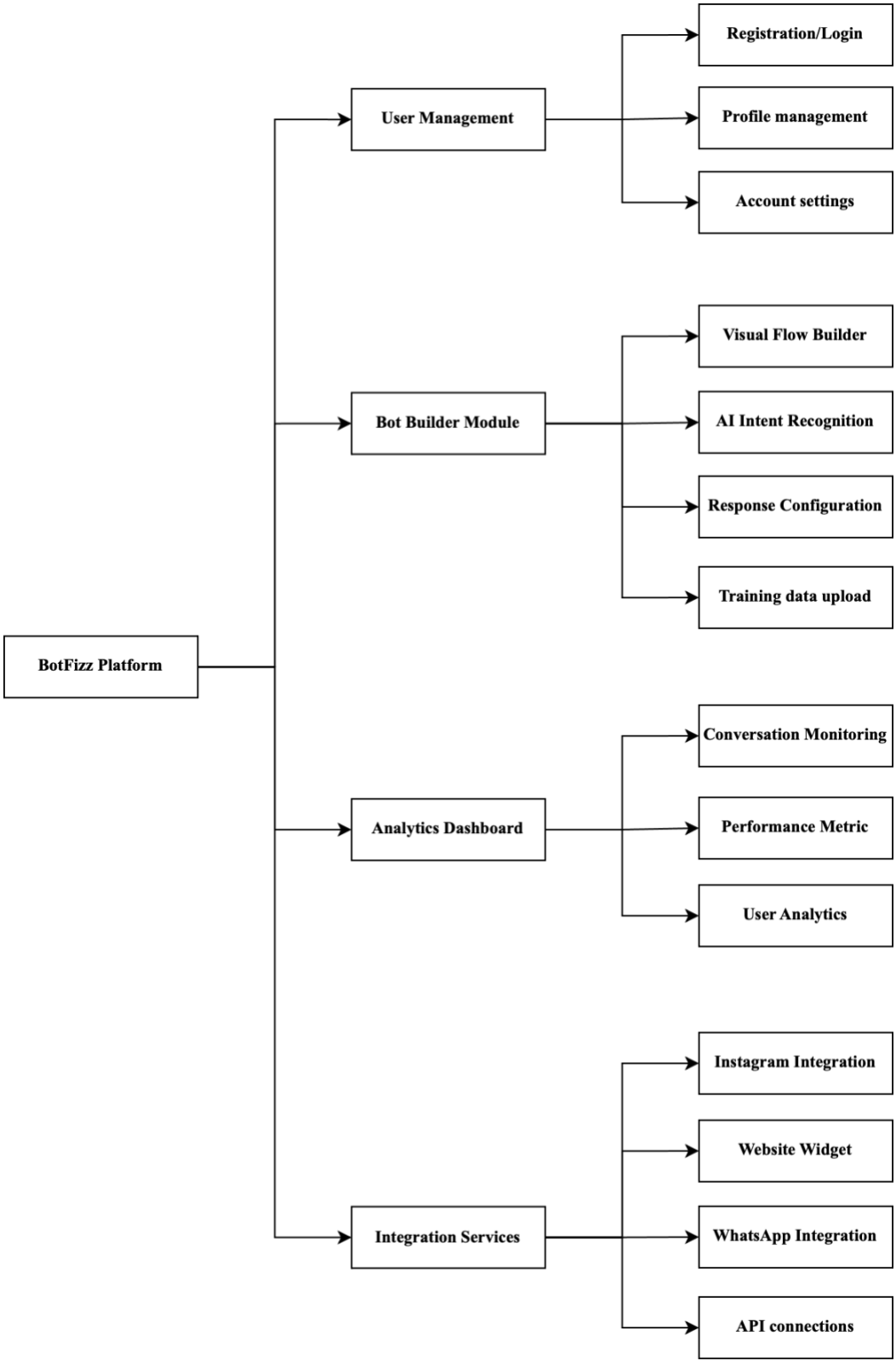


Figure 2.2: Organizational Chart

Scope of the System

The BotFizz platform scope is divided into phases to ensure systematic development and delivery.

Phase I (Current Scope)

Phase I includes the following core business areas:

User Account Management:

- User registration and authentication
- Profile creation and updates
- Password recovery
- Session management

Chatbot Builder Interface:

- Visual flow designer with drag-and-drop functionality
- Node-based conversation design
- Conditional logic configuration
- Response message templates
- AI settings configuration

Integration Services:

- WhatsApp Business API integration
- Instagram Messaging API integration
- Website widget embedding
- API key management

Basic Dashboard:

- Chatbot list and status overview
- Conversation history logs
- Active bot monitoring
- Basic analytics metrics

Phase II (Future Enhancement)

Phase II will expand functionality to include:

Advanced Analytics:

- Detailed conversation analysis
- User engagement metrics
- Response accuracy tracking
- Performance dashboards
- Export and reporting tools
-

Phase III (Long-term Vision)

Phase III covers enterprise-level features:

- Native apps
- Email automation
- E-commerce platforms
- Automated billing system

Summary of Requirements: (Initial Requirements)

Functional Requirements:

1: User Management

- Users can register using a valid email and password.
- The system must store passwords securely (encrypted).
- Users can recover forgotten passwords via email verification.
- The system must maintain secure user sessions until logout or timeout.
- Users can update their profile details (name, email, organization).
- The system must support multiple user accounts under one organization.

2: Chatbot Builder

- Users can create new chatbot projects with unique IDs.
- A visual drag-and-drop canvas must be available for conversation flow design.
- Users can add node types:
    - Message nodes
    - Question nodes
    - Condition nodes
    - Action nodes (API/data operations)
- Users can configure AI intent recognition.
- Users can upload training data (documents, FAQs).
- The system must validate flow logic before deployment.
- Users can save drafts and versions of chatbot configurations.
- Users can preview/test bot conversations.

3: Integrations

- Users can connect WhatsApp Business via Meta APIs.
- The system must generate and store WhatsApp API credentials securely.
- Users can connect Instagram business messaging.
- The system must provide embeddable website widget code with customization options.
- Users can test integrations before activation.
- The system must display integration status (connected/disconnected/error).
- Users can disconnect or reconnect integrations anytime.

4: AI & Training

- The system must integrate with external AI APIs for intent detection.
- Users can upload documents for training.
- The system must process uploaded files for domain understanding.
- Users can set AI confidence thresholds.

© University of Gujrat

- The system must provide fallback responses when confidence is low.

5: Dashboard & Analytics

- Users can view all created chatbots with status indicators.
- The system must show conversation history with timestamps and user details.
- Users can filter conversation logs by date, platform, and chatbot.
- The system must provide basic metrics (total conversations, active bots, response rate, avg. response time).
- Users can activate or deactivate chatbots.
- The system must display error logs and alerts.

6: Deployment

- Users can deploy chatbots to selected channels with one click.
- The system must generate WhatsApp QR codes for chatbot access.
- The system must show real-time deployment status.
- Users can update deployed bots and push changes live.

Non-Functional Requirements:

1: Security

- All passwords must be encrypted.
- API keys and integration credentials must be stored securely.
- System must use secure communication (HTTPS, TLS).

2: Performance

- Dashboard data must load within acceptable time.
- Bot responses in preview mode must be near real-time.

3: Reliability & Availability

- The system should provide high uptime.
- Auto-recovery mechanisms must handle failed integrations.

4: Scalability

- The system must support increasing numbers of users, bots, and messages.
- AI processing should scale with large training datasets.

5: Usability

- The interface must be intuitive for beginners (visual builder, tooltips).
- Navigation should be simple and consistent across modules.

6: Maintainability

- The system must allow easy updates to chatbot logic and integrations.
- Codebase should follow modular architecture.

7: Compliance

- Must comply with privacy and data protection regulations.
- Logging must not store sensitive user data unnecessarily.

© University of Gujrat

2.1.2 Identifying External Entities

The identification of external entities is based on the information contained in the system requirements. This identification is performed in two phases:

Phase A: Over-Specify Entities from Abstract

Based on the system description, the following entities are identified:

- **User** - Individual creating and managing chatbots
- **Chatbot** - AI-powered conversation agent
- **Conversation Flow** - Visual representation of bot logic
- **Node** - Individual component in conversation flow
- **Message** - Communication between bot and end-user
- **WhatsApp Platform** - Meta's messaging service
- **Instagram Platform** - Meta's messaging service
- **Website Widget** - Embeddable chat interface
- **AI Service** - External intent recognition API
- **Training Data** - Documents/content for AI improvement
- **Dashboard** - Analytics and monitoring interface
- **Integration** - Connection to messaging platforms
- **Session** - Active user login period
- **Analytics** - Performance metrics and reports

Phase B: Perform Refinement

After analyzing business logic, the refined external entities are:

Primary Entities:

1. **User** - Platform account holder who builds chatbots
2. **Chatbot** - Configured conversation agent
3. **Integration** - WhatsApp, Instagram, or Website connection
4. **Conversation** - Interaction between bot and end-user
5. **AI Service** - External API for intent recognition

© University of Gujrat

2.1.3 Context Level Data Flow Diagram:

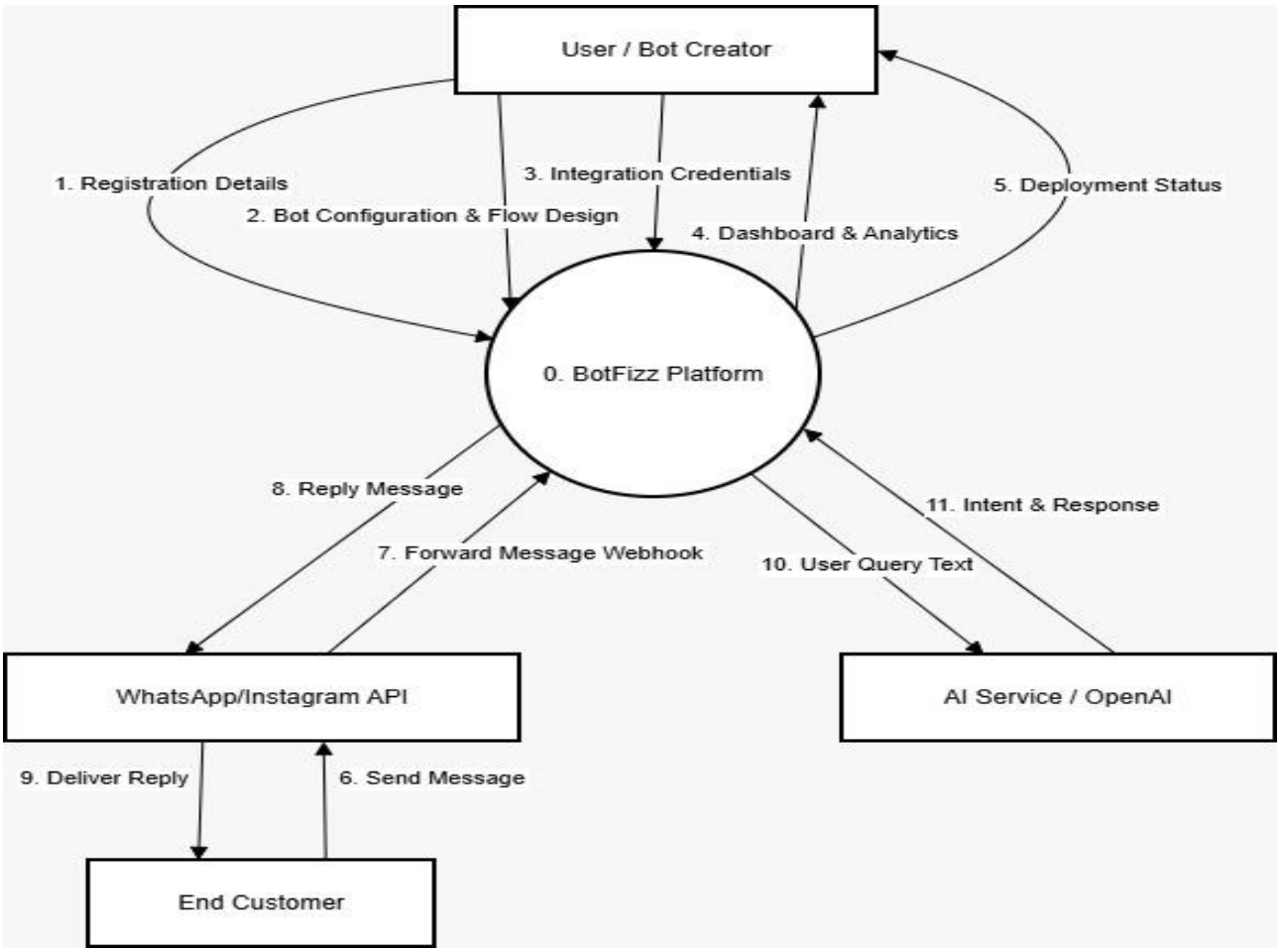


Figure 2.3: Context Level Data Flow Diagram

2.1.4 Capture "shall" Statements:

Functional requirements expressed using "shall" statements:

Table 2.1: Functional requirements

Sr#	Initial Requirement
1	The system **shall** allow users to register with valid email and password
2	The system **shall** authenticate users securely before granting access
3	The system **shall** provide password recovery through email verification
4	The system **shall** maintain secure user sessions until logout
5	The system **shall** allow users to create new chatbot projects
6	The system **shall** provide a visual drag-and-drop flow builder
7	The system **shall** allow users to add message, question, condition, and action nodes
8	The system **shall** enable users to configure AI intent recognition settings

© University of Gujrat

9	The system **shall** allow users to upload training data documents
10	The system **shall** validate conversation flows before deployment
11	The system **shall** enable WhatsApp Business API integration
12	The system **shall** enable Instagram Messaging API integration
13	The system **shall** generate embeddable website widget code
14	The system **shall** process messages from connected channels in real-time
15	The system **shall** use external AI API for intent recognition
16	The system **shall** provide fallback responses when AI confidence is low
17	The system **shall** display conversation history with filters
18	The system **shall** show basic analytics metrics on dashboard
19	The system **shall** allow users to activate/deactivate chatbots
20	The system **shall** deploy chatbots to selected channels
21	The system **shall** generate QR codes for WhatsApp access
22	The system **shall** update deployed chatbots with configuration changes
23	The system **shall** log all conversations for analytics
24	The system **shall** display integration status indicators
25	The system **shall** secure API credentials and tokens

2.1.5 Allocate Requirements:

Requirements are allocated to specific use cases:

Table 2.2: Allocate requirements

Sr#	Initial Requirement	UC-Name
1	The system shall allow users to register	UC-Register-User
2	The system shall authenticate users securely	UC-Login-User
3	The system shall provide password recovery	UC-Recover-Password
4	The system shall maintain secure sessions	UC-Manage-Session
5	The system shall allow creating chatbot projects	UC-Create-Chatbot
6	The system shall provide visual flow builder	UC-Build-Flow
7	The system shall allow adding flow nodes	UC-Add-Node
8	The system shall enable AI intent configuration	UC-Configure-AI
9	The system shall allow uploading training data	UC-Upload-Training-Data
10	The system shall validate conversation flows	UC-Validate-Flow
11	The system shall enable WhatsApp integration	UC-Connect-WhatsApp
12	The system shall enable Instagram integration	UC-Connect-Instagram
13	The system shall generate website widget code	UC-Generate-Widget
14	The system shall process messages in real-time	UC-Process-Message
15	The system shall use AI for intent recognition	UC-Recognize-Intent
16	The system shall provide fallback responses	UC-Provide-Fallback
17	The system shall display conversation history	UC-View-Conversations
18	The system shall show analytics metrics	UC-View-Analytics
19	The system shall activate/deactivate chatbots	UC-Manage-Chatbot-Status
20	The system shall deploy chatbots	UC-Deploy-Chatbot
21	The system shall generate QR codes	UC-Generate-QR-Code
22	The system shall update deployed chatbots	UC-Update-Deployed-Bot
23	The system shall log conversations	UC-Log-Conversation
24	The system shall display integration status	UC-View-Integration-Status
25	The system shall secure API credentials	UC-Secure-Credentials

2.1.6 Prioritize Requirements

Requirements prioritized by importance for Phase I implementation:

Table 2.3: Priorities requirement

Sr#	Rank	Initial Requirement	UC-ID	UC-Name
1	Highest	System shall allow user registration	UC-1	UC-Register-User
2	Highest	System shall authenticate users	UC-2	UC-Login-User
3	Highest	System shall allow creating chatbots	UC-5	UC-Create-Chatbot
4	Highest	System shall provide visual flow builder	UC-6	UC-Build-Flow
5	Highest	System shall allow adding nodes	UC-7	UC-Add-Node
6	Highest	System shall process messages real-time	UC-14	UC-Process-Message
7	Highest	System shall enable WhatsApp integration	UC-11	UC-Connect-WhatsApp
8	Highest	System shall deploy chatbots	UC-20	UC-Deploy-Chatbot
9	Medium	System shall provide password recovery	UC-3	UC-Recover-Password
10	Medium	System shall enable AI configuration	UC-8	UC-Configure-AI
11	Medium	System shall upload training data	UC-9	UC-Upload-Training-Data
12	Medium	System shall validate flows	UC-10	UC-Validate-Flow
13	Medium	System shall enable Instagram integration	UC-12	UC-Connect-Instagram
14	Medium	System shall generate widget code	UC-13	UC-Generate-Widget
15	Medium	System shall use AI for intents	UC-15	UC-Recognize-Intent
16	Medium	System shall display conversations	UC-17	UC-View-Conversations
17	Medium	System shall show analytics	UC-18	UC-View-Analytics
18	Medium	System shall manage bot status	UC-19	UC-Manage-Chatbot-Status
19	Medium	System shall generate QR codes	UC-21	UC-Generate-QR-Code
20	Low	System shall maintain sessions	UC-4	UC-Manage-Session
21	Low	System shall provide fallbacks	UC-16	UC-Provide-Fallback
22	Low	System shall update deployed bots	UC-22	UC-Update-Deployed-Bot
23	Low	System shall log conversations	UC-23	UC-Log-Conversation
24	Low	System shall display integration status	UC-24	UC-View-Integration-Status
25	Low	System shall secure credentials	UC-25	UC-Secure-Credentials

© University of Gujrat

2.1.7 Requirements Traceability Matrix

The Requirements Traceability Matrix ensures each requirement is tracked from inception to implementation:

Table 2.4: Traceability Matrix

Sr#	System Specification	UC-Name	Category
1	System shall allow user registration	UC-Register-User	Business
2	System shall authenticate users	UC-Login-User	Business
3	System shall provide password recovery	UC-Recover-Password	Business
4	System shall maintain secure sessions	UC-Manage-Session	Technical
5	System shall allow creating chatbots	UC-Create-Chatbot	Business
6	System shall provide visual flow builder	UC-Build-Flow	Business
7	System shall allow adding nodes	UC-Add-Node	Business
8	System shall enable AI configuration	UC-Configure-AI	Business
9	System shall upload training data	UC-Upload-Training-Data	Business
10	System shall validate flows	UC-Validate-Flow	Technical
11	System shall enable WhatsApp integration	UC-Connect-WhatsApp	Business
12	System shall enable Instagram integration	UC-Connect-Instagram	Business
13	System shall generate widget code	UC-Generate-Widget	Business
14	System shall process messages real-time	UC-Process-Message	Technical
15	System shall use AI for intents	UC-Recognize-Intent	Technical
16	System shall provide fallbacks	UC-Provide-Fallback	Technical
17	System shall display conversations	UC-View-Conversations	Business
18	System shall show analytics	UC-View-Analytics	Business
19	System shall manage bot status	UC-Manage-Chatbot-Status	Business
20	System shall deploy chatbots	UC-Deploy-Chatbot	Business
21	System shall generate QR codes	UC-Generate-QR-Code	Business
22	System shall update deployed bots	UC-Update-Deployed-Bot	Technical
23	System shall log conversations	UC-Log-Conversation	Technical
24	System shall display integration status	UC-View-Integration-Status	Business
25	System shall secure credentials	UC-Secure-Credentials	Security

2.1.8 High Level Usecase Diagram:

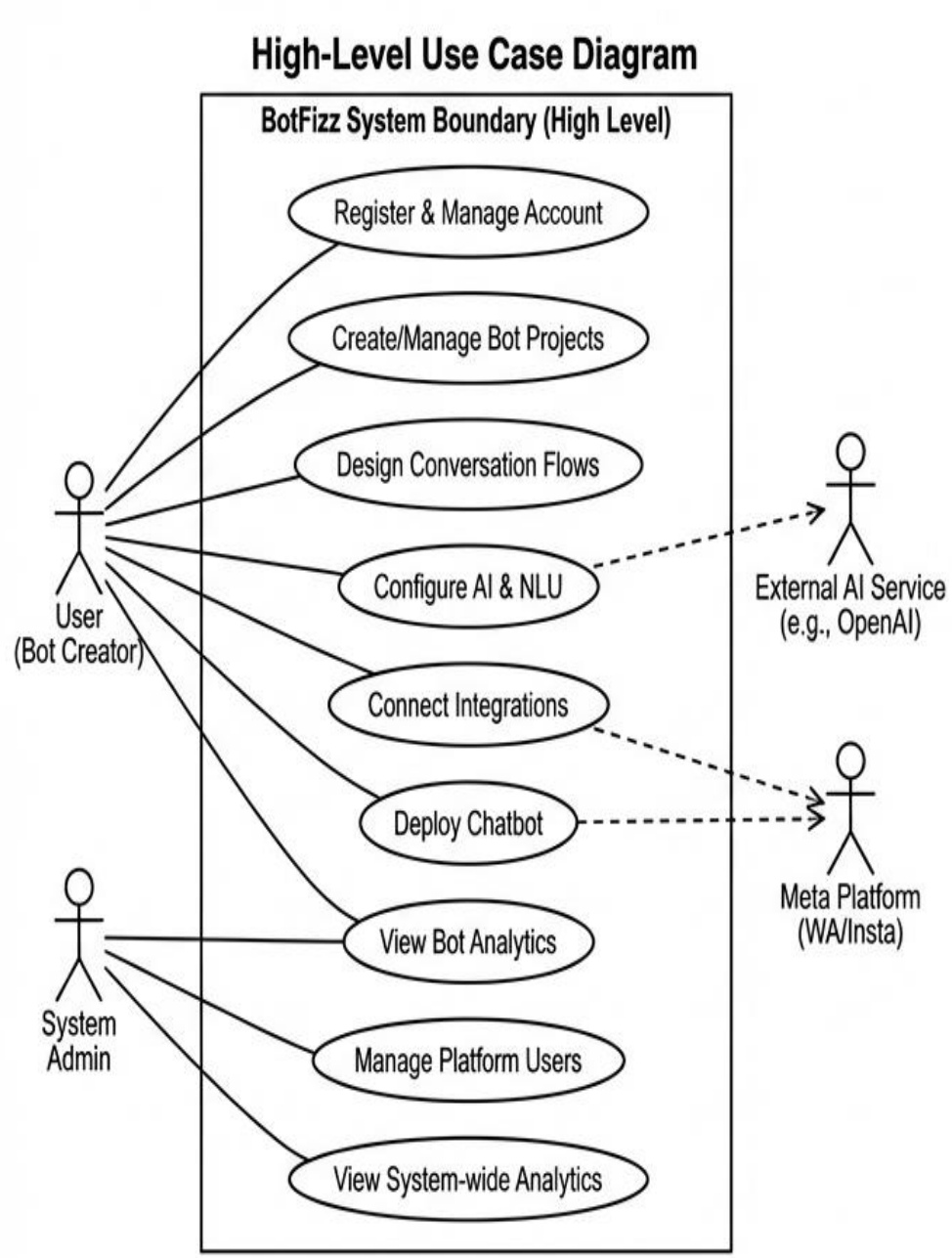


Figure 2.3: High Level Usecase

2.1.9 Analysis Level Usecase Diagram:

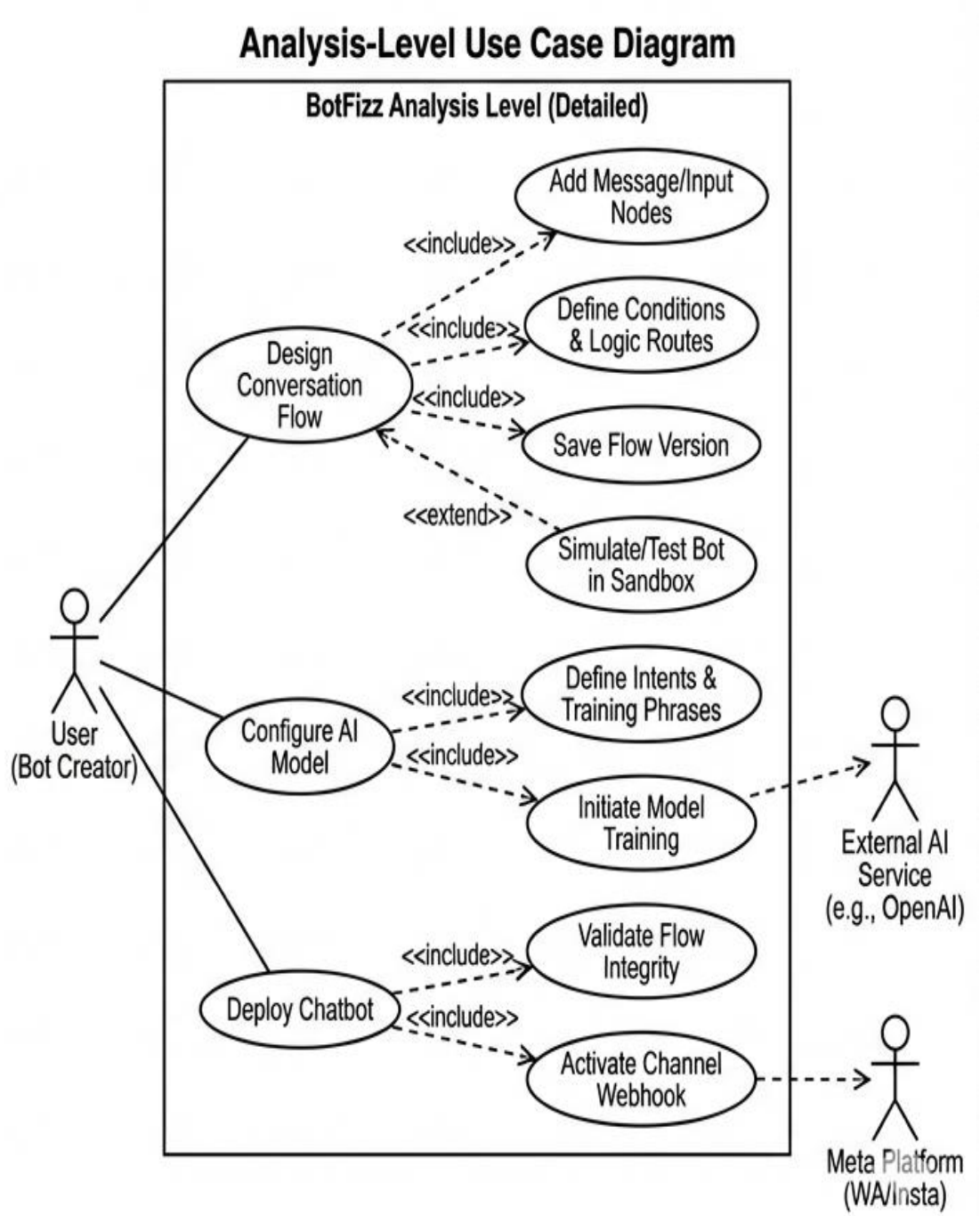


Figure 2.4: Analysis Level Usecase

2.1.10 Use case Description

Use case 1: Register user

Use Case Name: UC-Register-User	**Use Case ID:** UC-1
Actors: User	**Created By:** M. Hanzala Zaheer, M. Waleed Shams
Date: November 2025	
Description: This use case explains how a new user registers on the website by submitting their details. The system validates the information, creates the account, and activates it after email verification.	
Pre-Condition: User has access to the BotFizz website.	
Basic flow:	

Actor's Action	System's Response
User navigates to registration page	System displays registration form
User enters email, full name, organization, password	System validates input format
User submits registration form	System checks if email already exists
System creates user account	
System sends verification email	
User puts the OTP received via mail	System activates account
System redirects to login page	

Alternative flow: If the user enters invalid details, uses an existing email, or clicks an invalid verification link, the system displays an error and prompts for correction or link resend.
Post Condition: User account created and ready for login after verification.

Use case 2: Login User

Use Case Name: UC- Login User	**Use Case ID:** UC-2
Actors: User	**Created By:** M. Hanzala Zaheer, M. Waleed Shams
Date: November 2025	
Description: This use case explains how a verified user logs into the system by entering valid credentials, after which the system authenticates the user and grants access to the dashboard.	
Pre-Condition: User has a verified account.	
Basic flow:	

Actor's Action	System's Response
User navigates to login page	System displays login form
User enters email and password	System validates credentials
User clicks login button	System authenticates user
System creates session token	
System redirects to dashboard	

Alternative flow: If the user enters incorrect credentials or an unverified email, the system shows an error and prompts the user to retry.
Post Condition: User session is active and dashboard is accessible.

Use case 3: Create Chatbot

Use Case Name: UC- Create Chatbot	**Use Case ID:** UC-5
Actors: User	**Created By:** M. Hanzala Zaheer, M. Waleed Shams
Date: November 2025	
Description: This use case explains how a logged-in user creates a new chatbot by entering basic details, after which the system generates a unique chatbot project and redirects the user to the flow builder.	
Pre-Condition: User is logged in to BotFizz platform.	
Basic flow:	

Actor's Action	System's Response
User clicks "Create New Chatbot"	System displays chatbot creation form
User enters chatbot name and description	System validates input
User selects primary language	System prepares default configuration
User clicks create button	System creates chatbot project
System generates unique chatbot ID	
System redirects to flow builder	

Alternative flow: If the user enters invalid or incomplete details, the system shows an error and requests corrections before creating the chatbot.
Post Condition: Chatbot exists in draft state, ready for flow design.

Use case 4: Build Flow

Use Case Name: UC- Build flow	**Use Case ID:** UC-6
Actors: User	**Created By:** M. Hanzala Zaheer, M. Waleed Shams
Date: November 2025	
Description: This use case describes how a user builds a chatbot conversation flow by adding, configuring, and connecting nodes, after which the system saves the flow for testing.	
Pre-Condition: User has created a chatbot and is in flow builder.	
Basic flow:	

Actor's Action	System's Response
User views blank canvas	System displays node palette and tools
User drags message node to canvas	System creates node instance
User double-clicks node	System opens node configuration panel

User enters message text	System updates node content
User drags question node to canvas	System creates input collection node
User connects nodes with arrows	System establishes conversation flow
User adds condition node for branching	System enables logic configuration
User saves flow	System validates and stores configuration
Alternative flow: If the user configures nodes incorrectly or leaves required fields empty, the system shows an error and prevents saving until corrected.	
Post Condition: Chatbot has configured conversation flow ready for testing.	

Use case 5: Configure AI

Use Case Name: UC- Configure AI	**Use Case ID:** UC-8
Actors: User	**Created By:** M. Hanzala Zaheer, M. Waleed Shams
Date: November 2025	
Description: This use case describes how a user configures AI for a chatbot by setting intent recognition, training phrases, and fallback responses, after which the system enables AI-driven understanding.	
Pre-Condition: User has a chatbot with conversation flow.	
Basic flow:	
Actor's Action	**System's Response**
User navigates to AI settings	System displays AI configuration panel
User enables intent recognition	System shows confidence threshold slider
User sets confidence threshold (0-100%)	System saves threshold value
User adds sample phrases for intents	System processes training examples
User configures fallback response	System stores default response
User tests AI understanding	System demonstrates intent matching
User saves AI configuration	System activates AI for chatbot
Alternative flow: If the user enters invalid training data or skips required settings, the system shows an error and prevents activation until corrected.	
Post Condition: Chatbot uses AI for natural language understanding.	

Use case 6: Connect WhatsApp

Use Case Name: UC- Connect WhatsApp	**Use Case ID:** UC-11
Actors: User	**Created By:** M. Hanzala Zaheer, M. Waleed Shams
Date: November 2025	
Description: This use case explains how a user connects their chatbot to WhatsApp by providing required business credentials, after which the system verifies and establishes the integration.	
Pre-Condition: User has WhatsApp Business account and verified chatbot.	
Basic flow:	
Actor's Action	**System's Response**
User selects WhatsApp integration	System displays connection wizard

User enters Business Account ID	System validates account
User enters API key and phone number	System tests connection
System prompts for webhook configuration	User confirms webhook URL
User submits integration	System establishes connection
System displays success message	
System shows connected status	
Alternative flow: If the user enters invalid credentials or fails connection tests, the system shows an error and stops the integration until corrected.	
Post Condition: Chatbot receives and responds to WhatsApp messages.	

Use case 7: Deploy Chatbot

Use Case Name: UC- Deploy Chabot	**Use Case ID:** UC-20
Actors: User	**Created By:** M. Hanzala Zaheer, M. Waleed Shams
Date: November 2025	
Description: This use case explains how a user deploys a fully configured chatbot, after which the system activates it across selected channels and provides access methods like QR codes and embed codes.	
Pre-Condition: Chatbot is configured with flow and integrations.	
Basic flow:	
Actor's Action	**System's Response**
User clicks "Deploy" button	System validates chatbot configuration
System checks integration connections	
User reviews deployment summary	System displays platforms and settings
User confirms deployment	System begins deployment process
System activates chatbot on channels	
System generates access methods	
System displays QR code for WhatsApp	
System provides widget embed code	
User tests deployed chatbot	System confirms bot is responding
Alternative flow: If configuration or integrations are incomplete, the system shows an error and prevents deployment until all requirements are fixed.	
Post Condition: End users can interact with chatbot on deployed channels.	

Use case 8: View Conversations

Use Case Name: UC- View Conversations	**Use Case ID:** UC-16
Actors: User	**Created By:** M. Hanzala Zaheer, M. Waleed Shams
Date: November 2025	
Description: This use case explains how a user views and filters chatbot conversations, allowing them to inspect full message threads and analyze interaction history.	
Pre-Condition: User has deployed chatbot with active conversations.	
Basic flow:	
Actor's Action	**System's Response**
User navigates to dashboard	System displays conversation overview

User selects chatbot	System shows conversation list
User applies date filter	System filters by selected date range
User applies platform filter	System shows selected channel only
User clicks on conversation	System displays full conversation thread
System shows timestamps and user info	
User searches for keyword	System highlights matching conversations
Alternative flow: If no conversations match the selected filters or search keywords, the system shows an empty result and prompts the user to adjust filters.	
Post Condition: User understands chatbot performance and user interactions.	

Use case 9: View Analytics

Use Case Name: UC- View Analytics	**Use Case ID:** UC-17
Actors: User	**Created By:** M. Hanzala Zaheer, M. Waleed Shams
Date: November 2025	
Description: This use case explains how a user accesses and analyzes chatbot performance metrics through the analytics dashboard, with options to filter data and view detailed reports.	
Pre-Condition: User has deployed chatbot with conversation data.	
Basic flow:	
Actor's Action	**System's Response**
User opens analytics dashboard	System displays key metrics
System shows total conversations	
System shows active chatbots count	
System shows response rate percentage	
System shows average response time	
User selects date range	System updates metrics for period
User views platform breakdown	System shows distribution by channel
User clicks on metric	System displays detailed breakdown
User exports report	System generates downloadable file
Alternative flow: If analytics data is unavailable or filters return no results, the system shows an empty or partial dataset and prompts the user to adjust selections.	
Post Condition: User has insights for optimizing chatbot effectiveness.	

© University of Gujrat

Chapter 3: Design Document (For Object Oriented Approach)

3.1 Introduction

The third deliverable focuses on the software design of the **BotFizz** platform. In the previous deliverable (Chapter 2), the analysis of the system was completed, allowing us to understand the current situation of the problem domain and the specific requirements for the chatbot builder.

Now, we strive for a solution to the problem domain by using an object-oriented approach. This phase bridges the gap between the requirements analysis and the final implementation. The following artifacts are included in this design deliverable:

- **Domain Model:** Illustrating meaningful conceptual classes in the real-world problem domain.
- **System Sequence Diagram:** Defining system events and operations.
- **Sequence Diagram:** Depicting the sequence of actions and invocation of methods.
- **Collaboration Diagram:** showing object interactions.
- **Operation Contracts:** Identifying system state changes.
- **Design Class Diagram:** Describing the classes that perform the work.
- **State Chart Diagram:** Describing the states of objects.
- **Data Model:** Describing the logical and physical representation of persistent data.

3.2 Domain Model

The Domain Model represents the set of requirements common to the system. It provides a "big picture" of the interrelationships among business entities within the **BotFizz** environment. It focuses on the product and deliverables important to the business domain, illustrating meaningful conceptual classes and the associations between them.

For **BotFizz**, the domain model captures the relationship between the User (the bot creator), the Chatbot project itself, the building blocks of the conversation (Flows and Nodes), the integrations (WhatsApp/Instagram), and the analytics data generated by End Customers.

Key Domain Objects identified for BotFizz:

1. **User:** The registered account holder managing the bots.
2. **ChatbotProject:** The core entity representing a specific bot instance.
3. **ConversationFlow:** The visual logic structure of the bot.
4. **Node:** The individual components of a flow (Message, Question, Logic).
5. **Integration:** The connection to external platforms (WhatsApp, Instagram).
6. **AI_Config:** The settings for intent recognition and training data.
7. **EndCustomer:** The individual interacting with the deployed bot.
8. **Conversation:** The session history between a bot and a customer.

© University of Gujrat

Domain Model Diagram

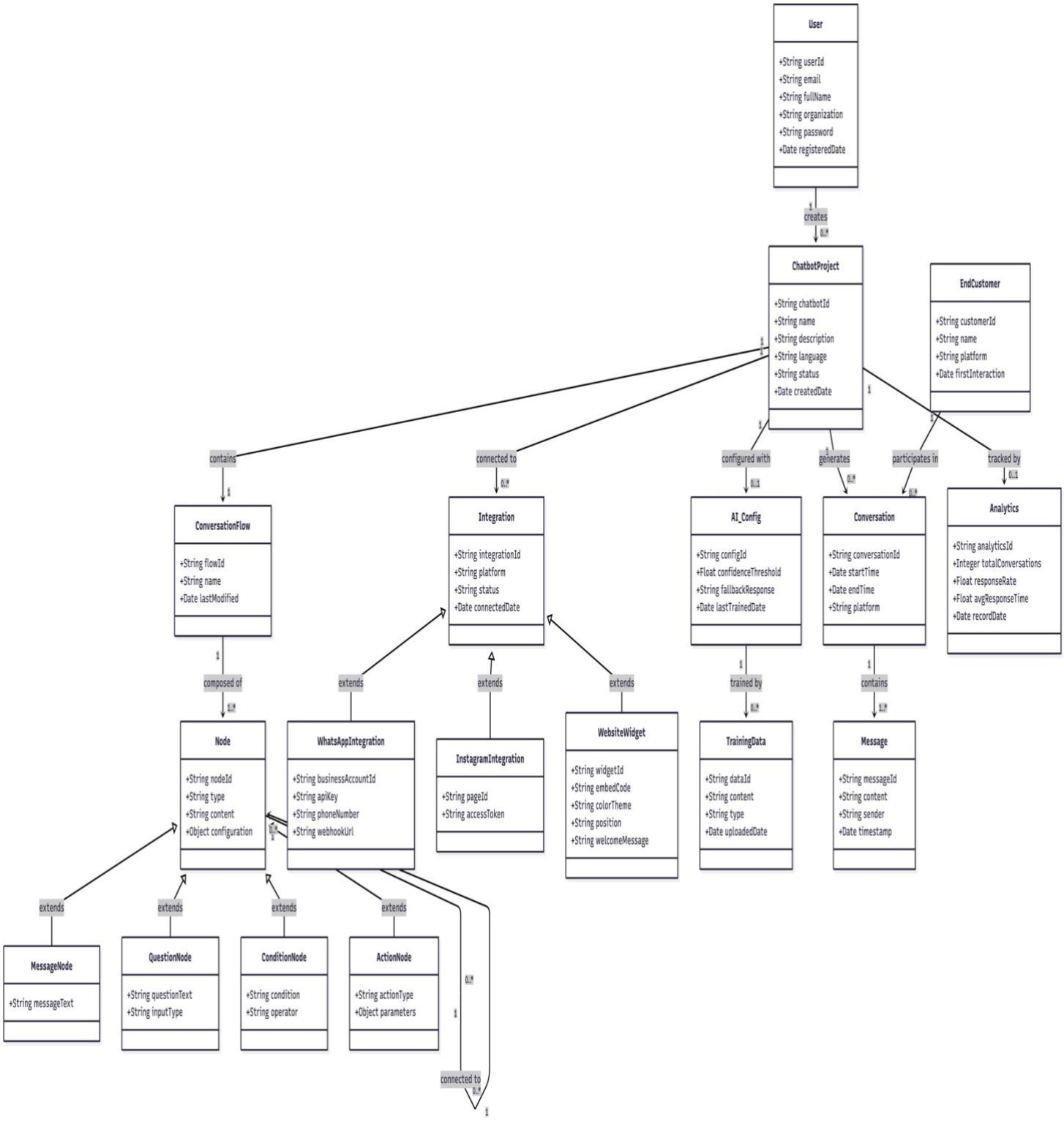


Figure 3.1: Domain Model Diagram

3.3 System Sequence Diagram

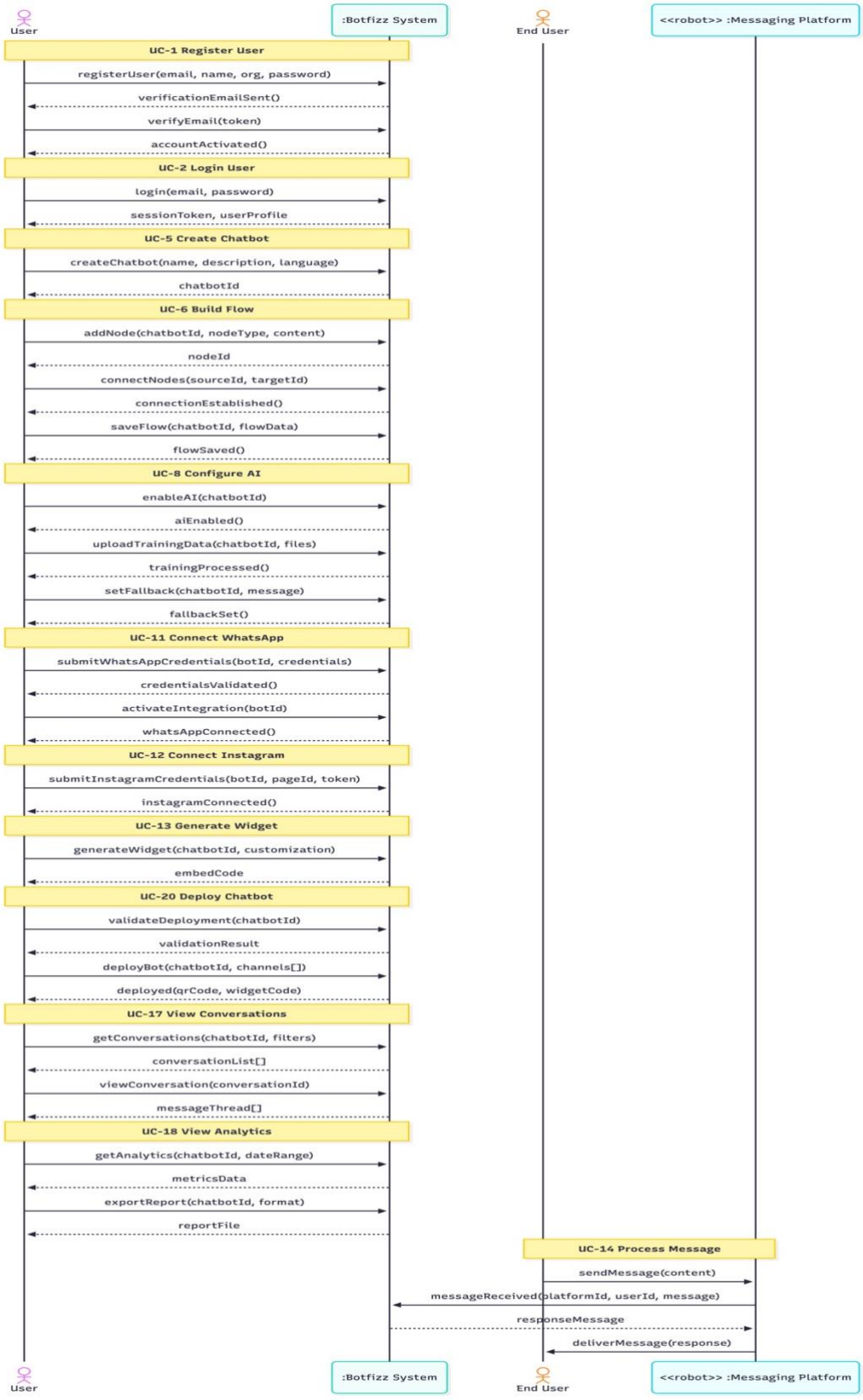


Figure 3.2: System Sequence Diagram

3.4 Sequence Diagram



© University of Gujrat

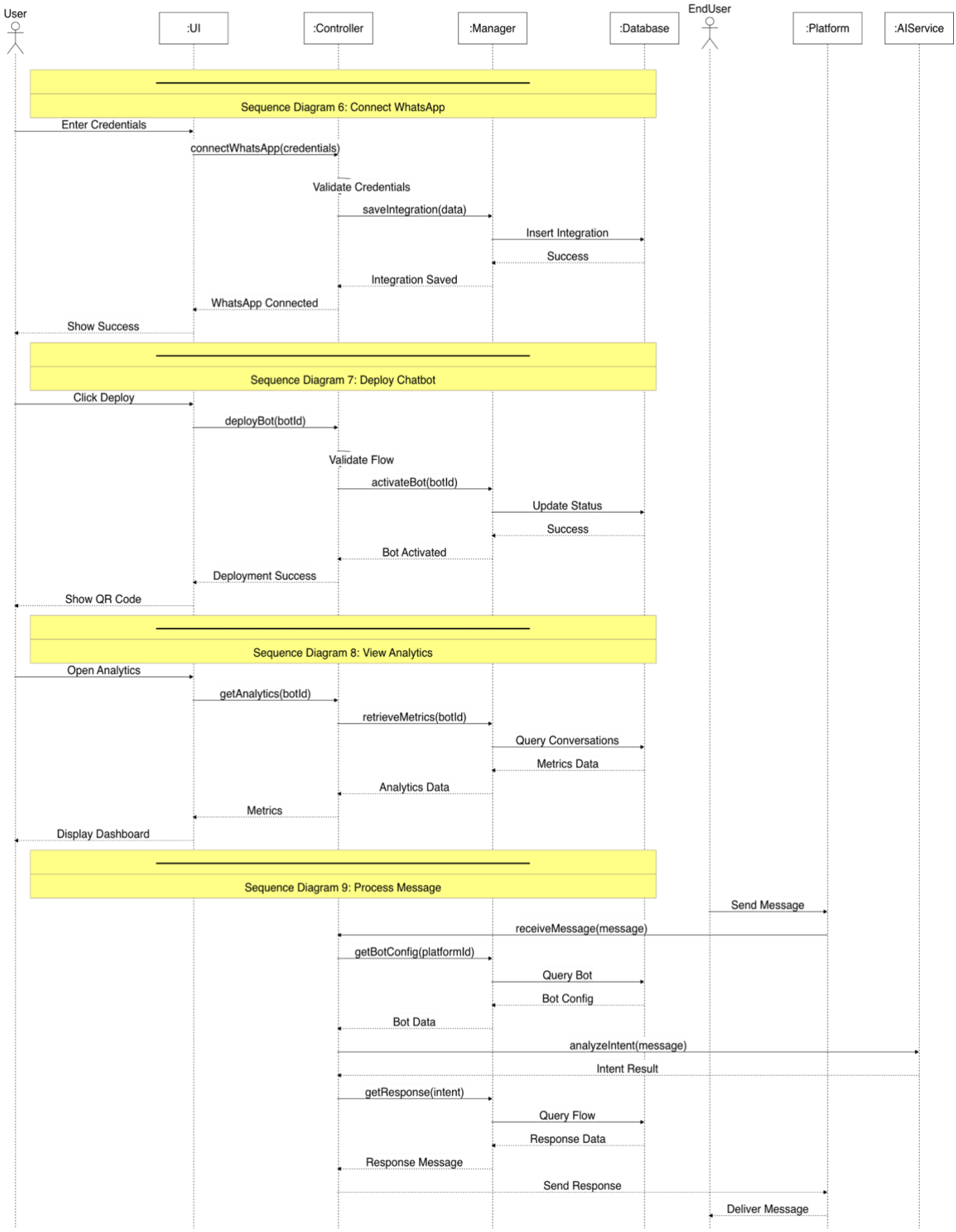


Figure 3.3: Sequence Diagram

3.5 Collaboration Diagram

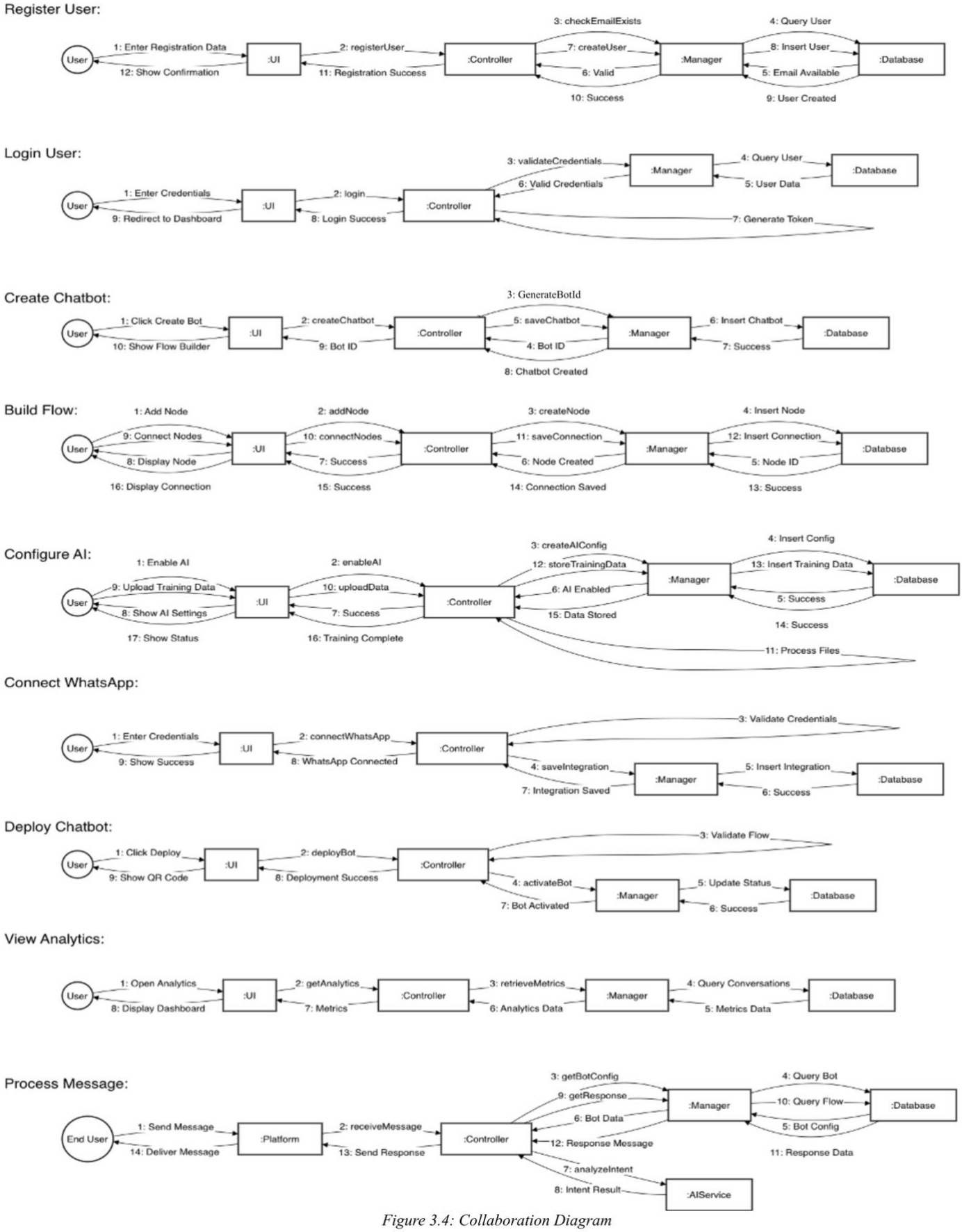


Figure 3.4: Collaboration Diagram

3.6 Operation Contracts

Operation Contract 1: registerUser

Operation: registerUser(email: String, name: String, organization: String, password: String)

Cross References: Use Case UC-1: Register User **Preconditions:**

- Email format is valid
- Email does not already exist in the system **Postconditions:**
- A new User instance was created
- User.email was set to email parameter
- User.name was set to name parameter
- User.organization was set to organization parameter
- User.password was encrypted and stored
- User.isVerified was set to false
- A verification token was generated
- A verification email was sent to the user's email address

Operation Contract 2: login

Operation: login(email: String, password: String) **Cross References:** Use Case UC-2: Login User **Preconditions:**

- User account exists with the given email
- User account is verified (isVerified = true)
- User account is active (status = active) **Postconditions:**
- User credentials were validated
- A new Session instance was created
- Session.token was generated
- Session.userId was associated with the User
- Session.createdAt was set to current timestamp
- Session token was returned to the user

Operation Contract 3: createChatbot

Operation: createChatbot(userId: String, name: String, description: String, language: String)

Cross References: Use Case UC-5: Create Chatbot **Preconditions:**

- User is authenticated
- User has an active session **Postconditions:**
- A new Chatbot instance was created
- Chatbot.id was generated (UUID)
- Chatbot.name was set to name parameter
- Chatbot.description was set to description parameter
- Chatbot.language was set to language parameter
- Chatbot.userId was associated with the authenticated User
- Chatbot.status was set to "draft"
- Chatbot.createdAt was set to current timestamp
- A default Flow instance was created for the Chatbot
- An association between Chatbot and Flow was formed

© University of Gujrat

Operation Contract 4: addNode
Operation: addNode(botId: String, nodeType: String, content: String, position: Object)
Cross References: Use Case UC-6: Build Flow **Preconditions:**

- Chatbot exists with the given botId
- User owns the Chatbot
- NodeType is valid (message, question, condition, action) **Postconditions:**
- A new Node instance was created
- Node.id was generated (UUID)
- Node.botId was associated with the Chatbot
- Node.type was set to nodeType parameter
- Node.content was set to content parameter
- Node.position was set to position parameter
- Node.createdAt was set to current timestamp
- An association between Node and Flow was formed

Operation Contract 5: connectNodes
Operation: connectNodes(botId: String, sourceNodeId: String, targetNodeId: String) **Cross References:** Use Case UC-6: Build Flow **Preconditions:**

- Both source and target Nodes exist
- Both Nodes belong to the same Chatbot
- Source Node type supports outgoing connections **Postconditions:**
- A new Connection instance was created
- Connection.id was generated (UUID)
- Connection.sourceNodeId was set to sourceNodeId parameter
- Connection.targetNodeId was set to targetNodeId parameter
- Connection.botId was associated with the Chatbot
- An association between Connection and both Nodes was formed

Operation Contract 6: enableAI
Operation: enableAI(botId: String)
Cross References: Use Case UC-8: Configure AI **Preconditions:**

- Chatbot exists with the given botId
- User owns the Chatbot
- AI configuration does not already exist for this Chatbot **Postconditions:**
- A new AIConfig instance was created
- AIConfig.botId was associated with the Chatbot
- AIConfig.isEnabled was set to true
- AIConfig.confidenceThreshold was set to default value (0.7)
- AIConfig.createdAt was set to current timestamp
- An association between AIConfig and Chatbot was formed

Operation Contract 7: uploadTrainingData
Operation: uploadTrainingData(botId: String, files: Array) **Cross References:** Use Case UC-8: Configure AI **Preconditions:**

© University of Gujrat

- Chatbot exists with the given botId
- AIConfig exists for the Chatbot
- Files are in supported format (PDF, TXT, DOCX) **Postconditions:**
- New TrainingData instances were created for each file
- TrainingData.botId was associated with the Chatbot
- TrainingData.content was extracted from files
- TrainingData.uploadedAt was set to current timestamp
- AIConfig.lastTrainedAt was updated to current timestamp
- Training data was sent to external AI service

Operation Contract 8: connectWhatsApp
Operation: connectWhatsApp(botId: String, businessId: String, apiKey: String, phoneNumber: String)
Cross References: Use Case UC-11: Connect WhatsApp **Preconditions:**

- Chatbot exists with the given botId
- WhatsApp credentials are valid
- Business account has necessary permissions **Postconditions:**
- A new Integration instance was created
- Integration.botId was associated with the Chatbot
- Integration.platform was set to "whatsapp"
- Integration.businessId was set to businessId parameter
- Integration.apiKey was encrypted and stored
- Integration.phoneNumber was set to phoneNumber parameter
- Integration.status was set to "active"
- Webhook was configured with WhatsApp API
- An association between Integration and Chatbot was formed

Operation Contract 9: connectInstagram
Operation: connectInstagram(botId: String, pageId: String, accessToken: String)
Cross References: Use Case UC-12: Connect Instagram **Preconditions:**

- Chatbot exists with the given botId
- Instagram page access token is valid
- Page has messaging permissions enabled **Postconditions:**
- A new Integration instance was created
- Integration.botId was associated with the Chatbot
- Integration.platform was set to "instagram"
- Integration.pageId was set to pageId parameter
- Integration.accessToken was encrypted and stored
- Integration.status was set to "active"
- Webhook subscription was created with Instagram API
- An association between Integration and Chatbot was formed

Operation Contract 10: deployBot
Operation: deployBot(botId: String, channels: Array) **Cross References:** Use Case UC-20: Deploy Chatbot **Preconditions:**

- Chatbot exists with the given botId

© University of Gujrat

- Chatbot has at least one Flow with valid Nodes
- At least one Integration is configured and active
- Flow validation passed successfully **Postconditions:**
- Chatbot.status was changed from "draft" to "active"
- Chatbot.deployedAt was set to current timestamp
- All specified Integrations were activated
- QR code was generated for WhatsApp (if applicable)
- Widget embed code was generated (if applicable)
- Chatbot is now accessible to end users on deployed channels

Operation Contract 11: processMessage
Operation: processMessage(platformId: String, userId: String, message: String)
Cross References: Use Case UC-14: Process Message **Preconditions:**

- Integration exists for the given platformId
- Associated Chatbot is in "active" status
- Message is not empty **Postconditions:**
- A new Conversation instance was created (if first message)
- Conversation.botId was associated with the Chatbot
- Conversation.userId was set to userId parameter
- Conversation.platform was set to platform type
- A new Message instance (incoming) was created
- Message.conversationId was associated with Conversation
- Message.content was set to message parameter
- Message.direction was set to "incoming"
- Message was analyzed by AI service for intent
- Appropriate response Node was retrieved from Flow
- A new Message instance (outgoing) was created with response
- Response was sent to the user via appropriate platform
- Conversation.lastMessageAt was updated

Operation Contract 12: getAnalytics
Operation: getAnalytics(botId: String, dateRange: Object) **Cross References:** Use Case UC-18: View Analytics **Preconditions:**

- Chatbot exists with the given botId
- User owns the Chatbot
- DateRange is valid **Postconditions:**
- Conversation count was retrieved from database
- Active user count was calculated
- Response rate percentage was calculated
- Average response time was calculated
- Platform distribution data was retrieved
- Analytics data was returned to user
- No instances were created or modified

© University of Gujrat

3.7 Design Class Diagram

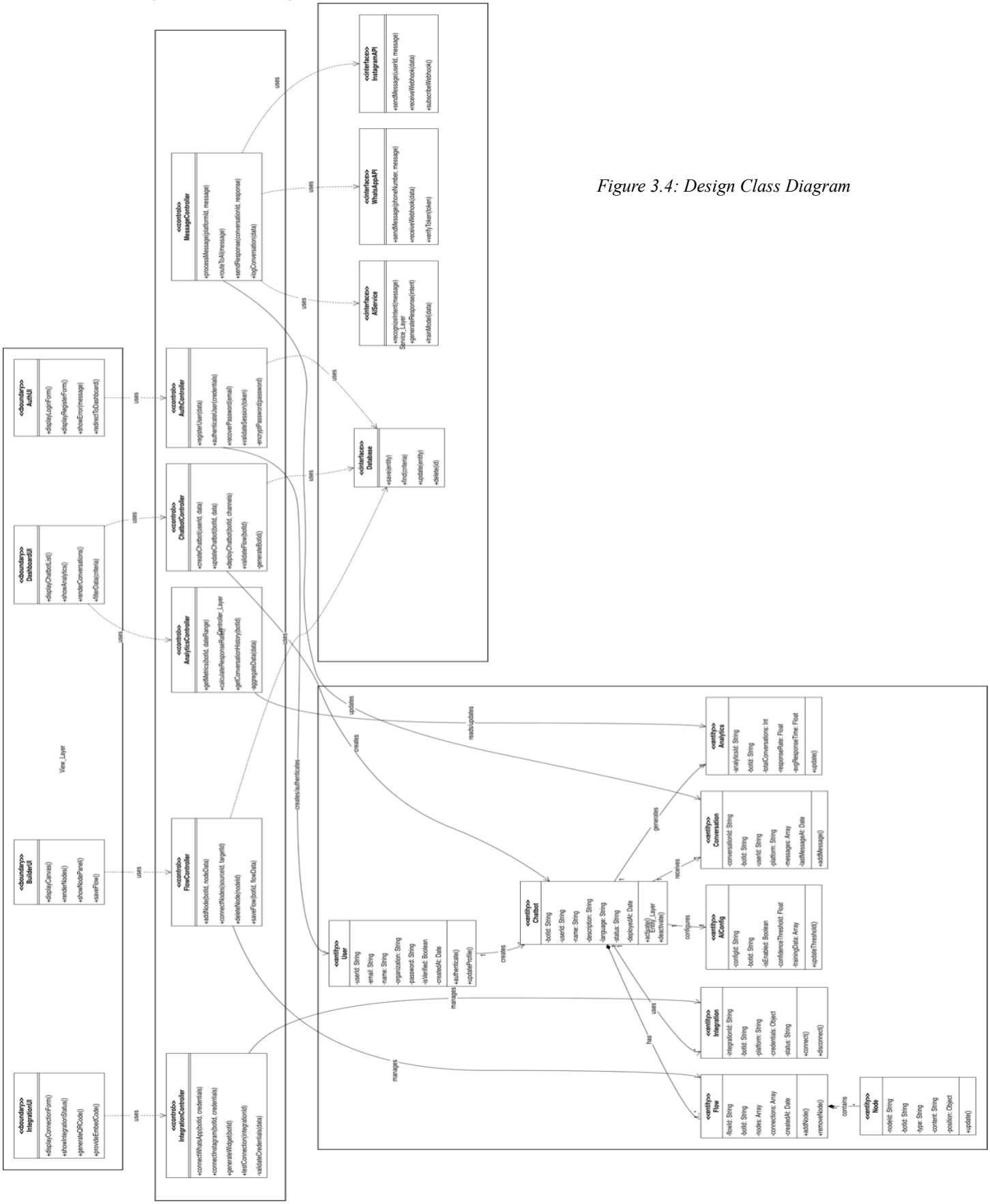


Figure 3.4: Design Class Diagram

© University of Gujrat

3.8 State Chart Diagram

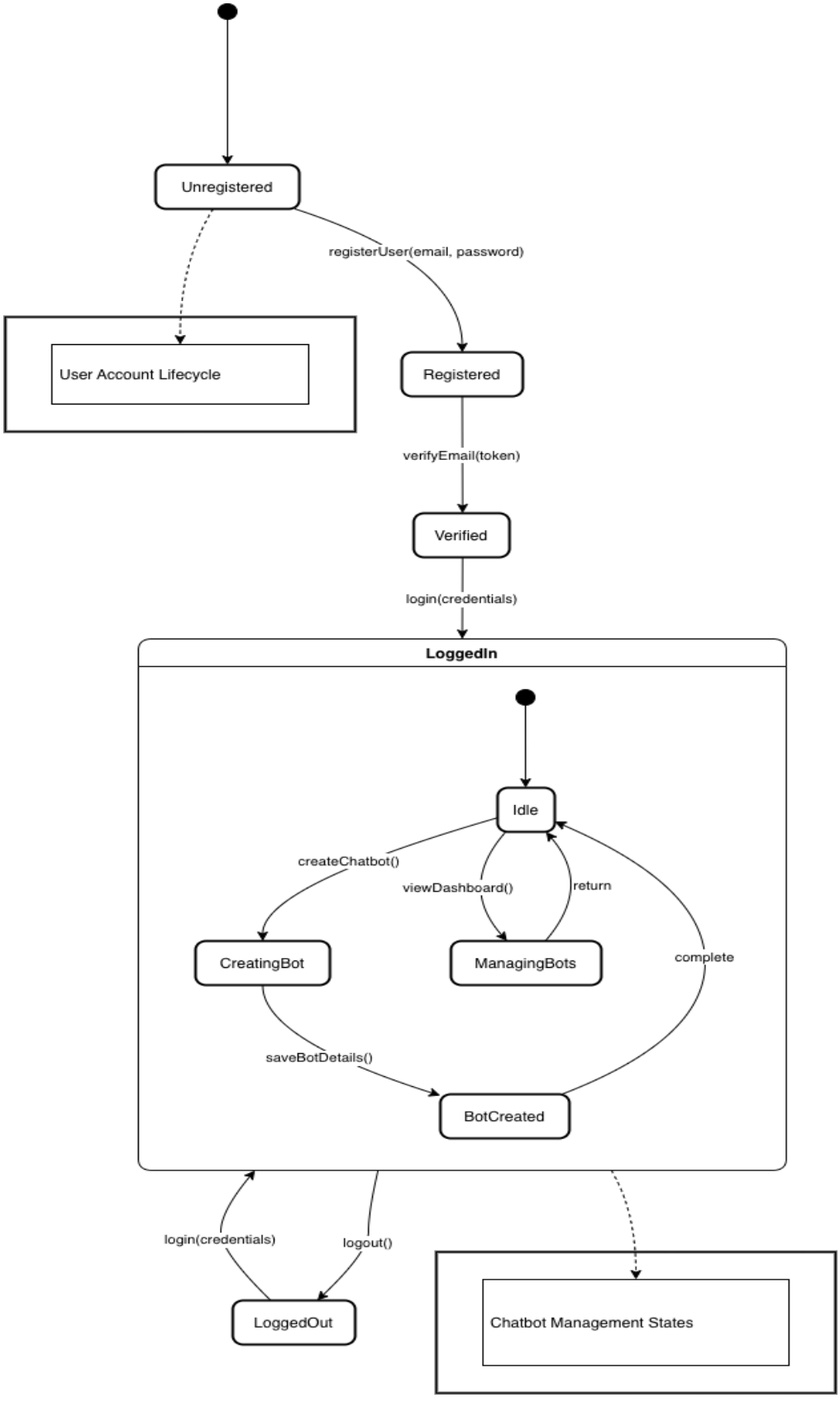


Figure 3.5: State Chart Diagram

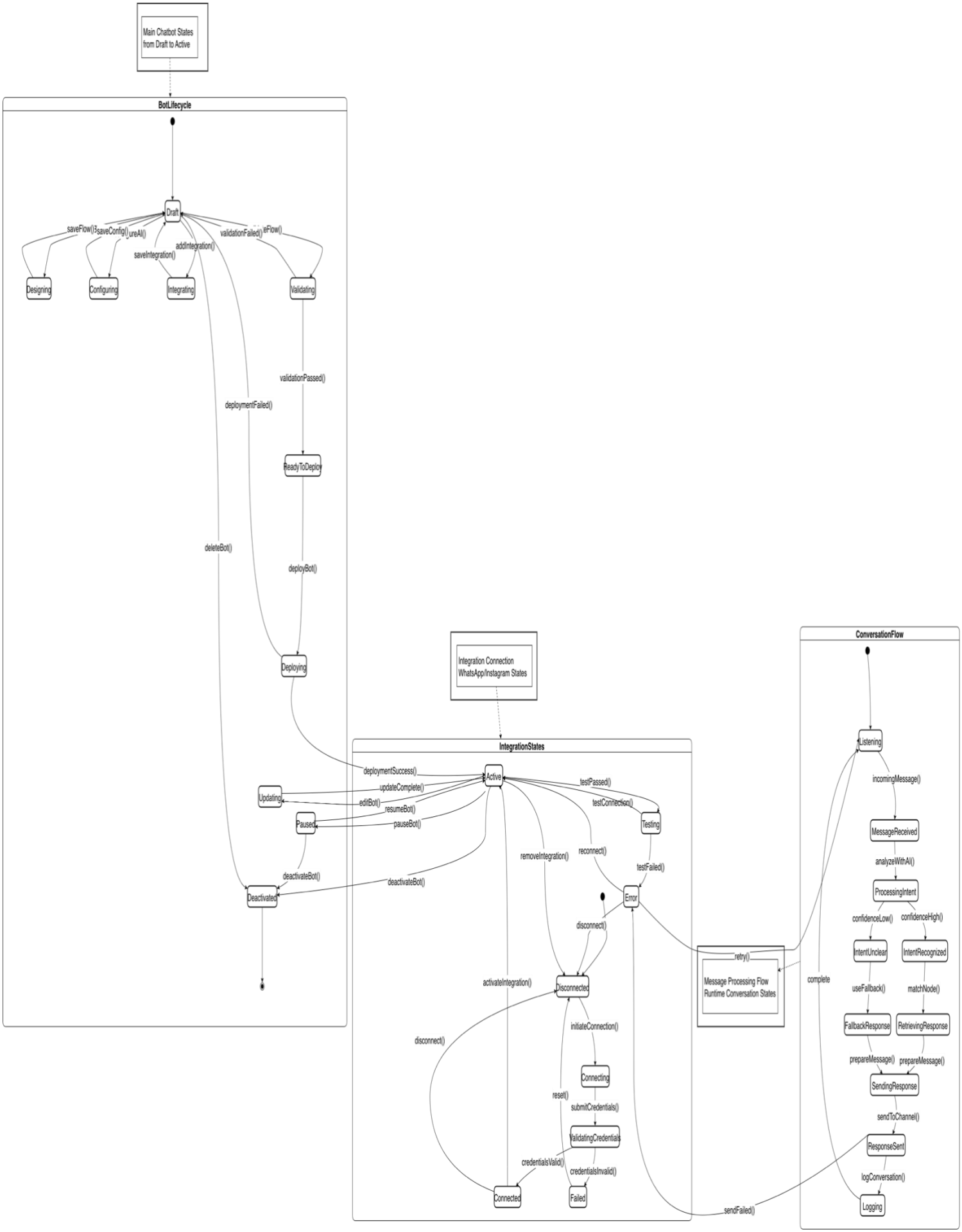


Figure 3.6: System Lifecycle Diagram

3.9 Data Model

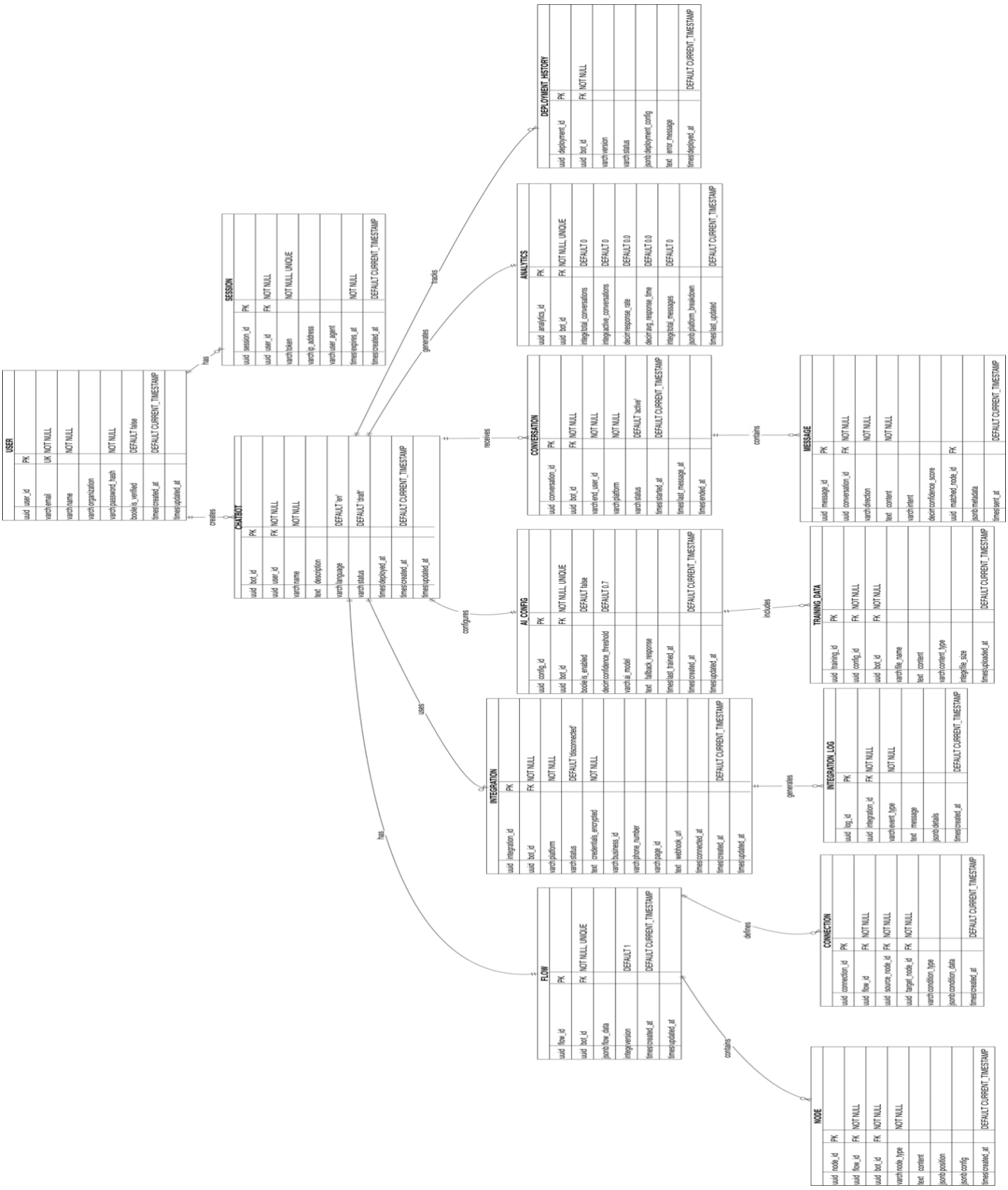


Figure 3.7: Data Model Diagram

© University of Gujrat

Chapter 4: User Interface Design

4.1 Introduction

A user interface design consists of three main parts. Page elements should be visualised on paper before building them in the computer. Just as you draw a site map to plan the site, use storyboards to begin blocking out the appearance and navigational scheme.

- Site Maps
- Storyboards
- Navigational Maps
- Traceability Matrix

BotFizz is a web-based SaaS platform built with React.js and Tailwind CSS. Its design philosophy prioritises no-code simplicity so that non-technical SME owners can build and deploy chatbots without any programming knowledge. Every screen is clean, label-heavy, and guides the user step-by-step.

4.2 Site Map

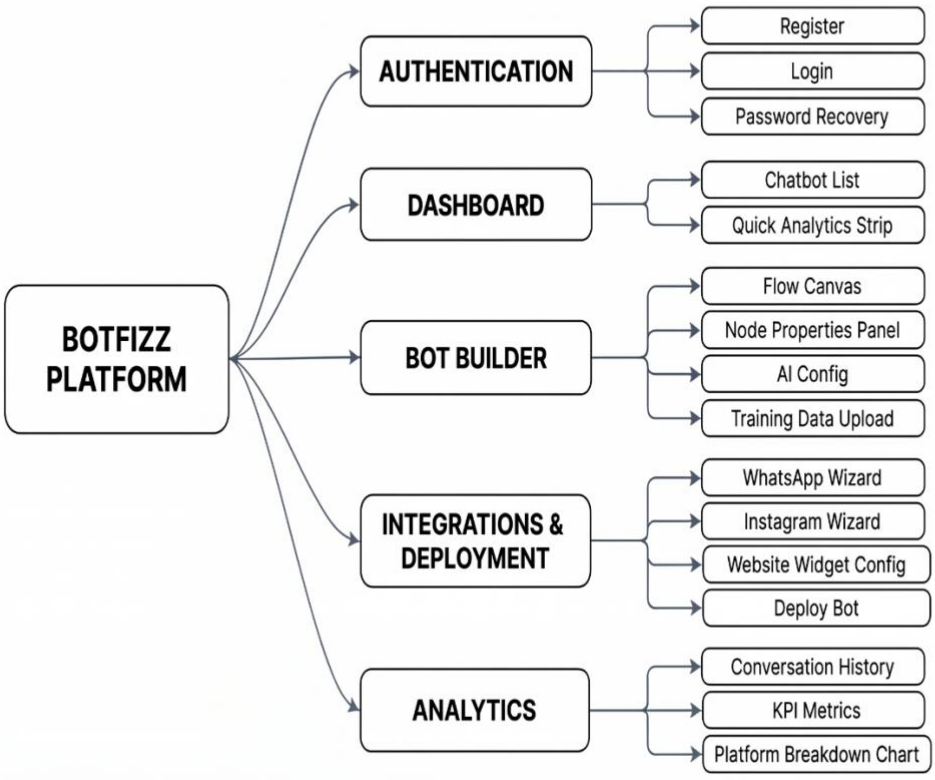


Figure 4.1: Site Map

4.3 Story boards

A storyboard is a sequence of single images, each representing a distinct event or interaction narrative. It is a visual representation of the script illustrating the interaction between the user and the BotFizz platform. It can be imagined as a film in visual-outline form.

A storyboard describes the task as a series of interaction states and also describes the interface by representing a series of screen states indicating the user's representation and the system's response. It shows interaction sequences briefly and helps develop usage scenarios.

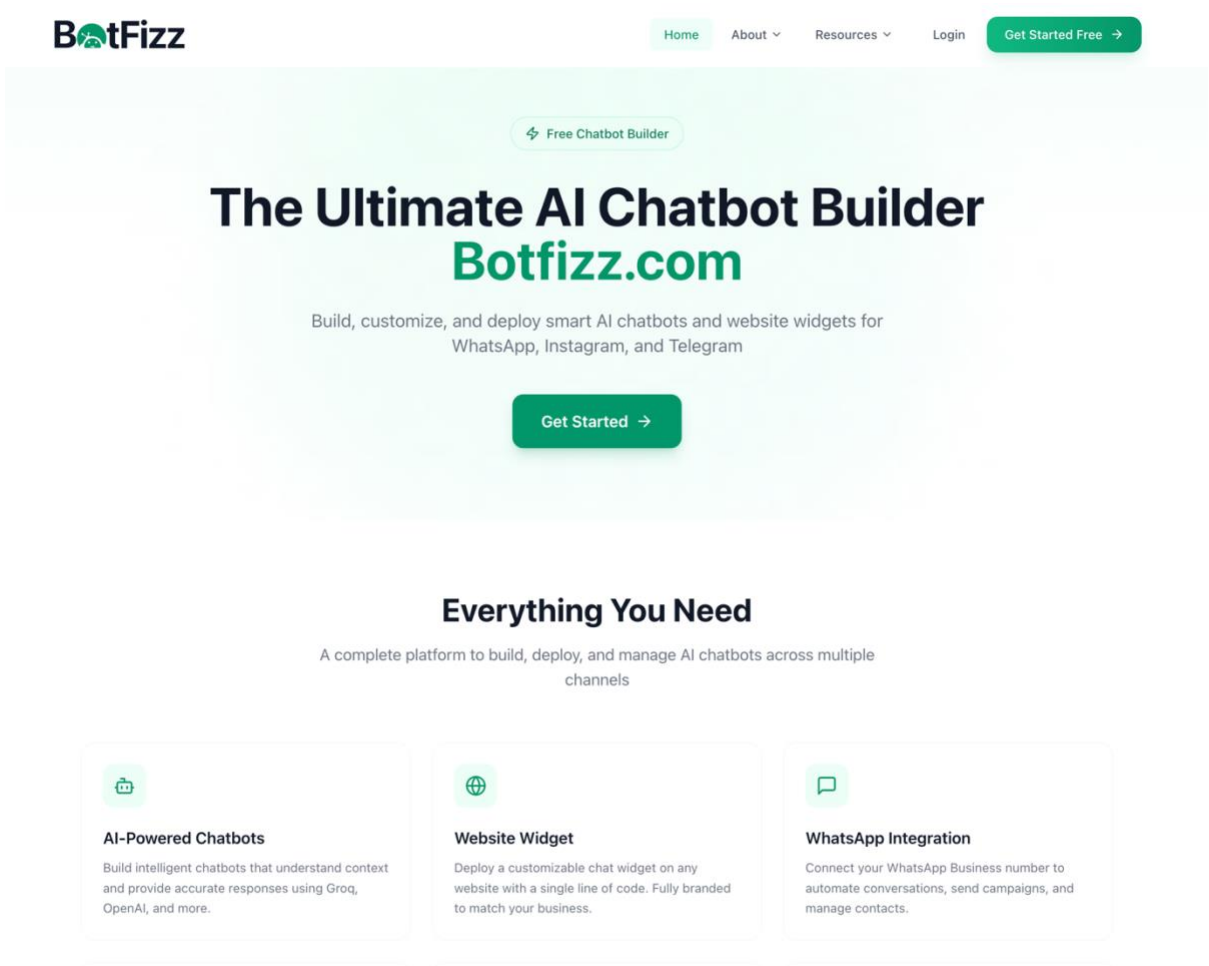
Environment:

BotFizz is a browser-based SaaS web application accessible on desktop and laptop screens. The platform enables SME owners to design, configure, and deploy AI chatbots to WhatsApp, Instagram, and website widgets without any coding knowledge. The environment is predominantly office-based, accessed via a web browser (Chrome, Firefox, Edge).

© University of Gujrat

Table 4.3.1

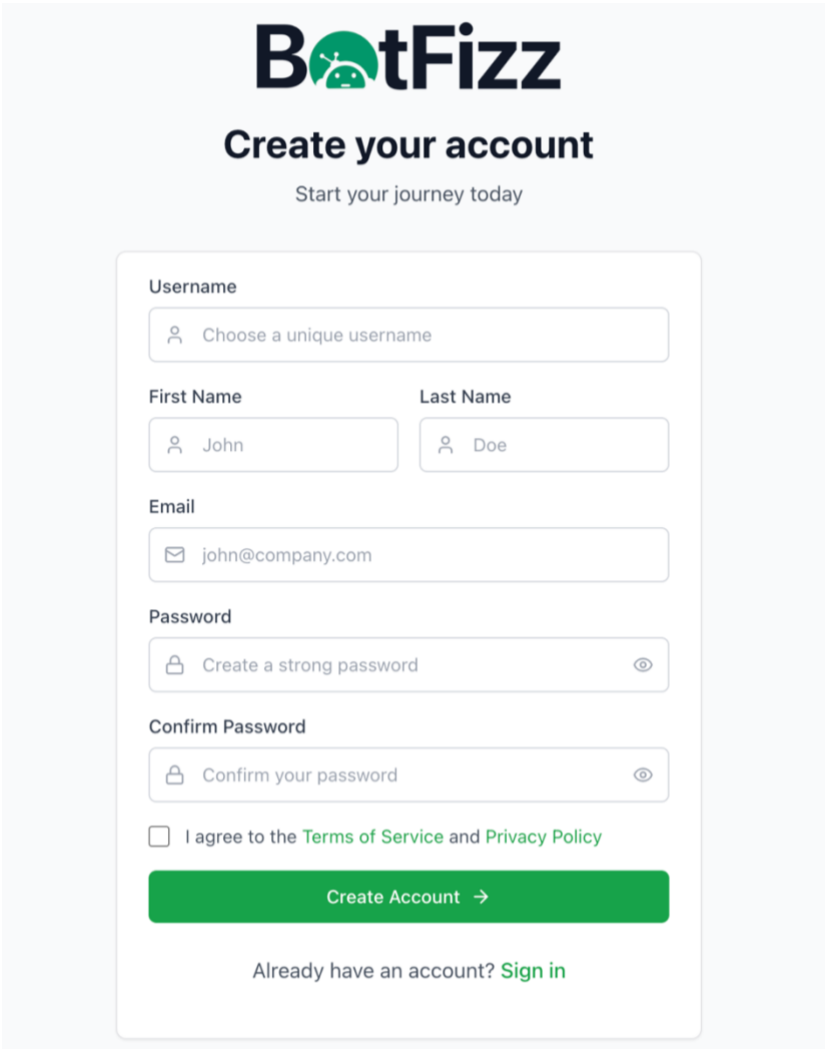
Screen: Landing Page / Homepage



Visual Cues	A clean landing page showing the BotFizz logo, tagline 'Build AI Chatbots Without Code', a Sign Up button, and a Login link. Feature highlights are visible as cards below.
Tactile Cues	User can click the 'Get Started Free' button or the 'Login' link to navigate.
User Input	User clicks 'Get Started Free' to initiate registration.
System Output	System navigates to the Registration form page displaying fields for full name, email, organization name, and password.
User's Emotions	Curious and Motivated
Technology	React.js, Tailwind CSS
Quality of Exp.	Excellent

Table 4.3.2

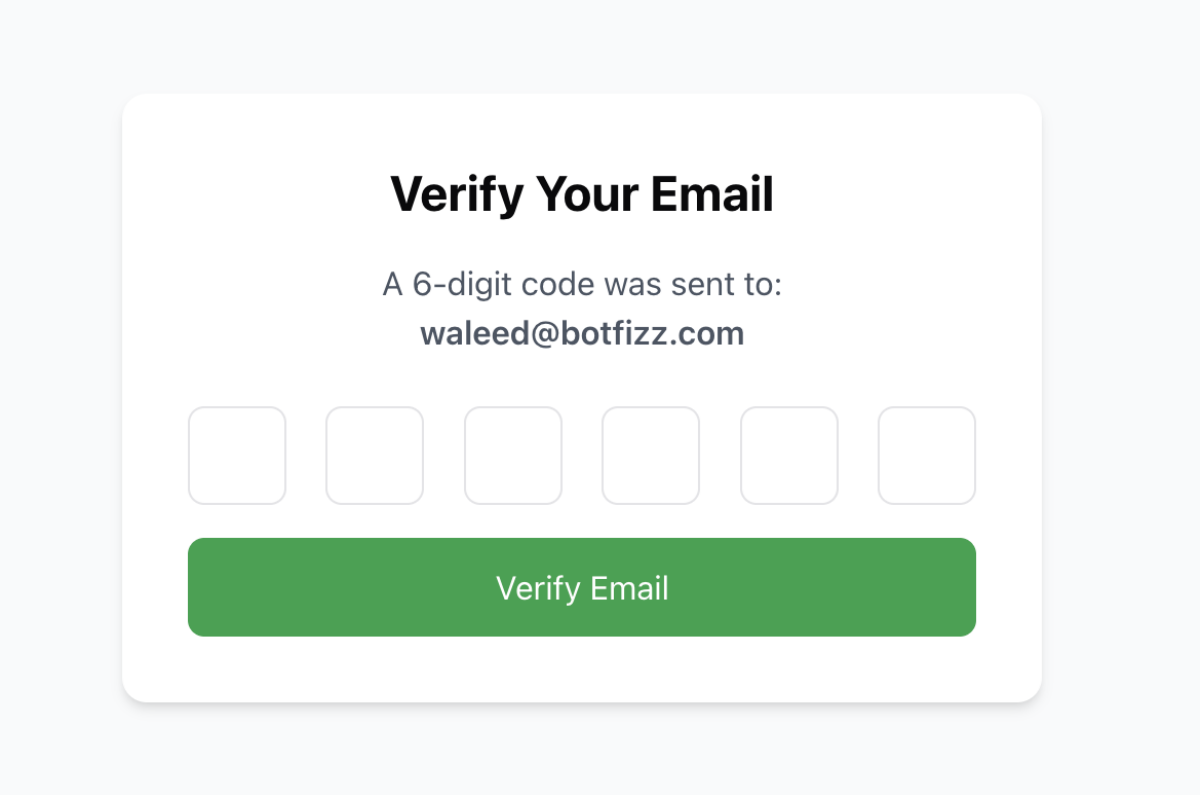
Screen: User Registration Form



Visual Cues	A registration form with input fields: Full Name, Email Address, Organization Name, Password, and Confirm Password. A 'Create Account' button is at the bottom with a 'Already have an account? Login' link.
Tactile Cues	User taps/clicks input fields to enter data. User can click the eye icon to reveal password. User clicks 'Create Account' button to submit.
User Input	User enters: Full Name (M. Hanzala), Email, Organization (e.g., ShopEase), Password, and confirms password.
System Output	System validates input format, checks for duplicate email, creates the user account, generates a 6-digit OTP, sends the OTP to the user's registered email via Nodemailer, and redirects to the OTP Verification screen.
User's Emotions	Satisfied and Engaged
Technology	React.js, Node.js/Express, PostgreSQL, Nodemailer, OTP Generator
Quality of Exp.	Excellent

Table 4.3.3

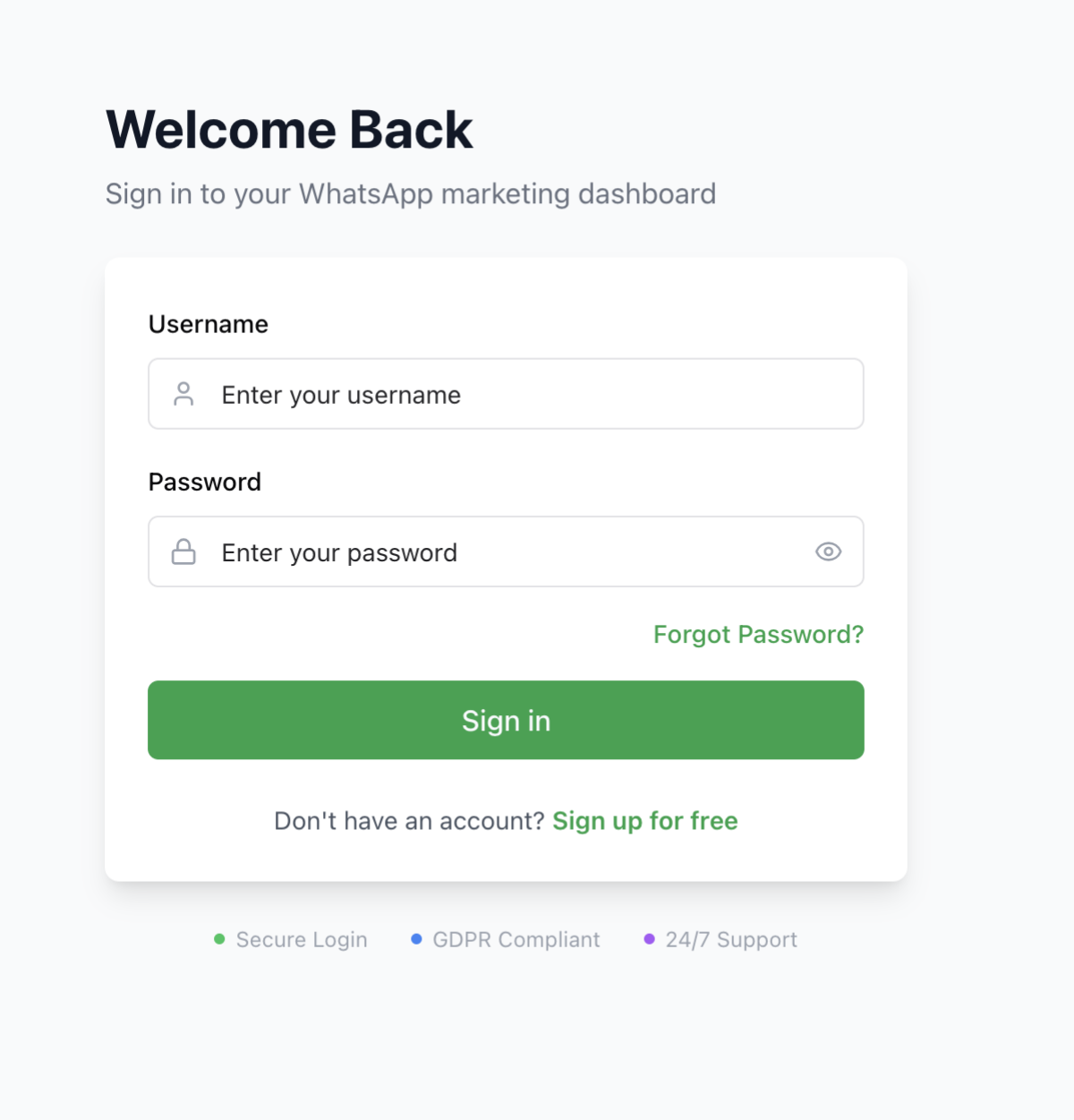
Screen: Email Verification



Visual Cues	An OTP Verification screen with a 6-digit OTP input field and a 'Verify Email' button. A 'Resend OTP' button is available below. A countdown timer shows OTP expiry (e.g., 5 minutes).
Tactile Cues	User opens their email inbox, reads the OTP sent by BotFizz, types the 6-digit OTP into the input fields on the verification screen, and clicks 'Verify Email'.
User Input	User retrieves the OTP from their email and enters the 6-digit OTP into the BotFizz verification screen, then clicks 'Verify Email'.
System Output	System validates the submitted OTP against the stored OTP (checking expiry), activates the user account, marks email as verified in the database, and redirects to the Login page with a success toast notification.
User's Emotions	Relieved and Satisfied
Technology	React.js, Nodemailer, OTP-based email verification (6-digit OTP with expiry)
Quality of Exp.	Excellent

Table 4.3.4

Screen: Login Screen

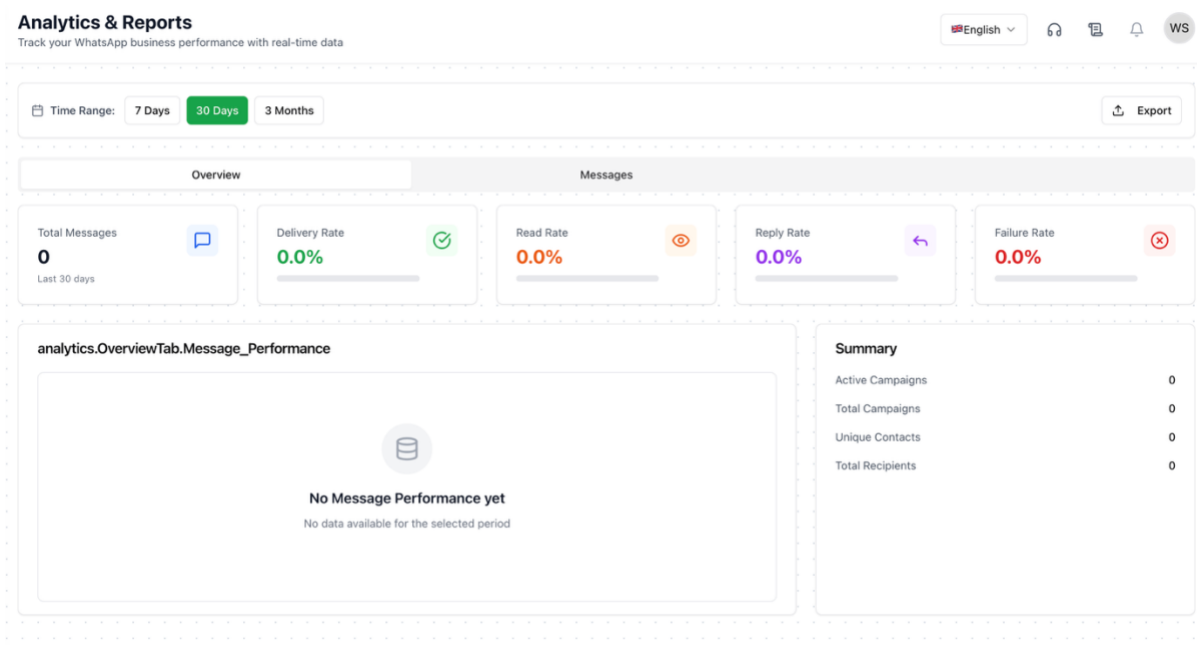


Visual Cues	A login form with Email and Password fields. A 'Login' button and 'Forgot Password?' link are visible. Social login is not required for Phase I.
Tactile Cues	User clicks the Email and Password input fields, types credentials, and clicks 'Login'.
User Input	User enters registered email and password, then clicks 'Login'.
System Output	System verifies credentials, creates a session JWT token, and redirects the user to the Main Dashboard.

User's Emotions	Confident and Ready
Technology	React.js, Node.js/Express, JWT Authentication, PostgreSQL
Quality of Exp.	Excellent

Table 4.3.5

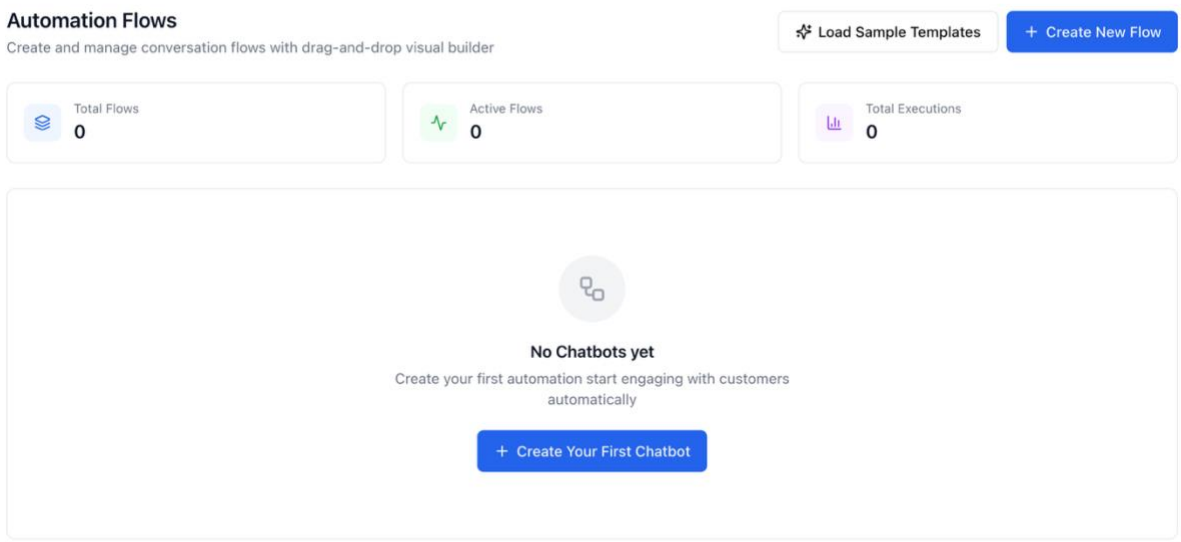
Screen: Main Dashboard



Visual Cues	Analytics dashboard featuring KPI cards (Total Messages, Delivery Rate, Read Rate, Reply Rate, Failure Rate) and a Message Performance chart area with a summary sidebar.
Tactile Cues	User selects Time Range (7 Days, 30 Days, 3 Months), toggles between Overview/Messages tabs, and clicks the Export button to download data.
User Input	User filters analytics by date range or clicks on specific metric cards to view detailed performance breakdowns.
System Output	System fetches real-time messaging data from the database, calculates percentages for rates (Delivery, Read, etc.), and populates the Chart.js graphs.
User's Emotions	Analytical and Informed.
Technology	React.js, Chart.js (for the graphs), PostgreSQL (data storage), Node.js (API).
Quality of Exp.	Data-driven and professional.

Table 4.3.6

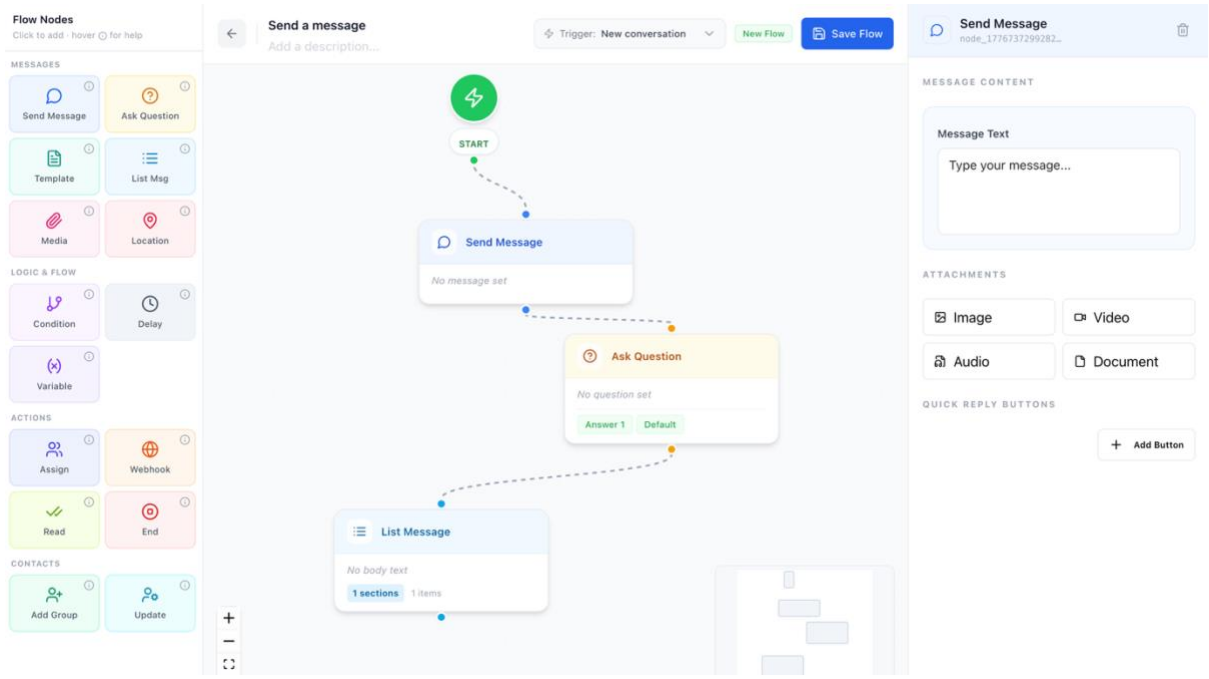
Screen: Create New Chatbot



Visual Cues	An empty state dashboard showing three statistic cards (Total Flows, Active Flows, Total Executions) and a prominent "Create Your First Chatbot" primary action button.
Tactile Cues	User clicks the "Create Your First Chatbot" button or the "Create New Flow" button in the top right corner.
User Input	A single click action on the primary call-to-action (CTA) button to initiate the automation setup.
System Output	System initializes a new flow session and immediately redirects the user to the interactive Flow Builder canvas to begin designing nodes.
User's Emotions	Determined and ready to build.
Technology	React.js (Frontend), Node.js (Backend), React Flow / Canvas API (for the Builder).
Quality of Exp.	Seamless and Frictionless.

Table 4.3.7

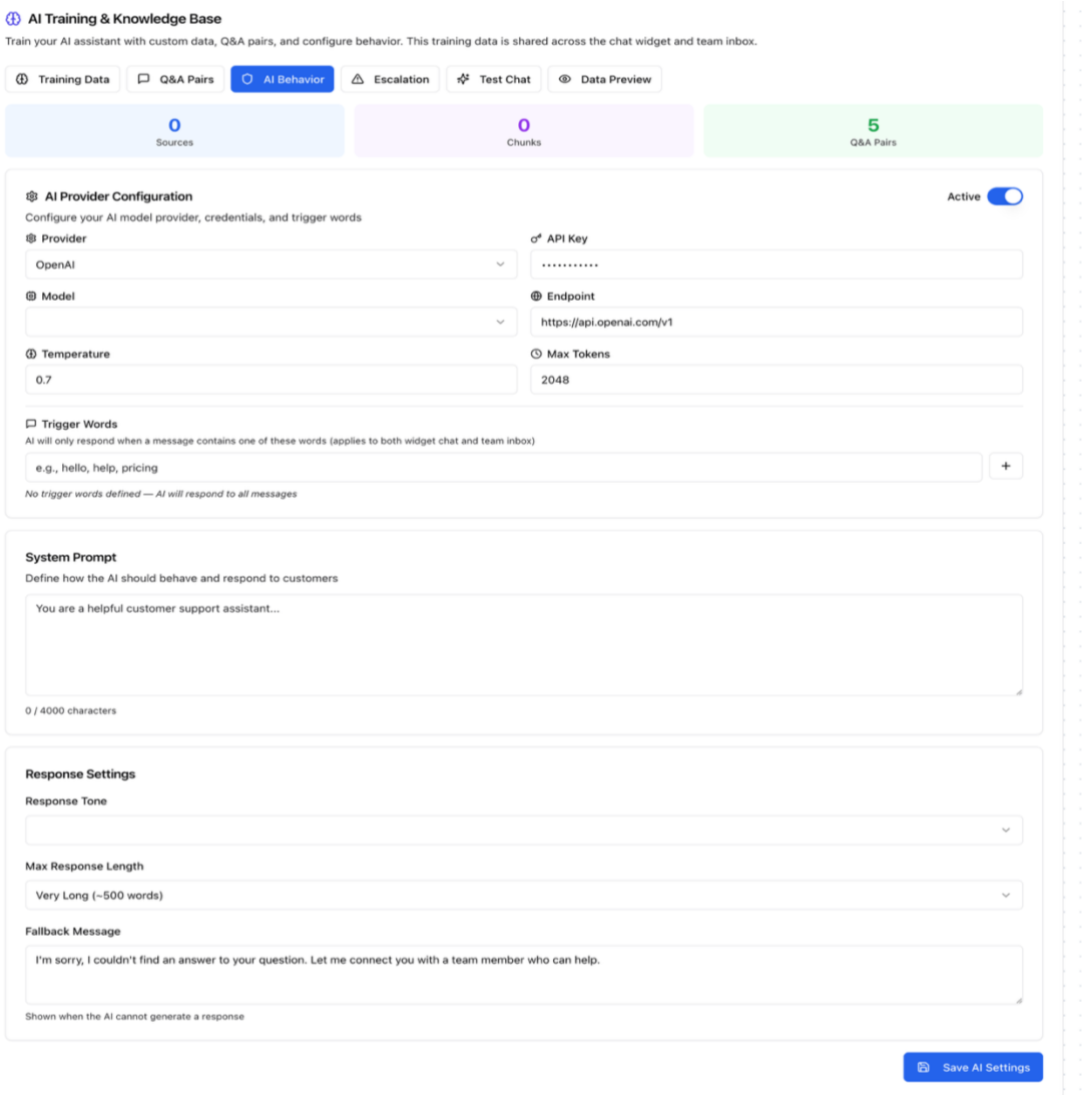
Screen: Flow Builder Canvas



Visual Cues	A full-screen drag-and-drop canvas with a node palette on the left (Message Node, Question Node, Condition Node, Action Node). Start Node is pre-placed. A top toolbar shows Save, Preview, and Deploy buttons.
Tactile Cues	User drags nodes from the palette onto the canvas. User connects nodes by drawing edges. User clicks a node to open its configuration panel on the right.
User Input	User drags a 'Message Node' onto the canvas, clicks it to configure the bot's greeting message text.
System Output	System renders the node on the canvas, opens the right-side configuration panel, and saves changes in real-time as user types.
User's Emotions	Creative and In-Control
Technology	React.js, React Flow library, Node.js, PostgreSQL
Quality of Exp.	Excellent

Table 4.3.8

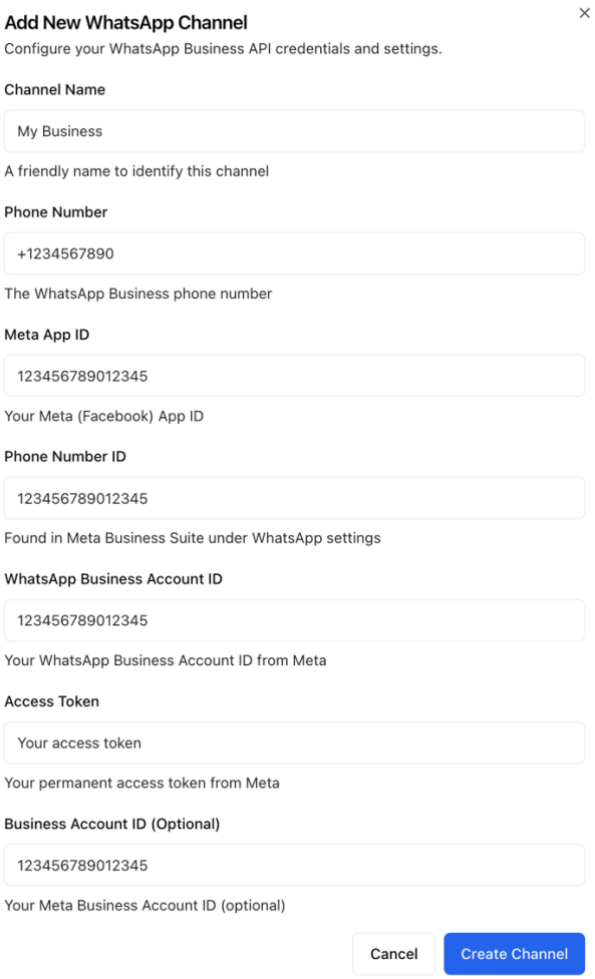
Screen: AI Configuration Panel



Visual Cues	A side panel showing AI settings: Enable AI Toggle, AI Service Provider (dropdown), API Key input field, Confidence Threshold slider (0%–100%), and Upload Training Data button.
Tactile Cues	User toggles 'Enable AI', selects provider (OpenAI), pastes API key, adjusts the threshold slider, and uploads a FAQ document.
User Input	User enables AI, selects OpenAI as provider, enters API key, sets confidence to 75%, and uploads a PDF containing FAQs.
System Output	System encrypts and stores the API key, saves the configuration, processes the uploaded document for training context, and confirms with a success alert.
User's Emotions	Confident and Informed
Technology	React.js, Node.js, OpenAI API, PostgreSQL (encrypted storage)
Quality of Exp.	Excellent

Table 4.3.9

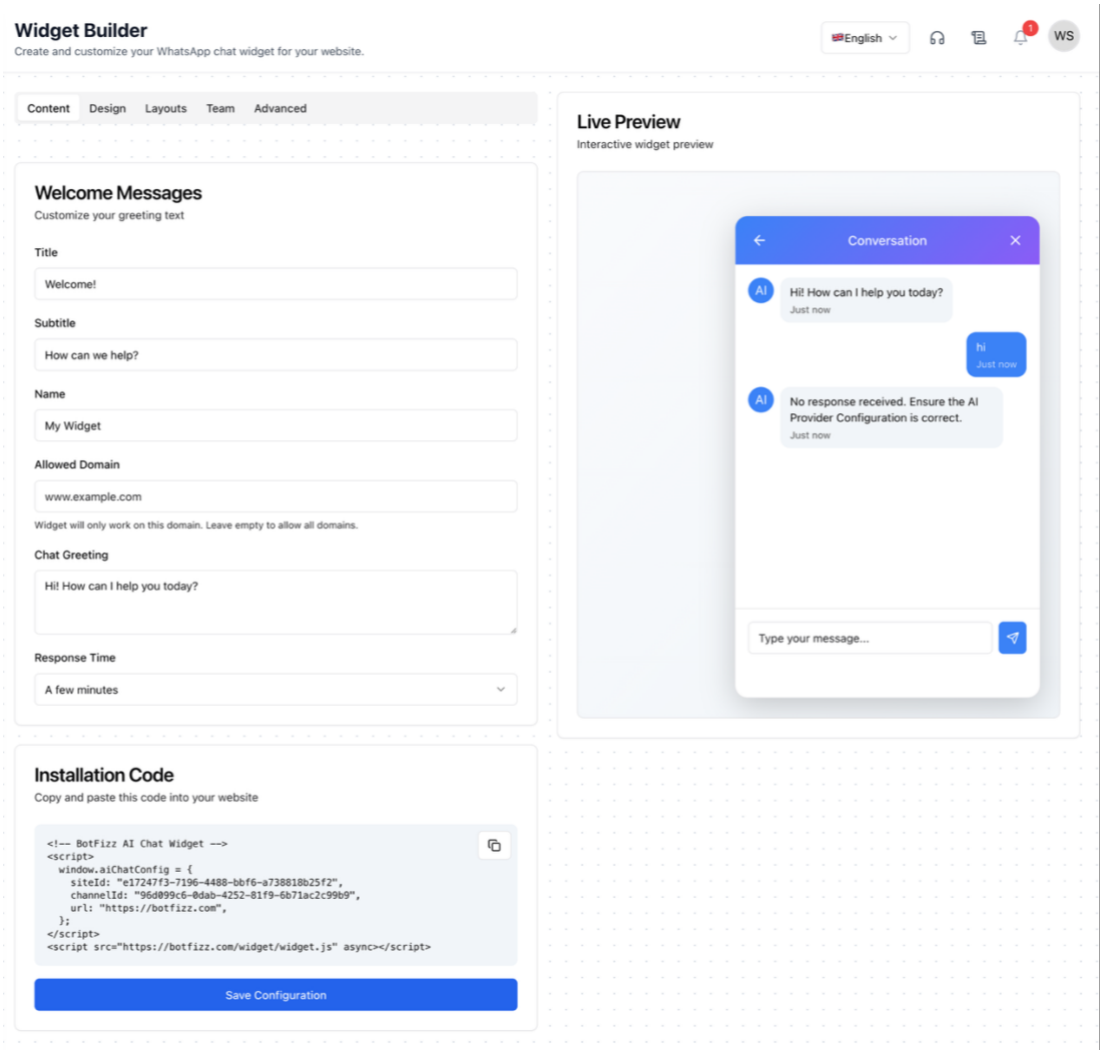
Screen: Integration Screen – WhatsApp



Visual Cues	A screen showing WhatsApp Business API integration steps: Enter Phone Number ID, Enter Business Account ID, Enter WhatsApp API Token, and a 'Connect WhatsApp' button. Status badge shows Disconnected/Connected/Error.
Tactile Cues	User reads instructions, copies API credentials from Meta Developer Portal, pastes them into input fields, and clicks 'Connect WhatsApp'.
User Input	User pastes Phone Number ID, Business Account ID, and WhatsApp Access Token, then clicks 'Connect'.
System Output	System validates credentials via Meta API test call, stores credentials securely, updates integration status badge to 'Connected', and shows QR code for bot access.
User's Emotions	Accomplished and Excited
Technology	React.js, Node.js, Meta WhatsApp Business API, PostgreSQL
Quality of Exp.	Excellent

Table 4.3.10

Screen: Chatbot Preview / Test Mode



Visual Cues	A chat simulation panel on the right side of the flow builder. Shows a mock chat window where the user can type messages and receive bot responses based on the configured flow.
Tactile Cues	User types test messages into the preview chat window to simulate an end-user conversation.
User Input	User types 'Hello' and the chatbot responds with the configured greeting message.
System Output	System processes the input through the configured flow logic, applies AI intent recognition if enabled, and displays the bot response in the preview window.
User's Emotions	Satisfied and Reassured
Technology	React.js, WebSocket (real-time preview), Node.js AI integration
Quality of Exp.	Excellent

4.4 Navigational maps:

The next step is of navigational maps. In these maps, the storyboards are used as an input. The different display buttons or action buttons show the navigation from one screen to the other. In other words when one action button is pressed it would lead to other screens. This path and navigation would be shown.

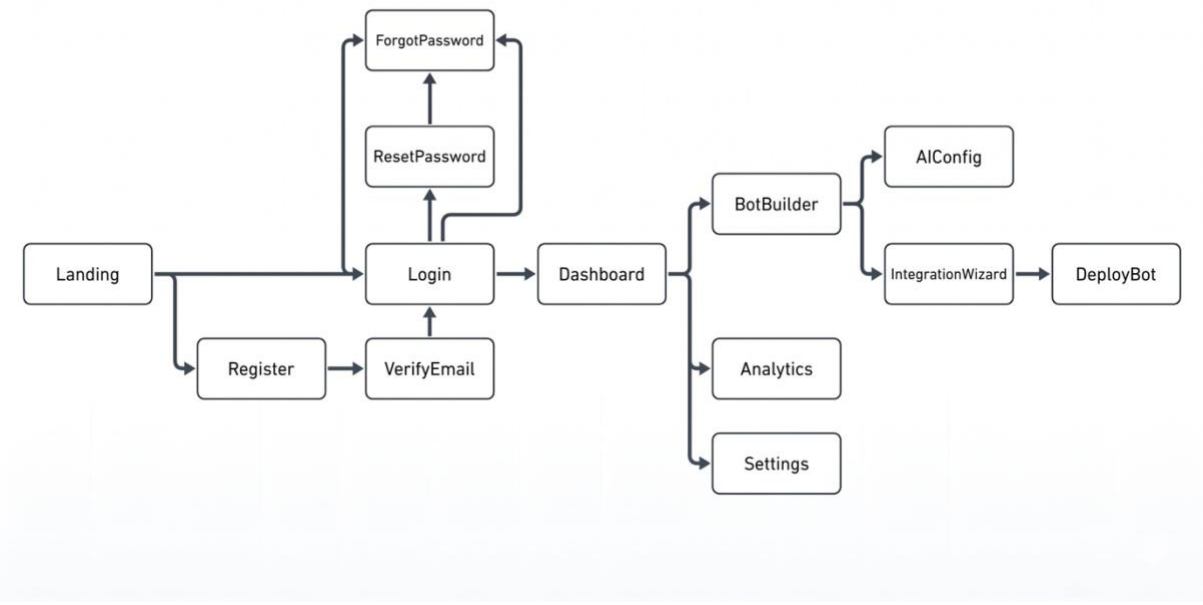


Figure 4.2: Navigational Maps

4.5 Trace-ability Matrix

User Registration

Table 4.5.1. Users can register with email and password

Feature	Users can register with email and password
Use Case ID	UC-1
Priority	Highest
Use Case Cross Ref	UC-Register-User
DB Table ID	Users
Dependent Classes	User, AuthController, EmailService

User Login

Table 4.5.2. Users can login with verified credentials

Feature	Users can login with verified credentials
Use Case ID	UC-2
Priority	Highest
Use Case Cross Ref	UC-Login-User
DB Table ID	Users, Sessions
Dependent Classes	User, AuthController, SessionManager

Password Recovery

Table 4.5.3. Users can recover forgotten password via email

Feature	Users can recover forgotten password via email
Use Case ID	UC-3
Priority	Medium
Use Case Cross Ref	UC-Recover-Password
DB Table ID	Users, PasswordResets
Dependent Classes	User, EmailService, AuthController

Create Chatbot

Table 4.5.4. Users can create new chatbot projects

Feature	Users can create new chatbot projects
Use Case ID	UC-5
Priority	Highest
Use Case Cross Ref	UC-Create-Chatbot
DB Table ID	Chatbots
Dependent Classes	User, ChatbotController, FlowBuilder

Build Conversation Flow

Table 4.5.5. Users can design chatbot flow with drag-and-drop nodes

Feature	Users can design chatbot flow with drag-and-drop nodes
Use Case ID	UC-6, UC-7
Priority	Highest

Use Case Cross Ref	UC-Build-Flow, UC-Add-Node
DB Table ID	Chatbots, Nodes, Edges
Dependent Classes	FlowCanvas, NodeEditor, FlowValidator

AI Intent Configuration

Table 4.5.6. Users can configure AI intent recognition settings

Feature	Users can configure AI intent recognition settings
Use Case ID	UC-8
Priority	Medium
Use Case Cross Ref	UC-Configure-AI
DB Table ID	ChatbotAIConfig, APICredentials
Dependent Classes	AIConfigPanel, EncryptionService, OpenAIAdapter

Upload Training Data

Table 4.5.7. Users can upload training documents for AI improvement

Feature	Users can upload training documents for AI improvement
Use Case ID	UC-9
Priority	Medium
Use Case Cross Ref	UC-Upload-Training-Data
DB Table ID	TrainingFiles, ChatbotAIConfig
Dependent Classes	FileUploader, DocumentProcessor, AITrainer

WhatsApp Integration

Table 4.5.8. Users can connect WhatsApp Business API to chatbot

Feature	Users can connect WhatsApp Business API to chatbot
Use Case ID	UC-11
Priority	Highest
Use Case Cross Ref	UC-Connect-WhatsApp
DB Table ID	Integrations, APICredentials
Dependent Classes	IntegrationController, MetaAPIAdapter, QRGenerator

Instagram Integration

Table 4.5.9. Users can connect Instagram Business Messaging API

Feature	Users can connect Instagram Business Messaging API
Use Case ID	UC-12
Priority	Medium
Use Case Cross Ref	UC-Connect-Instagram
DB Table ID	Integrations, APICredentials
Dependent Classes	IntegrationController, MetaAPIAdapter

Website Widget Generation

Table 4.5.10. System generates embeddable widget code for websites

Feature	System generates embeddable widget code for websites
Use Case ID	UC-13
Priority	Medium
Use Case Cross Ref	UC-Generate-Widget
DB Table ID	Chatbots, Integrations
Dependent Classes	WidgetGenerator, EmbedCodeBuilder

Deploy Chatbot

Table 4.5.11. Users can deploy chatbot to selected platforms in one click

Feature	Users can deploy chatbot to selected platforms in one click
Use Case ID	UC-20
Priority	Highest
Use Case Cross Ref	UC-Deploy-Chatbot
DB Table ID	Deployments, Chatbots
Dependent Classes	DeploymentController, StatusTracker

Generate QR Code

Table 4.5.12. System generates WhatsApp QR codes for bot access

Feature	System generates WhatsApp QR codes for bot access
Use Case ID	UC-21
Priority	Medium

Use Case Cross Ref	UC-Generate-QR-Code
DB Table ID	Deployments
Dependent Classes	QRGenerator, WhatsAppAdapter

View Analytics Dashboard

Table 4.5.13. Users can view analytics metrics on dashboard

Feature	Users can view analytics metrics on dashboard
Use Case ID	UC-18
Priority	Medium
Use Case Cross Ref	UC-View-Analytics
DB Table ID	Conversations, Analytics
Dependent Classes	AnalyticsController, DashboardWidget, ChartRenderer

View Conversation History

Table 4.5.14. Users can view and filter all conversation logs

Feature	Users can view and filter all conversation logs
Use Case ID	UC-17
Priority	Medium
Use Case Cross Ref	UC-View-Conversations
DB Table ID	Conversations, Messages
Dependent Classes	ConversationViewer, FilterPanel

Manage Chatbot Status

Table 4.5.15. Users can activate and deactivate chatbots

Feature	Users can activate and deactivate chatbots
Use Case ID	UC-19
Priority	Medium
Use Case Cross Ref	UC-Manage-Chatbot-Status
DB Table ID	Chatbots
Dependent Classes	ChatbotController, StatusManager

© University of Gujrat

Secure API Credentials

Table 4.5.16. System securely stores integration API keys and tokens

Feature	System securely stores integration API keys and tokens
Use Case ID	UC-25
Priority	Low
Use Case Cross Ref	UC-Secure-Credentials
DB Table ID	APICredentials
Dependent Classes	EncryptionService, SecureVault

4.5.17 Consolidated Traceability Summary

The following table provides the complete traceability matrix linking all features to their Use Cases, UI Screens, and Database Tables:

Table 4.5.17. BotFizz Complete Requirements Traceability Matrix

Feature	UC-ID	UI Screen	Priority	DB Table(s)
User Registration	UC-1	UI-01 (Registration)	Highest	Users
User Login	UC-2	UI-02 (Login)	Highest	Users, Sessions
Password Recovery	UC-3	UI-03 (Forgot Pwd)	Medium	Users, PasswordResets
Create Chatbot	UC-5	UI-04 (Create Modal)	Highest	Chatbots
Build Flow	UC-6,7	UI-05 (Flow Builder)	Highest	Chatbots, Nodes
AI Config	UC-8	UI-06 (AI Panel)	Medium	ChatbotAIConfig
Upload Training	UC-9	UI-07 (AI Panel)	Medium	TrainingFiles
WhatsApp Integ.	UC-11	UI-08 (Integration)	Highest	Integrations
Instagram Integ.	UC-12	UI-09 (Integration)	Medium	Integrations
Website Widget	UC-13	UI-10 (Integration)	Medium	Chatbots
Deploy Bot	UC-20	UI-11 (Deploy)	Highest	Deployments
Generate QR	UC-21	UI-12 (Deploy)	Medium	Deployments

Analytics View	UC-18	UI-13 (Dashboard)	Medium	Analytics
Conv. History	UC-17	UI-14 (Dashboard)	Medium	Conversations
Manage Status	UC-19	UI-15 (Dashboard)	Medium	Chatbots
Secure Creds	UC-25	UI-16 (Settings)	Low	APICredentials

© University of Gujrat

Chapter 5: Software Testing

5.1. Introduction

This deliverable follows IEEE Software Test Documentation Standard 829-1998. The standard describes basic test documents associated with the dynamic aspects of software testing. It is applicable to commercial software on any digital computer regardless of size or criticality.

The following standard artifacts are included:

- Test Plan
- Test Design Specification
- Test Case Specification
- Test Procedure Specification
- Test Item Transmittal Report
- Test Log
- Test Incident Report
- Test Summary Report

5.2. Test Plan

5.2.1. Purpose

To prescribe the scope, approach, resources, and schedule of testing. To identify items being tested, features to test, tasks to perform, persons responsible, and risks associated with this plan.

5.2.2. Outline

A test plan shall have the following structure:

- Test Plan Identifier
- Introduction
- Test Items
- Features to be Tested
- Features Not to be Tested
- Approach
- Pass/Fail Criteria
- Suspension/Resumption
- Test Deliverables
- Testing Tasks
- Environmental Needs
- Responsibilities
- Staffing
- Schedule
- Risks
- Approvals.

5.2.2.1. Test Plan Identifier

Plan ID	BotFizz-TP-001
Version	1.0
Date	April 2026
Prepared By	M. Hanzala Zaheer & M. Waleed Shams
Supervised By	Dr. Hafsa Sharif Dar

5.2.2.2. Introduction

BotFizz v1.0 is a web-based AI chatbot builder for SMEs. It allows non-technical users to design, train, and deploy chatbots on WhatsApp, Instagram, and website channels. This plan covers testing of the complete functional prototype from unit level through UAT.

5.2.2.3. Test Items

Item ID	Component	Version	References
TI-01	User Authentication Module	1.0	UC-1, UC-2, UC-3
TI-02	Chatbot Flow Builder (Frontend)	1.0	UC-5, UC-6, UC-7
TI-03	Backend REST API (Node.js/Express)	1.0	All UC
TI-04	PostgreSQL Database Layer	1.0	Data Model – Ch.3
TI-05	AI Integration (OpenAI API)	1.0	UC-8, UC-9, UC-15
TI-06	WhatsApp Integration (Meta Cloud API)	1.0	UC-11, UC-14
TI-07	Instagram Integration (Meta Graph API)	1.0	UC-12, UC-14
TI-08	Website Widget (Embeddable JS)	1.0	UC-13, UC-14
TI-09	Analytics Dashboard	1.0	UC-17, UC-18
TI-10	Deployment Module	1.0	UC-20

5.2.2.4. Features to be Tested

- User registration with email verification and encrypted password storage
- Login / logout with JWT session management
- Chatbot creation, flow building (add / connect / delete nodes)
- AI intent recognition and fallback response handling
- WhatsApp API credential validation and webhook activation
- Instagram API credential validation and webhook subscription
- Website widgets embed code generation
- Bot deployment
- Real-time message processing and AI-driven reply
- Conversation logging with filters; analytics metric computation

5.2.2.5. Features Not to be Tested

- SMS channel integration — Phase II; not yet implemented
- Automated billing — Phase III; out of scope
- Native mobile app — Phase III; out of scope
- CRM integration — Phase III; out of scope

5.2.2.6. Approach

Testing is conducted in four sequential phases:

1. **Unit Testing:** individual functions tested in isolation using Jest.
2. **Integration Testing:** API endpoints tested via Postman/Supertest; third-party APIs tested with sandbox accounts.
3. **System Testing:** End-to-end flows executed on staging server.
4. **UAT:** Real-scenario tasks performed by two simulated SME users)

5.2.2.7. Item Pass / Fail Criteria

PASS: all associated test cases execute and actual results match expected results.

FAIL: any Severity-1 (Critical) or Severity-2 (Major) deviation. Severity-3 cosmetic issues are logged but do not block release.

5.2.2.8. Suspension / Resumption

Suspension Criteria: If any of the selected tests do not produce the expected result or if there are any network or hardware problems, testing will be stopped until the issue is fixed. **Resumption Criteria:** Testing will be done again to ensure that the bug has been fixed and the selected text gives the expected results. Only then the testing will be resumed.

5.2.2.9. Test Deliverables

- Test Plan
- Test Design Specifications
- Test Case Specifications
- Test Procedure Specifications
- Test Item Transmittal Reports
- Test Logs
- Test Incident Reports

- Test Summary Report

5.2.2.10. Testing Tasks

Task	Description	Dependency	Assignee
T1	Set up staging environment (Ubuntu VPS)	None	M. Hanzala
T2	Unit test authentication API endpoints	T1	M. Hanzala
T3	Unit test flow builder components	T1	M. Waleed
T4	Integration test WhatsApp / Instagram webhooks	T2, T3	M. Hanzala
T5	Integration test AI intent recognition	T2	M. Hanzala
T6	System test full deployment workflow	T4, T5	M. Hanzala & M. Waleed
T7	UAT with simulated business user scenarios	T6	M. Waleed
T8	Document results and issue incident reports	All	Both

5.2.2.11. Environmental Needs

Component	Specification
Staging Server	Ubuntu 22.04 LTS VPS (Digital Ocean – 2 vCPU, 2 GB RAM)
Node.js	18.x LTS
PostgreSQL	15.x (separate test database instance)
Browser	Google Chrome 120+ / Firefox 118+
API Testing	Postman v10+
Unit Testing	Jest 29.x (backend), React Testing Library 14.x (frontend)

Component	Specification
WhatsApp Sandbox	Meta Business Test Sandbox account
Instagram Sandbox	Meta Developer Test User account
OpenAI API	Development-tier key with usage cap

5.2.2.12. Responsibilities

Role	Person	Responsibility
Backend Test Lead	M. Hanzala Zaheer	API, DB, WhatsApp, Instagram, AI integration testing
Frontend Test Lead	M. Waleed Shams	React component, flow builder, UI/UX testing
UAT Coordinator	M. Waleed Shams	Scenario design, user guidance, feedback collection
Supervisor / Approver	Dr. Hafsa Sharif Dar	Review and sign-off on test summary report

5.2.2.13. Staffing and Training Needs

Both team members are proficient in Node.js, React.js, Jest, and Postman. No additional training is required. Familiarity with Meta Developer Dashboard is assumed for integration testing.

5.2.2.14. Schedule

Milestone	Start	End	Duration
Environment Setup	01-Apr-2026	01-Apr-2026	1 day
Unit Testing (Auth + Builder)	02-Apr-2026	03-Apr-2026	2 days
Integration Testing	04-Apr-2026	06-Apr-2026	3 days
System Testing	07-Apr-2026	08-Apr-2026	2 days
UAT	09-Apr-2026	10-Apr-2026	2 days
Incident Resolution & Reporting	11-Apr-2026	12-Apr-2026	2 days

5.2.2.15. Risks and Contingencies

Risk	Impact	Contingency
Meta API outage during integration test	High – blocks WhatsApp/Instagram testing	Use mock server (Mockoon) to simulate webhook responses
OpenAI rate limit exceeded during testing	Medium – slows AI test cases	Implement exponential back-off; reduce batch size
Staging server downtime	High – blocks all testing	Maintain localhost fallback environment for critical tests
Third-party API policy change	Medium – may invalidate test credentials	Monitor Meta changelog; update test accounts promptly

5.2.2.16. Approvals

Project Supervisor	Dr. Hafsa Sharif Dar Signature: _____ Date: _____

5.3. Test Design Specification

5.3.1. Purpose

To prescribe the scope, approach, resources, and schedule of testing activities. Specifies the features covered, test cases to apply, and pass/fail criteria for each design group.

5.3.2. Outline

- Test Plan Identifier
- Introduction
- Test Items
- Features to be Tested
- Features Not to be Tested
- Approach
- Pass/Fail Criteria
- Suspension/Resumption
- Test Deliverables
- Testing Tasks

- Environmental Needs
- Responsibilities
- Staffing
- Schedule
- Risks
- Approvals.

5.3.2.1. Test Design Identifier: TDS-01: User Authentication

Design ID	TDS-01
Feature Under Test	User Authentication (TI-01): Register, Login, Session, Recovery
Technique	Black-box equivalence partitioning and boundary value analysis
Test Cases	TC-01 through TC-06
Pass Criteria	All auth flows respond per UC-1, UC-2, UC-3 expected outputs
UC References	UC-1 (Register), UC-2 (Login), UC-3 (Password Recovery)

5.3.2.2. Test Design Identifier: TDS-02: Chatbot Flow Builder

Design ID	TDS-02
Feature Under Test	Flow Builder (TI-02): Create bot, add nodes, connect, save, test
Technique	Functional testing; state-based testing for canvas interactions
Test Cases	TC-07 through TC-12
Pass Criteria	Node operations and flow persistence work per UC-5, UC-6, UC-7
UC References	UC-5 (Create Bot), UC-6 (Build Flow), UC-7 (Add Node)

5.3.2.3. Test Design Identifier: TDS-03: AI and Integration

Design ID	TDS-03
Feature Under Test	AI config, training upload, WhatsApp, Instagram, Widget (TI-05-08)
Technique	Integration testing with sandbox credentials; mock server for offline runs
Test Cases	TC-13 through TC-20
Pass Criteria	Credentials validate, webhooks activate, messages flow per UC-11, UC-12, UC-14, UC-15
UC References	UC-8 (AI Config), UC-9 (Training), UC-11 (WA), UC-12 (IG), UC-13 (Widget)

5.3.2.4. Test Design Identifier: TDS-04: Deployment and Analytics

Design ID	TDS-04
Feature Under Test	Bot Deployment (TI-10) and Analytics Dashboard (TI-09)
Technique	End-to-end system testing covering deploy → conversation → analytics pipeline
Test Cases	TC-21 through TC-26
Pass Criteria	Bot deploys, conversations are logged, and metrics are computed correctly per UC-17, UC-18, UC-20
UC References	UC-20 (Deploy), UC-17 (Conversations), UC-18 (Analytics)

5.4. Test Case Specification

5.4.1. Purpose

The primary purpose of a Test Case is to provide a specific, repeatable set of instructions to verify that a particular feature of your software works exactly as intended.

5.4.2. Outline

A test case specification shall have the following structure:

- Test Case Identifier
- Test Items
- Input Specifications
- Output Specifications
- Environmental Needs
- Special Procedural Requirements
- Inter-Case Dependencies

5.4.2.1. Test Cases TDS-01: User Authentication

TC-01: Valid User Registration

Test Case ID	TC-01
Test Case Name	Valid User Registration
Test Case Developer	M. Hanzala Zaheer
Date	02 April 2026
Design Reference	TDS-01
UC Reference	UC-1 (Register User)
Test Item	User Authentication Module (TI-01)
Input	Email: test@smebiz.com | Full Name: Ahmed Khan | Org: Khan Traders | Password: Test@1234
Expected Output	HTTP 201 Created; user record inserted in USER table; verification email sent within 60 s
Environmental Needs	Staging server; PostgreSQL test DB; SMTP test account (Gmail sandbox)
Special Procedure	Check inbox for verification email after form submission; reveive OTP; confirm redirect to Login
Inter-Case Dependency	Must pass before TC-05 (Login) can run
Pass / Fail	Pass

TC-02: Duplicate Email Registraion

Test Case ID	TC-02
Test Case Name	Duplicate Email Registration
Test Case Developer	M. Hanzala Zaheer
Date	02 April 2026
Design Reference	TDS-01
UC Reference	UC-1 (Register User) Alternative Flow
Test Item	User Authentication Module (TI-01)
Input	Email already registered: test@smebiz.com (same as TC-01)

Expected Output	HTTP 409 Conflict; response body: 'Email already exists'; no new DB record
Environmental Needs	Staging server: TC-01 must have been executed first
Special Procedure	Re-submit registration form with same email used in TC-01
Inter-Case Dependency	TC-01 must have run (email must exist in DB)
Pass / Fail	Pass

TC-03: Invalid Email Format

Test Case ID	TC-03
Test Case Name	Invalid Email Format
Test Case Developer	M. Waleed Shams
Date	02 April 2026
Design Reference	TDS-01
UC Reference	UC-1 - Alternative Flow
Test Item	User Authentication Module (TI-01)
Input	Email: notanemail | Password: Test@1234
Expected Output	HTTP 400 Bad Request; field-level error: 'Enter a valid email address'
Environmental Needs	Staging server
Special Procedure	Attempt form submission with malformed email; observe inline validation
Inter-Case Dependency	None
Pass / Fail	Pass

TC-04: Weak Password Rejection

Test Case ID	TC-04
Test Case Name	Weak Password Rejection
Test Case Developer	M. Waleed Shams
Date	02 April 2026
Design Reference	TDS-01
UC Reference	UC-1- Alternative Flow
Test Item	User Authentication Module (TI-01)
Input	Email: new@test.com | Password: 123 (3 characters)
Expected Output	HTTP 400; error: 'Password must be at least 8 characters'
Environmental Needs	Staging server
Special Procedure	Submit form; observe password strength indicator and error message
Inter-Case Dependency	None
Pass / Fail	Pass

TC-05: Valid Login with Verified Account

Test Case ID	TC-05
Test Case Name	Valid Login with Verified Account
Test Case Developer	M. Hanzala Zaheer
Date	02 April 2026
Design Reference	TDS-01
UC Reference	UC-2 (Login User)
Test Item	User Authentication Module (TI-01)
Input	Email: test@smebiz.com | Password: Test@1234 (account verified via TC-01)
Expected Output	HTTP 200; JWT token returned; redirect to Dashboard; SESSION record created in DB
Environmental Needs	Staging server: TC-01 verification step completed

© University of Gujrat

Special Procedure	Navigate to Login page; enter credentials; click Login button
Inter-Case Dependency	TC-01 (account must exist and be verified)
Pass / Fail	Pass

TC-06: Login with Wrong Password

Test Case ID	TC-06
Test Case Name	Login with Wrong Password
Test Case Developer	M. Hanzala Zaheer
Date	02 April 2026
Design Reference	TDS-01
UC Reference	UC-2- Alternative Flow
Test Item	User Authentication Module (TI-01)
Input	Email: test@smebiz.com | Password: WrongPass999
Expected Output	HTTP 401 Unauthorized; message: 'Invalid credentials'; no session token created
Environmental Needs	Staging server: TC-01 account must exist
Special Procedure	Enter correct email but wrong password; click Login
Inter-Case Dependency	TC-01
Pass / Fail	Pass

© University of Gujrat

5.4.2.2. Test Cases TDS-02: Chatbot Flow Builder

TC-07: Create New Chatbot Project

Test Case ID	TC-07
Test Case Name	Create New Chatbot Project
Test Case Developer	M. Waleed Shams
Date	03 April 2026
Design Reference	TDS-02
UC Reference	UC-5 (Create Chatbot)
Test Item	Chatbot Flow Builder (TI-02)
Input	Bot Name: 'FAQ Bot' | Description: 'Customer FAQ bot' | Language: English
Expected Output	HTTP 201; bot record with unique UUID inserted in CHATBOT table; user redirected to Flow Builder canvas
Environmental Needs	Staging server: logged-in session (TC-05 must pass)
Special Procedure	Click '+ Create New Chatbot' from Dashboard; fill modal; click Create
Inter-Case Dependency	TC-05 (must be logged in)
Pass / Fail	Pass

TC-08: Add Message Node to Canvas

Test Case ID	TC-08
Test Case Name	Add Message Node to Canvas
Test Case Developer	M. Waleed Shams
Date	03 April 2026
Design Reference	TDS-02
UC Reference	UC-7 (Add Node)
Test Item	Chatbot Flow Builder (TI-02)
Input	Drag 'Message Node' from palette onto canvas; enter text: 'Welcome to our support bot!'

Expected Output	Node appears on canvas; node ID generated; NODE record inserted in DB; right panel shows config
Environmental Needs	Staging server; Bot Builder open (TC-07 complete)
Special Procedure	Drag node; double-click to open properties panel; type message; click outside to save
Inter-Case Dependency	TC-07
Pass / Fail	Pass

TC-09: Connect Two Nodes

Test Case ID	TC-09
Test Case Name	Connect Two Nodes
Test Case Developer	M. Waleed Shams
Date	03 April 2026
Design Reference	TDS-02
UC Reference	UC-6 (Build Flow)
Test Item	Chatbot Flow Builder (TI-02)
Input	Source: Message Node (TC-08); Target: a newly dragged Question Node
Expected Output	Directed animated edge drawn; CONNECTION record inserted in DB with source_node_id and target_node_id
Environmental Needs	Staging server; both nodes on canvas
Special Procedure	Hover over source output handle; drag to target input handle; release
Inter-Case Dependency	TC-08
Pass / Fail	Pass

TC-10: Save Flow Successfully

Test Case ID	TC-10
Test Case Name	Save Flow Successfully
Test Case Developer	M. Waleed Shams
Date	03 April 2026
Design Reference	TDS-02
UC Reference	UC-6 (Build Flow)
Test Item	Chatbot Flow Builder (TI-02)
Input	Canvas with 2 connected nodes from TC-09; click 'Save Flow' button
Expected Output	HTTP 200; FLOW record updated in DB; toast notification: 'Flow saved successfully'
Environmental Needs	Staging server
Special Procedure	Click Save Flow in toolbar; observe toast; verify DB record updated
Inter-Case Dependency	TC-09
Pass / Fail	Pass

TC-11: Test Bot in Sandbox

Test Case ID	TC-11
Test Case Name	Test Bot in Sandbox
Test Case Developer	M. Waleed Shams
Date	03 April 2026
Design Reference	TDS-02
UC Reference	UC-6-Extend: Simulate/Test Bot in Sandbox
Test Item	Chatbot Flow Builder (TI-02)
Input	Click 'Test Bot'; type 'Hello' in sandbox chat panel
Expected Output	Bot replies with the Message Node text: 'Welcome to our support bot!'
Environmental Needs	Staging server; saved flow from TC-10

© University of Gujrat

Special Procedure	Open sandbox panel; send message; observe bot reply
Inter-Case Dependency	TC-10
Pass / Fail	Pass

TC-12: Deploy Blocked on Invalid Flow (No Start Node)

Test Case ID	TC-12
Test Case Name	Deploy Blocked on Invalid Flow (No Start Node)
Test Case Developer	M. Hanzala Zaheer
Date	04 April 2026
Design Reference	TDS-02
UC Reference	UC-20-Alternative Flow (validation failure)
Test Item	Chatbot Flow Builder (TI-02) + Deployment Module (TI-10)
Input	Canvas with orphan nodes only (no node connected to Start); click Deploy
Expected Output	Validation error displayed: 'Flow must have a valid start node'; deployment blocked
Environmental Needs	Staging server; bot with invalid flow
Special Procedure	Create bot; add 2 disconnected nodes; click Deploy; observe error
Inter-Case Dependency	TC-07
Pass / Fail	Pass

5.4.2.3. Test Cases — TDS-03: AI and Integration

TC-13: Enable AI Intent Recognition

Test Case ID	TC-13
Test Case Name	Enable AI Intent Recognition
Test Case Developer	M. Hanzala Zaheer
Date	05 April 2026
Design Reference	TDS-03
UC Reference	UC-8 (Configure AI)
Test Item	AI Integration Module (TI-05)
Input	Toggle AI enabled ON; set confidence threshold: 0.75; configure fallback: 'I did not understand, please rephrase.'
Expected Output	AI_CONFIG record created in DB; isEnabled = true; confidence Threshold = 0.75; fallback Response saved
Environmental Needs	Staging server; valid OpenAI API key in. env
Special Procedure	Navigate to AI settings tab inside Bot Builder; toggle AI; set slider; enter fallback; click Save
Inter-Case Dependency	TC-07
Pass / Fail	Pass

TC-14: Upload Training Data Document

Test Case ID	TC-14
Test Case Name	Upload Training Data Document
Test Case Developer	M. Hanzala Zaheer
Date	05 April 2026
Design Reference	TDS-03
UC Reference	UC-9 (Upload Training Data)
Test Item	AI Integration Module (TI-05)
Input	Upload file: FAQ document.pdf (2 MB; contains product FAQs)

Expected Output	HTTP 200; TRAINING DATA record created; file content extracted and stored; status: 'Training Complete'
Environmental Needs	Staging server: TC-13 must have run (AI CONFIG must exist)
Special Procedure	Click 'Upload Training Data'; select PDF; wait for processing indicator to clear
Inter-Case Dependency	TC-13
Pass / Fail	Pass

TC-15: Connect WhatsApp Business API

Test Case ID	TC-15
Test Case Name	Connect WhatsApp Business API
Test Case Developer	M. Hanzala Zaheer
Date	05 April 2026
Design Reference	TDS-03
UC Reference	UC-11 (Connect WhatsApp)
Test Item	WhatsApp Integration (TI-06)
Input	Business Account ID: 123456789 | API Key: (Meta sandbox key) | Phone Number ID: 987654321
Expected Output	Credentials validated; webhook registered with Meta; INTEGRATION record status = 'active'; 'Connected' badge shown
Environmental Needs	Staging server; Meta WhatsApp Business test sandbox account; public webhook URL (ngrok or VPS)
Special Procedure	Open WhatsApp Wizard; enter sandbox credentials; click 'Test Connection'; observe status badge
Inter-Case Dependency	TC-10 (bot must have a saved flow)
Pass / Fail	Pass

TC-16: Invalid WhatsApp API Key Rejected

Test Case ID	TC-16
Test Case Name	Invalid WhatsApp API Key Rejected
Test Case Developer	M. Hanzala Zaheer
Date	05 April 2026
Design Reference	TDS-03
UC Reference	UC-11- Alternative Flow
Test Item	WhatsApp Integration (TI-06)
Input	Business Account ID: 123456789 | API Key: INVALID_KEY_XYZ | Phone Number ID: 987654321
Expected Output	HTTP 401 from Meta API proxied to frontend; status badge: 'Error'; error message: 'Invalid API credentials'
Environmental Needs	Staging server
Special Procedure	Enter deliberately wrong API key; click Test Connection; observe error badge
Inter-Case Dependency	TC-07
Pass / Fail	Pass

TC-17: Connect Instagram Messaging API

Test Case ID	TC-17
Test Case Name	Connect Instagram Messaging API
Test Case Developer	M. Hanzala Zaheer
Date	05 April 2026
Design Reference	TDS-03
UC Reference	UC-12 (Connect Instagram)
Test Item	Instagram Integration (TI-07)
Input	Page ID: 111222333 | Access Token: (Meta test long-lived token with messaging permissions)
Expected Output	Webhook subscription created with Instagram Graph API; INTEGRATION record status = 'active'

© University of Gujrat

Environmental Needs	Staging server; Meta Developer test user account with Instagram Business page
Special Procedure	Open Instagram Wizard; enter Page ID and token; click Connect; observe Connected badge
Inter-Case Dependency	TC-10
Pass / Fail	Pass

TC-18: Process Incoming WhatsApp Message -AI Reply

Test Case ID	TC-18
Test Case Name	Process Incoming WhatsApp Message -AI Reply
Test Case Developer	M. Hanzala Zaheer
Date	06 April 2026
Design Reference	TDS-03
UC Reference	UC-14 (Process Message), UC-15 (Recognize Intent)
Test Item	WhatsApp Integration (TI-06), AI Module (TI-05)
Input	End-user sends 'What are your working hours?' to the WhatsApp test number
Expected Output	Webhook fires; OpenAI analyzes intent; matching flow node response returned; reply delivered to user via Meta API; MESSAGE record logged in DB
Environmental Needs	Deployed bot (TC-15 complete); WhatsApp test number active
Special Procedure	Send message from test WhatsApp number; verify reply received; check CONVERSATION and MESSAGE tables in DB
Inter-Case Dependency	TC-15
Pass / Fail	Pass

TC-19: Fallback Response on Low AI Confidence

Test Case ID	TC-19
Test Case Name	Fallback Response on Low AI Confidence
Test Case Developer	M. Hanzala Zaheer
Date	06 April 2026
Design Reference	TDS-03
UC Reference	UC-16 (Provide Fallback)
Test Item	AI Integration Module (TI-05)
Input	End-user sends: 'xjqpwz mnbv' (random unrecognisable string) to bot
Expected Output	AI confidence < configured threshold (0.75); fallback message sent: 'I did not understand, please rephrase.'
Environmental Needs	Staging server: TC-13 (AI enabled with fallback configured); deployed bot
Special Procedure	Send nonsense string; verify fallback text received by user
Inter-Case Dependency	TC-13, TC-18
Pass / Fail	Pass

TC-20: Generate Website Widget Embed Code

Test Case ID	TC-20
Test Case Name	Generate Website Widget Embed Code
Test Case Developer	M. Waleed Shams
Date	06 April 2026
Design Reference	TDS-03
UC Reference	UC-13 (Generate Widget)
Test Item	Website Widget (TI-08)
Input	Colour: #2E75B6 | Position: bottom-right | Welcome message: 'Hi! How can I help?'
Expected Output	HTTP 200; embed script tag generated with correct widget_id and customisation params; embed code shown in UI

© University of Gujrat

Environmental Needs	Staging server; saved bot (TC-10)
Special Procedure	Open Website Widget config; set options; click Generate; copy embed code; paste in blank HTML file; verify widget loads
Inter-Case Dependency	TC-07
Pass / Fail	Pass

5.4.2.4. Test Cases — TDS-04: Deployment and Analytics

TC-21: Successful Bot Deployment to WhatsApp

Test Case ID	TC-21
Test Case Name	Successful Bot Deployment to WhatsApp
Test Case Developer	M. Hanzala Zaheer
Date	07 November 2025
Design Reference	TDS-04
UC Reference	UC-20 (Deploy Chatbot)
Test Item	Deployment Module (TI-10)
Input	Bot with valid saved flow + active WhatsApp integration; click 'Deploy'
Expected Output	Bot status updated to 'Active' in chatbot table; deploy history record created; success screen displayed
Environmental Needs	Staging server; TC-10 and TC-15 complete
Special Procedure	Click Deploy; confirm deployment modal; verify status badge on Dashboard changes to 'Active'
Inter-Case Dependency	TC-10, TC-15
Pass / Fail	Pass

TC-22: Deploy Blocked Without Active Integration

Test Case ID	TC-22
Test Case Name	Deploy Blocked Without Active Integration
Test Case Developer	M. Hanzala Zaheer
Date	07 April 2026
Design Reference	TDS-04
UC Reference	UC-20-Alternative Flow
Test Item	Deployment Module (TI-10)
Input	Bot with valid flow but no integrations configured; click Deploy
Expected Output	Deployment blocked; error: 'No active integration found. Please connect at least one channel.'
Environmental Needs	Staging server; bot with flow only (no integration)
Special Procedure	Create bot; save flow; attempt Deploy without completing any integration wizard
Inter-Case Dependency	TC-10
Pass / Fail	Pass

TC-23: Deactivate Active Bot

Test Case ID	TC-23
Test Case Name	Deactivate Active Bot
Test Case Developer	M. Waleed Shams
Date	07 April 2026
Design Reference	TDS-04
UC Reference	UC-19 (Manage Chatbot Status)
Test Item	Deployment Module (TI-10)
Input	Active bot from TC-21; click 'Deactivate' toggle on Dashboard card
Expected Output	Bot status = 'Inactive' in DB; webhook deactivated; incoming messages no longer processed

© University of Gujrat

Environmental Needs	Staging server: TC-21 (bot must be Active)
Special Procedure	From Dashboard, click Deactivate on bot card; confirm; send test WA message; verify no reply received
Inter-Case Dependency	TC-21
Pass / Fail	Pass

TC-24: View Conversation History with Date Filter

Test Case ID	TC-24
Test Case Name	View Conversation History with Date Filter
Test Case Developer	M. Waleed Shams
Date	08 April 2026
Design Reference	TDS-04
UC Reference	UC-16 (View Conversations)
Test Item	Analytics Dashboard (TI-09)
Input	Filter: Platform = WhatsApp; Date Range = today; Bot = FAQ Bot
Expected Output	Filtered list shows only today's WhatsApp conversations; full message thread accessible on click
Environmental Needs	Staging server: TC-18 must have created conversation records
Special Procedure	Go to Analytics; apply filters; click a conversation row; verify full thread displayed
Inter-Case Dependency	TC-18
Pass / Fail	Pass

TC-25: Analytics Metrics Accuracy

Test Case ID	TC-25
Test Case Name	Analytics Metrics Accuracy
Test Case Developer	M. Waleed Shams
Date	08 April 2026
Design Reference	TDS-04
UC Reference	UC-18 (View Analytics)
Test Item	Analytics Dashboard (TI-09)
Input	Bot with 10 conversations (9 received AI replies, 1 triggered fallback)
Expected Output	Total Conversations = 10; Response Rate = 90.00%; metrics match DB records exactly
Environmental Needs	Staging server; pre-seeded test conversation data
Special Procedure	Check DB for accurate counts; compare with displayed dashboard values
Inter-Case Dependency	TC-18, TC-19
Pass / Fail	Pass

TC-26: Platform Distribution Chart Accuracy

Test Case ID	TC-26
Test Case Name	Platform Distribution Chart Accuracy
Test Case Developer	M. Waleed Shams
Date	08 April 2026
Design Reference	TDS-04
UC Reference	UC-18 (View Analytics)
Test Item	Analytics Dashboard (TI-09)
Input	Bot active on WhatsApp (6 conversations) and Website Widget (4 conversations)
Expected Output	Pie chart shows WhatsApp = 60%, Website = 40%; percentages sum to 100%

Environmental Needs	Staging server; pre-seeded platform-split conversation data
Special Procedure	Navigate to Analytics; view Platform Distribution chart; verify segment percentages
Inter-Case Dependency	TC-24
Pass / Fail	Pass

5.5. Test Procedure Specification

5.5.1. Purpose

To specify the steps for executing a set of test cases or analysing a software item to evaluate a set of features.

5.5.2. Outline

A test procedure specification shall have the following structure:

- Test Procedure Identifier
- Purpose
- Special Requirements
- Procedure Steps

Procedure TP-01 — User Registration End-to-End (TC-01)

Step	Action	Checkpoint
Set Up	Open Chrome; clear cookies; navigate to https://botfizz.com/register	Registration form loads within 3 s
1	Enter Full Name: Ahmed Khan	Field accepts input
2	Enter Email: test@smebiz.com	Green border- format valid
3	Enter Password: Test@1234	Strength indicator: Strong
4	Click 'Create Account'	Loading spinner; success notification
5	Open email inbox; receive OTP	Come back to Login page with OTP 'Account verified'
Measure	Record time from form submit to email receipt	Must be < 60 s

Step	Action	Checkpoint
Wrap Up	Log result; capture screenshot of success page	Evidence stored in test log

Procedure TP-02 — Build and Deploy Flow (TC-07 through TC-21)

Step	Action	Checkpoint
Set Up	Log in as test@smebiz.com (TC-05 verified)	Dashboard loads
1	Click '+ Create New Chatbot'; enter details; click Create	Redirect to empty flow canvas
2	Drag Message Node; enter: 'Welcome to our support bot!'	Node visible on canvas
3	Drag Question Node; connect Message → Question	Animated edge drawn
4	Click Save Flow	Toast: 'Flow saved successfully'
5	Go to AI tab; enable AI; set threshold 0.75; save	AI_CONFIG record visible in DB
6	Go to Integrations; enter WhatsApp sandbox credentials; test	Status badge: Connected
7	Click Deploy Bot; confirm	Bot status: Active; QR code displayed
Measure	Record deploy time from click to QR display	Must be < 10 s
Wrap Up	Send test WA message; verify bot replies, capture screenshots	Evidence stored

5.6. Test Item Transmittal Report

5.6.1. Purpose

To identify the test items being transmitted for testing, the persons responsible, their physical location, and status. Variations from current requirements are noted.

5.6.2. Outline

A test item transmittal report shall have the following structure:

- Transmittal report identifier
- Transmitted items
- Location
- Status
- Approvals

Details on the content of each section are contained in the following sub clauses.

5.6.2.1. Transmittal Report Identifier: BotFizz-TITR-001

5.6.2.2. Transmitted Items

Item ID	Component	Version	Responsible Person
TI-01	User Authentication Module	1.0	M. Hanzala Zaheer
TI-02	Chatbot Flow Builder	1.0	M. Waleed Shams
TI-03	Backend REST API	1.0	M. Hanzala Zaheer
TI-04	PostgreSQL Database Layer	1.0	M. Hanzala Zaheer
TI-05	AI Integration Module	1.0	M. Hanzala Zaheer
TI-06	WhatsApp Integration	1.0	M. Hanzala Zaheer
TI-07	Instagram Integration	1.0	M. Hanzala Zaheer
TI-08	Website Widget	1.0	M. Waleed Shams
TI-09	Analytics Dashboard	1.0	M. Waleed Shams
TI-10	Deployment Module	1.0	M. Hanzala Zaheer

5.6.2.3. Location

All items reside on the project are private. Staging deployment runs on the Digital Ocean VPS (Ubuntu 22.04). Test database is isolated from production.

5.6.2.4. Status

All ten components pass their respective unit tests at time of transmittal. No outstanding incident reports at transmittal date. Database migration scripts run successfully on staging server.

5.6.2.5. Approvals

Supervisor	Dr. Hafsa Sharif Dar Signature: _____ Date: _____

5.7. Test Log

5.7.1. Purpose

The purpose of test log is to record all relevant details about the test cases executed including the output received against each test case. The log represents the observed results of the system for the tests conducted.

5.7.2. Outline - Activity and Event Entries

Log ID	Date	Time	Tester	TC ID	Procedure	Result	Remarks
TL-01	02-Nov-25	09:00	M. Hanzala	TC-01	TP-01	PASS	Email delivered in 28 s
TL-02	02-Nov-25	09:20	M. Hanzala	TC-02	TP-01	PASS	409 returned correctly
TL-03	02-Nov-25	09:35	M. Waleed	TC-03	TP-01	PASS	Inline validation triggered
TL-04	02-Nov-25	09:50	M. Waleed	TC-04	TP-01	PASS	Error shown immediately
TL-05	02-Nov-25	10:00	M. Hanzala	TC-05	TP-01	PASS	JWT valid; dashboard loaded
TL-06	02-Nov-25	10:15	M. Hanzala	TC-06	TP-01	PASS	401 returned; no session
TL-07	03-Nov-25	09:00	M. Waleed	TC-07	TP-02	PASS	UUID generated; redirect OK

Log ID	Date	Time	Tester	TC ID	Procedure	Result	Remarks
TL-08	03-Nov-25	09:20	M. Waleed	TC-08	TP-02	PASS	Node rendered in 85 ms
TL-09	03-Nov-25	09:40	M. Waleed	TC-09	TP-02	PASS	Edge drawn and saved
TL-10	03-Nov-25	10:00	M. Waleed	TC-10	TP-02	PASS	Flow persisted to DB
TL-11	03-Nov-25	10:20	M. Waleed	TC-11	TP-02	PASS	Sandbox reply in 1.2 s
TL-12	04-Nov-25	09:00	M. Hanzala	TC-12	TP-02	PASS	Validation error shown
TL-13	05-Nov-25	09:00	M. Hanzala	TC-13	TP-02	PASS	AI_CONFIG created
TL-14	05-Nov-25	09:30	M. Hanzala	TC-14	TP-02	PASS	PDF processed in 4.1 s
TL-15	05-Nov-25	10:00	M. Hanzala	TC-15	TP-02	PASS	WA webhook registered
TL-16	05-Nov-25	10:30	M. Hanzala	TC-16	TP-02	PASS	Error badge shown
TL-17	05-Nov-25	11:00	M. Hanzala	TC-17	TP-02	PASS	IG webhook subscribed
TL-18	06-Nov-25	09:00	M. Hanzala	TC-18	TP-02	PASS	AI reply in 2.1 s
TL-19	06-Nov-25	09:30	M. Hanzala	TC-19	TP-02	FAIL	Fallback not triggered at exactly 0.75 see IR-001

Log ID	Date	Time	Tester	TC ID	Procedure	Result	Remarks
TL-20	06-Nov-25	11:00	M. Hanzala	TC-19	TP-02	PASS	Fix applied; retest passed
TL-21	06-Nov-25	12:00	M. Waleed	TC-20	TP-02	PASS	Widget embed code correct
TL-22	07-Nov-25	09:00	M. Hanzala	TC-21	TP-02	PASS	Deploy in 3.4 s; QR shown
TL-23	07-Nov-25	09:30	M. Hanzala	TC-22	TP-02	PASS	Deploy blocked correctly
TL-24	07-Nov-25	10:00	M. Waleed	TC-23	TP-02	PASS	Bot deactivated; no reply
TL-25	08-Nov-25	09:00	M. Waleed	TC-24	TP-02	PASS	Filters work correctly
TL-26	08-Nov-25	09:30	M. Waleed	TC-25	TP-02	PASS	Metrics match DB values
TL-27	08-Nov-25	10:00	M. Waleed	TC-26	TP-02	FAIL	60/40 shown as 61/39 see IR-002
TL-28	08-Nov-25	11:30	M. Waleed	TC-26	TP-02	PASS	Fix applied; retest passed

5.8. Test Incident Report

5.8.1. Purpose

To document any event that occurs during testing that requires investigation.

5.8.2. Outline

Incident Report IR-001

Incident ID	IR-001
Date / Time	06 April 2026 / 09:30
Test Case	TC-19 (Fallback Response on Low AI Confidence)
Test Log Ref	TL-19
Summary	Fallback response not triggered when confidence score equals threshold exactly (0.75)
Inputs	User message: random nonsense string | Configured threshold: 0.75
Expected Result	Fallback response sent: 'I did not understand, please rephrase.'
Actual Result	Bot attempted to match a flow node instead of falling back; empty response delivered to user
Anomaly	Boundary condition defect threshold comparison used strict greater-than instead of greater-than-or-equal.
Environment	Staging server; OpenAI API; TC-13 configuration active
Testers	M. Hanzala Zaheer
Impact	Affects UC-16 (Provide Fallback); all messages at exact threshold would receive incorrect responses
Severity	Severity 2 Major
Resolution	Updated threshold comparison in AI processing service from > to >=; retest (TL-20) passed
Status	Resolved

Incident Report IR-002

Incident ID	IR-002
Date / Time	08 April 2026 / 10:00
Test Case	TC-26 (Platform Distribution Chart Accuracy)
Test Log Ref	TL-27
Summary	Platform distribution pie chart displayed 61/39 instead of correct 60/40 due to rounding error

© University of Gujrat

Inputs	6 WhatsApp conversations, 4 Website conversations (total: 10)
Expected Result	Chart: WhatsApp 60.00%, Website 40.00%
Actual Result	Chart: WhatsApp 61%, Website 39% (floating point truncation error)
Anomaly	Analytics service used Math.floor() instead of Math.round() for percentage formatting
Environment	Staging server; Analytics module (TI-09)
Testers	M. Waleed Shams
Impact	Affects UC-18 (View Analytics); minor visual inaccuracy in pie chart
Severity	Severity 3 Minor
Resolution	Replaced Math.floor() with parseFloat().toFixed(2) in analytics formatter; retest (TL-28) passed
Status	Resolved

5.9. Test Summary Report

5.9.1. Purpose

To summarise the results of the designated testing activities and provide evaluations based on these results.

5.9.2. Outline

A test summary report shall have the following structure:

- Test summary report identifier
- Summary
- Variances
- Comprehensive assessment
- Summary of results
- Evaluation
- Summary of activities
- Approvals

Details on the content of each section are contained in the following sub clauses.

5.9.2.1. Test Summary Report Identifier: BotFizz-TSR-001

5.9.2.2. Summary

Report ID	BotFizz-TSR-001
Version	1.0
Test Period	02 April 2026 – 10 April 2026
Environment	Ubuntu 22.04 VPS (Staging); Chrome 120; Meta Sandbox; OpenAI Dev tier
Items Tested	TI-01 through TI-10 (all 10 components)
Test Plans Referenced	BotFizz-TP-001
Transmittal Reports	BotFizz-TITR-001

5.9.2.3. Variances

All test cases were executed as specified. Two variances were recorded: (1) TC-19 initially failed due to an off-by-one boundary defect in the confidence threshold check. (2) TC-26 initially failed due to floating-point rounding in the analytics service. Both were resolved and retested within the same testing period.

5.9.2.4. Comprehensiveness Assessment

26 test cases were designed and executed covering all 10 test items and 20 use cases. All high-priority (Highest) and medium-priority features were tested. Low-priority features (SMS, billing, native app) were excluded per the agreed scope boundary in the Test Plan.

5.9.2.5. Summary of Results

Metric	Value
Total Test Cases Designed	26
Total Executions (including retests)	28
Tests Passed (first run)	24
Tests Failed (first run)	2
Tests Passed After Retest	2 (TC-19, TC-26)
Incidents Reported	2 (IR-001, IR-002)
Incidents Resolved	2 (100%)
Incidents Outstanding	0
Severity-1 (Critical) Defects	0

Metric	Value
Severity-2 (Major) Defects	1-Resolved (IR-001)
Severity-3 (Minor) Defects	1-Resolved (IR-002)

5.9.2.6. Evaluation

BotFizz v1.0 meets all specified functional requirements. Both defects found were minor boundary/rounding issues resolved quickly. No critical architectural or security defects were identified. The system is evaluated as READY FOR DEPLOYMENT.

5.9.2.7. Summary of Activities

Phase	Duration	Staff Days	Key Activities
Unit Testing	2 days	3 person-days	Auth API, flow builder components; 12 cases executed
Integration Testing	3 days	5 person-days	WhatsApp, Instagram, AI, Widget; 8 cases executed; IR-001 found and resolved
System Testing	2 days	3 person-days	End-to-end deploy workflow; 4 cases; IR-002 found and resolved
UAT	2 days	2 person-days	2 simulated SME users; 5 task scenarios; all completed

5.9.2.8. Approvals

Project Supervisor	Dr. Hafsa Sharif Dar Signature: _____ Date: _____

Chapter 6: User Technical Manual

6.1 Introduction

This manual explains how to use BotFizz, a web-based platform that lets small and medium business owners create, configure, and deploy AI-powered chatbots without writing a single line of code. Every major task you need to perform is described here with step-by-step instructions and screen illustrations.

6.1.1 Who This Manual Is For

This manual is written for business owners, marketing managers, and customer support staff who want to automate conversations on WhatsApp, Instagram, or their own website. No programming knowledge is required. If you can use a web browser, you can use BotFizz.

6.1.2 What BotFizz Can Do For You

BotFizz provides the following capabilities out of the box:

- Design chatbot conversation flows with a drag and drop canvas.
- Connect OpenAI or another AI provider to recognise customer intent automatically.
- Deploy your chatbot to WhatsApp Business, Instagram Messaging, or your website in a single click.
- Monitor real time analytics covering message delivery, read rates, and reply rates.
- Review full conversation histories and filter them by date, platform, or status.

6.1.3 How This Manual Is Organised

The manual follows the natural sequence of actions a new user performs when starting with BotFizz. The sections are as follows:

- Section 6.2 covers browser and device requirements before you begin.
- Section 6.3 explains how to create an account, verify your email with OTP, and log in.
- Section 6.4 introduces the main dashboard.
- Sections 6.5 and 6.6 show how to create chatbots and build conversation flows.
- Section 6.7 covers the AI configuration panel.
- Section 6.8 walks through connecting WhatsApp, Instagram, and Website Widget.
- Sections 6.9 to 6.12 cover deployment, management, conversations, and analytics.
- Section 6.13 provides a troubleshooting reference, and Section 6.14 is a glossary.

© University of Gujrat

6.2 System Requirements

6.2.1 Supported Browsers

BotFizz runs entirely in your web browser. No software installation is needed on your computer. The platform is tested and fully supported on the browsers listed below.

Browser	Minimum Version
Google Chrome	Version 110 or later (Recommended)
Mozilla Firefox	Version 110 or later
Microsoft Edge	Version 110 or later
Apple Safari	Version 16 or later

WARNING: Internet Explorer is not supported. Please use one of the browsers listed above for the best experience.

6.2.2 Hardware and Network Requirements

BotFizz requires a standard internet connection and a desktop or laptop computer. Tablets may be used for viewing dashboards but the Flow Builder canvas works best on a larger screen.

Requirement	Minimum Specification
Screen Resolution	1280 x 720 pixels or higher
RAM	4 GB (8 GB recommended for Flow Builder)
Internet Speed	Minimum 1 Mbps download, 200 kbps upload
Operating System	Windows 10 or later, macOS 12 or later, Ubuntu 20.04 or later

6.2.3 External Account Requirements

Before you start using certain integrations, you will need accounts with third party services. These are only required if you want to use the relevant integration:

- WhatsApp Business API: A verified Meta Business account with a WhatsApp Business number.
- Instagram Messaging: An Instagram Business or Creator account connected to a Meta Page.
- AI Features: An OpenAI account with a valid API key (or another supported AI provider).

6.3 Account Management

6.3.1 Creating a New Account (Registration)

To get started with BotFizz, you must first create a free account. Follow the steps below.

1. Open your web browser and navigate to the BotFizz website.
2. On the landing page, click the Get Started Free button. The system will take you to the registration form.
3. Fill in all required fields: Full Name, Email Address, Organization Name, Password, and Confirm Password.
4. Click the Create Account button at the bottom of the form.
5. The system will validate your information, create your account, and send a 6 digit One Time Password (OTP) to your email address.
6. You will be automatically taken to the Email Verification screen.

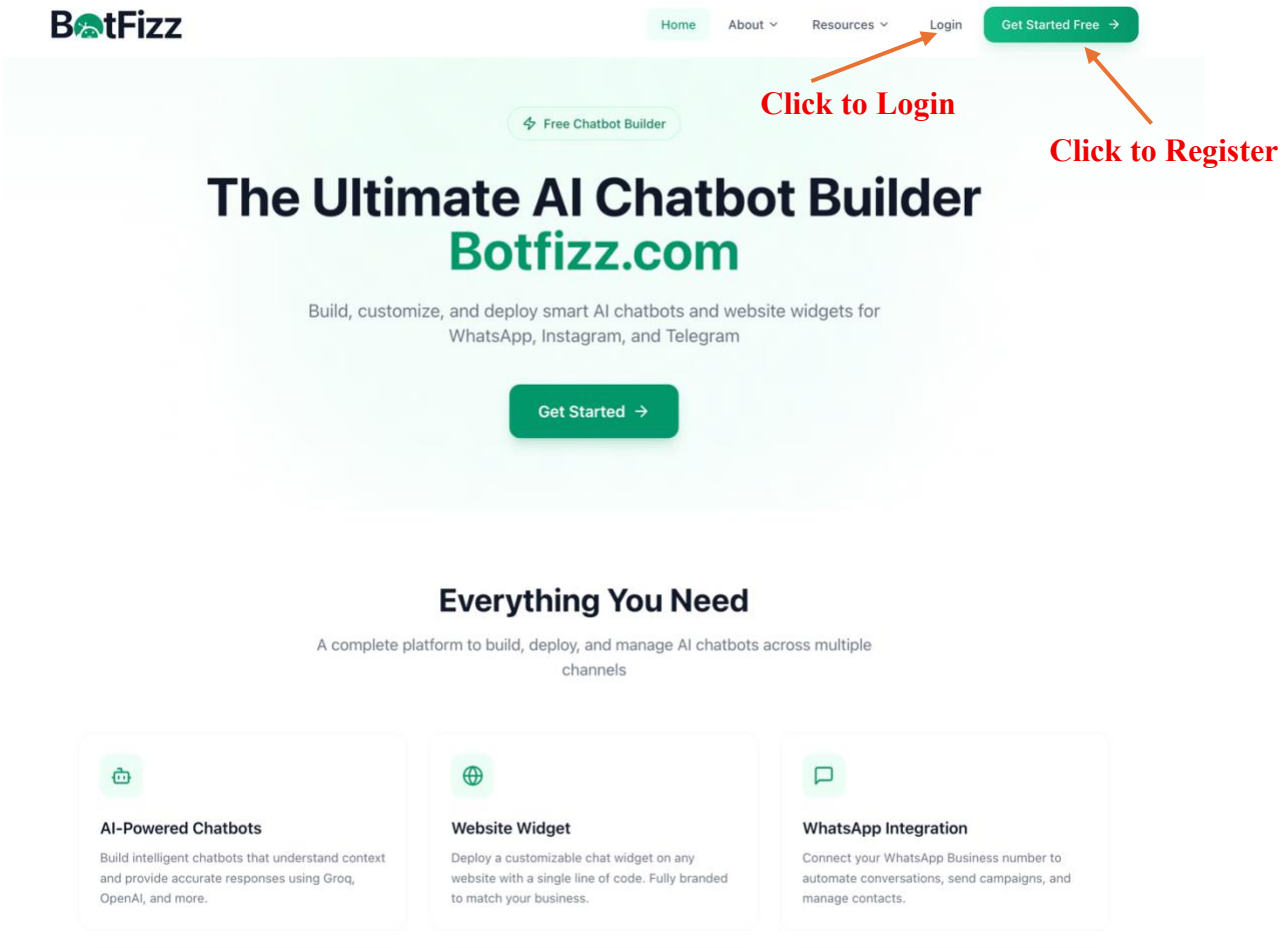


Figure 6.1 - BotFizz Landing Page. The Get Started Free button launches the registration process.

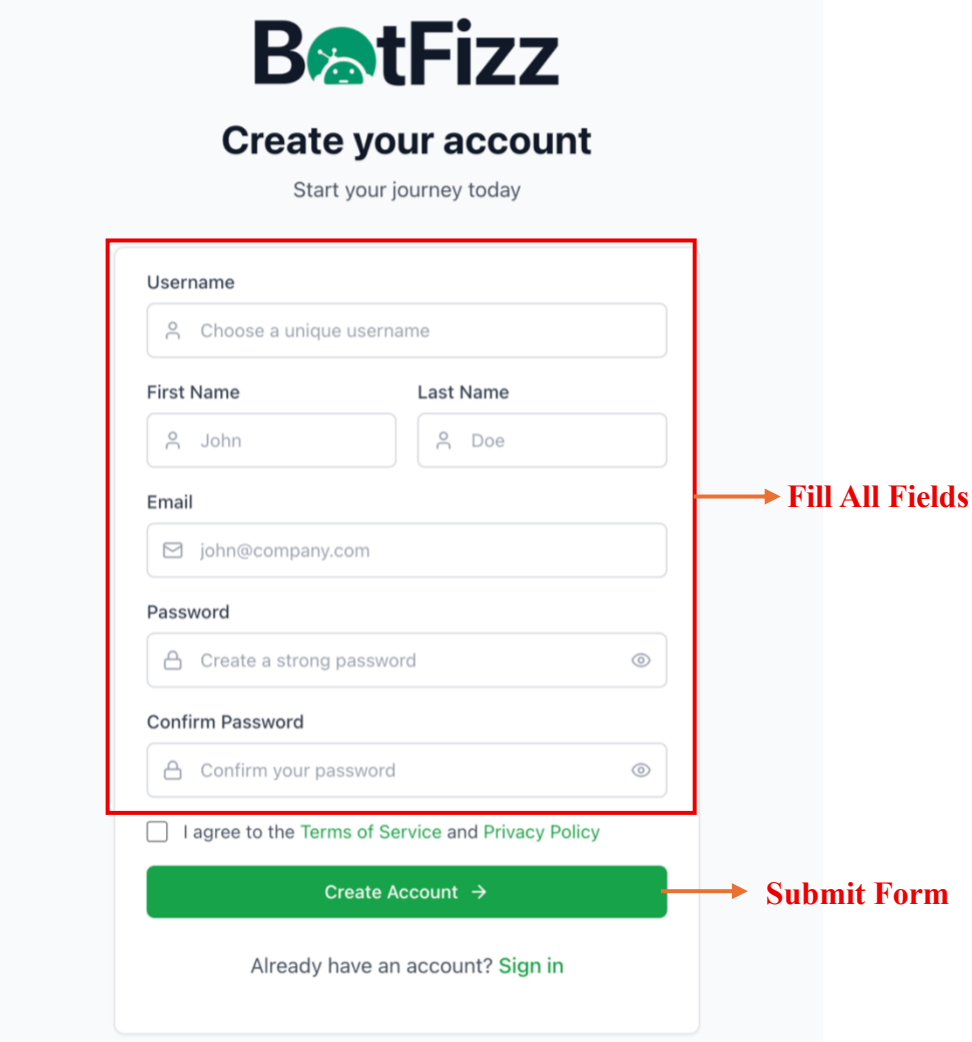


Figure 6.2 - Registration Form. Fill in all fields and click Create Account to proceed.

NOTE: Your password must be at least 8 characters long and contain at least one uppercase letter, one lowercase letter, and one number.

6.3.2 Email Verification (OTP)

After registration, BotFizz automatically sends a 6 digit One Time Password to the email address you provided. You must verify your email before you can log in.

7. Open your email inbox and look for an email from BotFizz with the subject Verify Your Email Address.
8. Note the 6 digit OTP code shown in the email.
9. Return to the BotFizz OTP Verification screen in your browser.
10. Enter the 6 digit OTP in the six input boxes provided.
11. Click the Verify Email button.

12. The system will validate the code. If correct, your account is activated and you are redirected to the Login screen with a success notification.
13. If the OTP has expired (after 5 minutes), click the Resend OTP button to receive a new code.

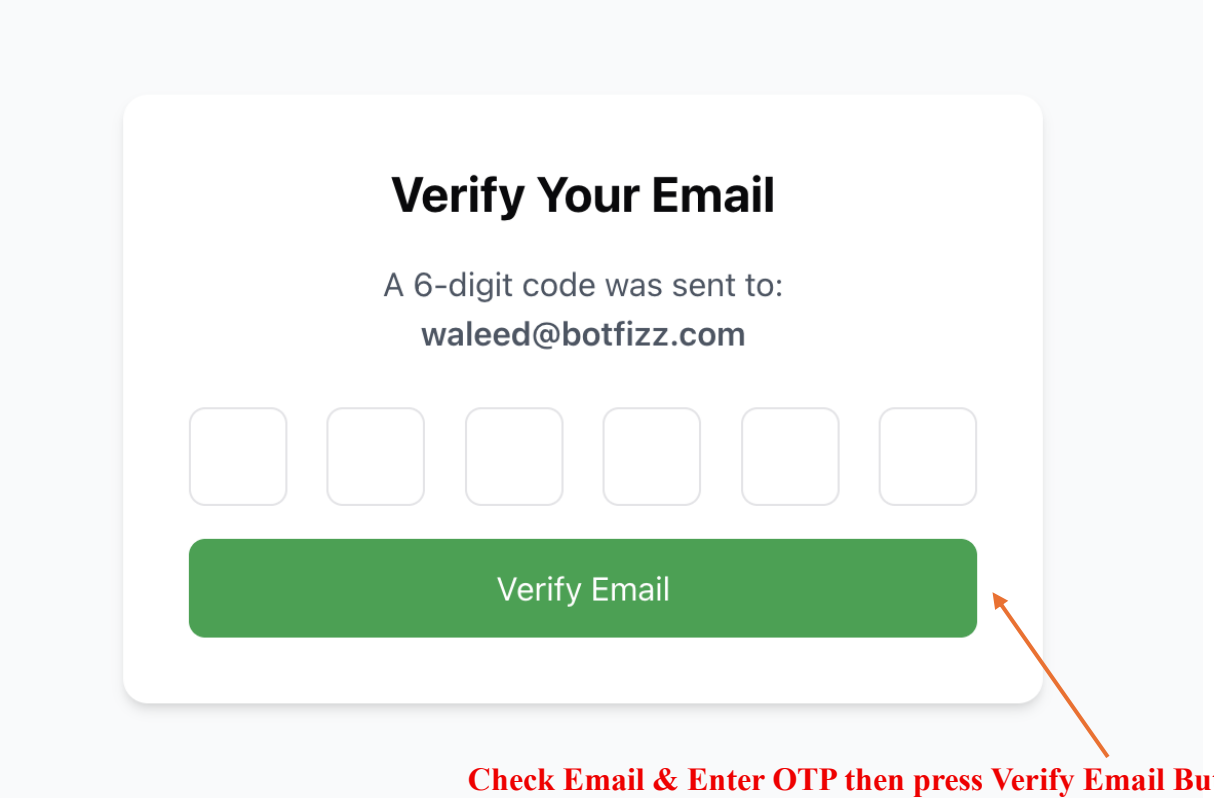


Figure 6.3 - OTP Verification Screen. Enter the 6 digit code from your email and click Verify Email.

6.3.3 Logging In

Once your email is verified, you can log in to BotFizz at any time.

14. Go to the BotFizz website and click Login in the top navigation or at the bottom of the landing page.
15. Enter your registered Email Address and Password.
16. Click the eye icon beside the Password field to reveal your password if you want to check it.
17. Click the Login button.
18. The system verifies your credentials, creates a secure session, and takes you to the Main Dashboard.

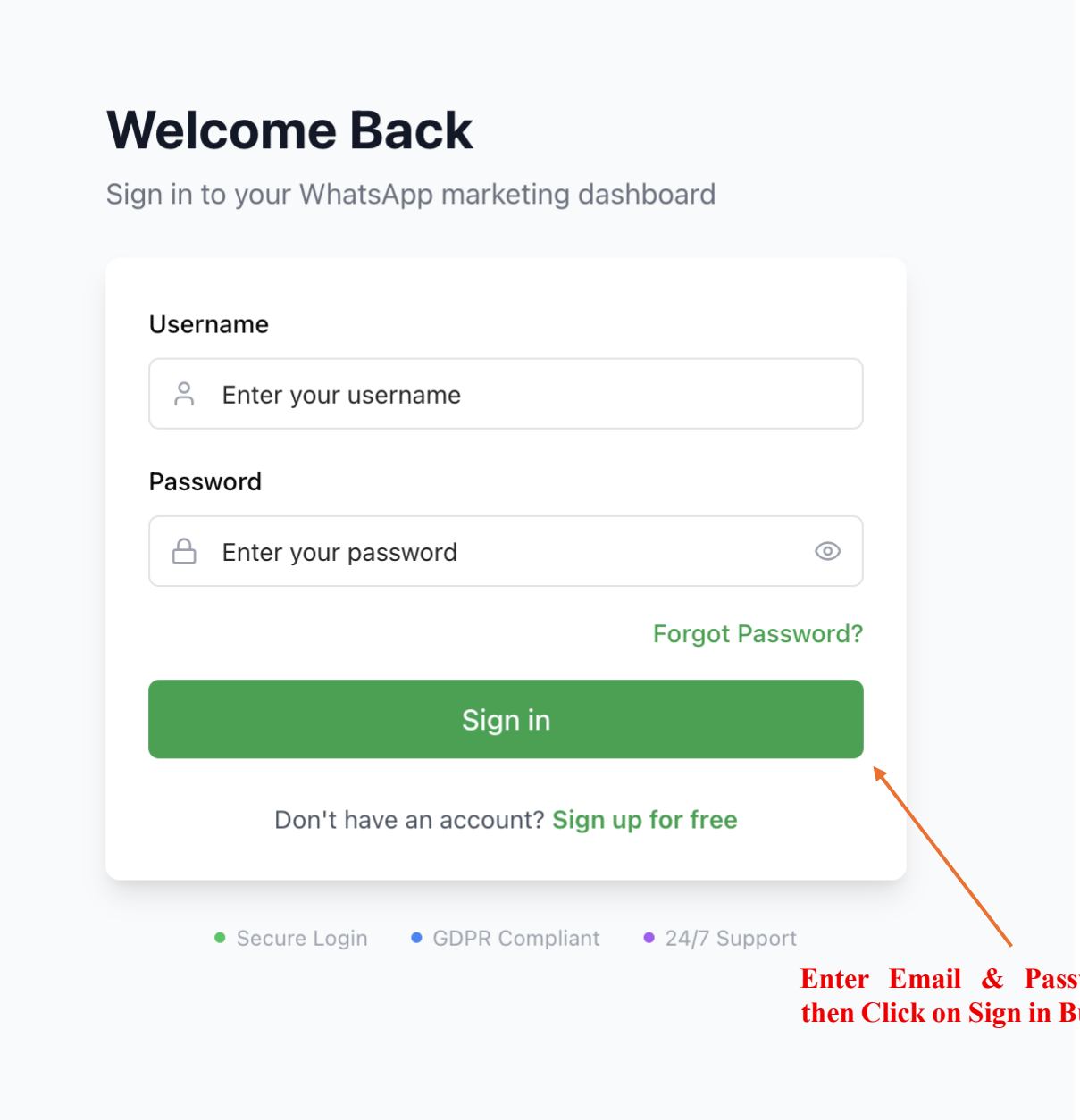


Figure 6.4 - Login Screen. Enter your email and password then click Login to access your dashboard.

6.3.4 Recovering a Forgotten Password

If you have forgotten your password, use the password recovery feature.

19. On the Login screen, click the Forgot Password link below the Login button.
20. Enter the email address you used when you registered.
21. Click Send Reset Link.
22. Open your email inbox and click the password reset link sent by BotFizz.
23. Enter your new password on the reset page, confirm it, and click Save New Password.
24. Log in with your new password.

6.3.5 Updating Your Profile and Logging Out

To update your name, email, or password, click your profile avatar in the top right corner of any screen and select Profile Settings. Make your changes and click Save.

To log out, click your profile avatar in the top right corner and select Logout. Your session will end and you will be returned to the landing page.

© University of Gujrat

6.4 Dashboard Overview

The Main Dashboard is the first screen you see after logging in. It gives you a real time summary of how your chatbots are performing across all platforms.

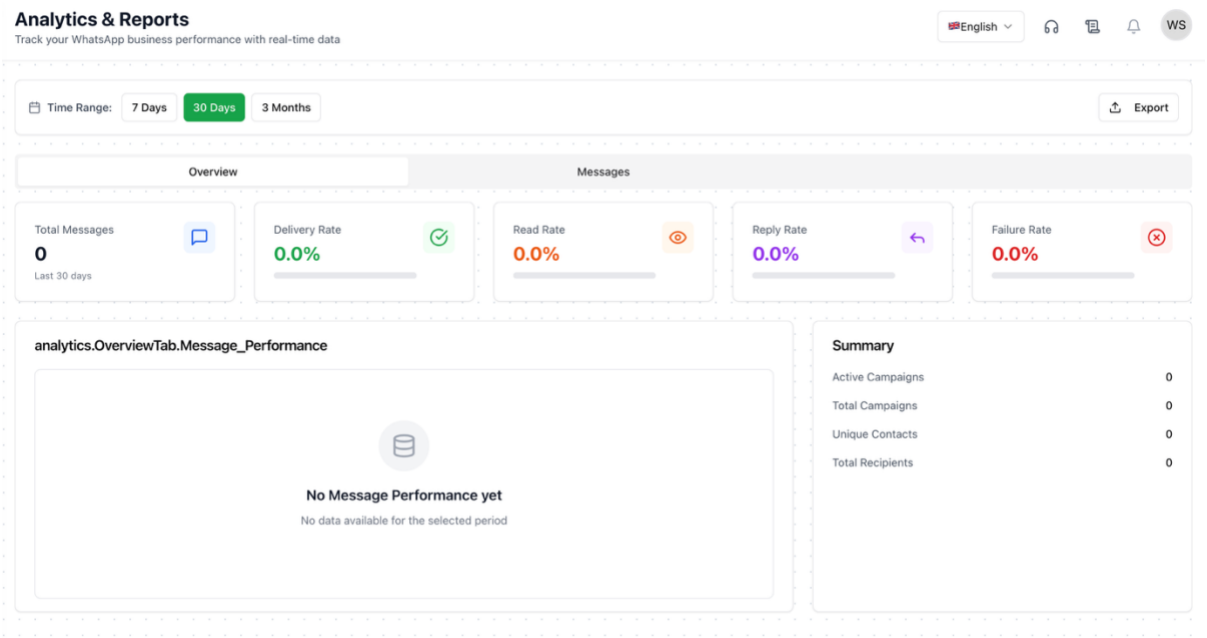


Figure 6.5 - Main Dashboard showing KPI cards, a message performance bar chart, and the navigation sidebar.

6.4.1 Dashboard Layout

The dashboard is divided into three main areas:

- Left Sidebar: The main navigation menu. Click any item to go to that section of the platform.
- Top Bar: Shows the current page name, Export Data button, and your profile icon.
- Main Content Area: Displays KPI cards, the performance chart, and the summary panel.

6.4.2 Understanding the KPI Metric Cards

At the top of the content area you will see five KPI (Key Performance Indicator) cards. Each card shows a headline metric for your chatbot activity during the selected time range.

Metric Card	What It Measures
Total Messages	The total number of messages sent by your chatbots across all platforms.
Delivery Rate	The percentage of messages that reached the recipient's device successfully.
Read Rate	The percentage of delivered messages that the recipient opened and read.
Reply Rate	The percentage of messages that received a reply from the recipient.
Failure Rate	The percentage of messages that could not be delivered due to errors.

6.4.3 Navigating the Platform

Use the left sidebar to move between sections. The sidebar contains the following navigation items:

- Dashboard: Returns you to this overview screen.
- My Chatbots: View, create, and manage your chatbot projects.
- Analytics: Access detailed analytics charts and the conversation history.
- Integrations: Connect WhatsApp, Instagram, and website widget integrations.
- Settings: Manage your account, profile, and billing.

TIP: Use the time range buttons (7 Days, 30 Days, 3 Months) at the top left of the chart to change the period shown in the performance graph.

© University of Gujrat

6.5 Creating Your First Chatbot

6.5.1 Starting a New Chatbot Project

A chatbot project is a container that holds one conversation flow, its AI settings, and its integration connections. Follow these steps to create your first chatbot.

25. Click My Chatbots in the left sidebar.
26. You will see the Chatbots screen. If you have no chatbots yet, it will show an empty state with a message and a central Create Your First Chatbot button.
27. Click either the Create Your First Chatbot button in the centre of the screen or the Create New Flow button in the top right corner.
28. The system creates a new chatbot project and immediately opens the Flow Builder canvas so you can start designing your conversation.

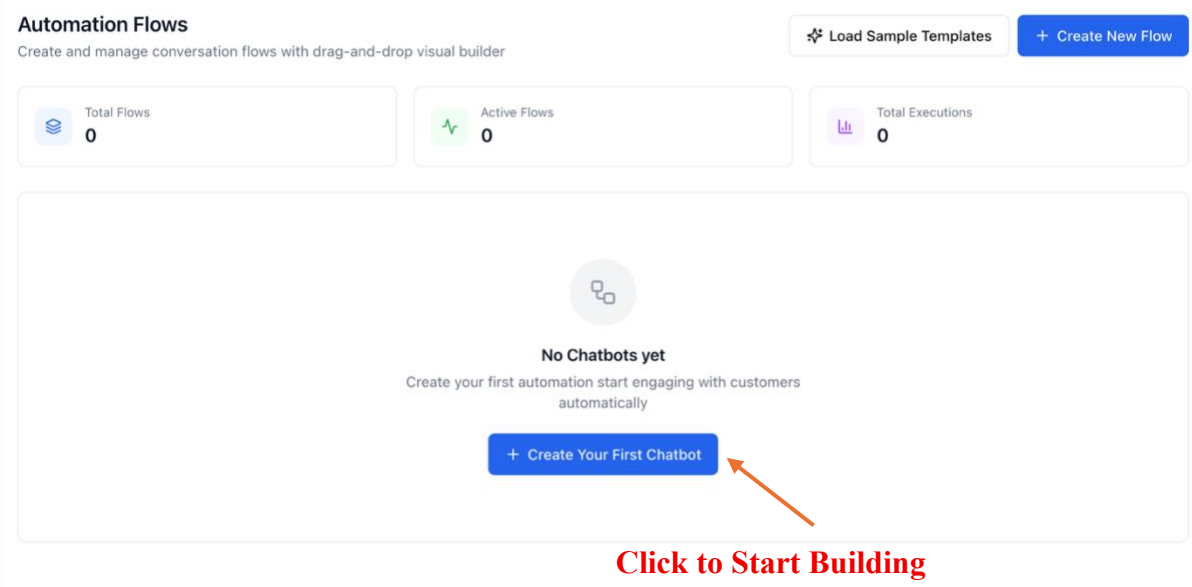


Figure 6.6 - My Chatbots empty state. Click Create Your First Chatbot to open the Flow Builder.

6.5.2 Understanding Chatbot Status

Each chatbot card on the My Chatbots screen shows a status badge. The possible statuses are described in the table below.

Status Badge	Meaning
Draft	The chatbot has been created but has not been deployed. It is not yet live.
Active	The chatbot is deployed and is responding to real users on at least one platform.
Paused	The chatbot is deployed but has been temporarily deactivated by you.
Error	A configuration or connection problem is preventing the chatbot from working.

6.6 Building Conversation Flows

The Flow Builder is where you design the conversation your chatbot will have with users. It uses a visual canvas where you drag and connect nodes to create a flow.

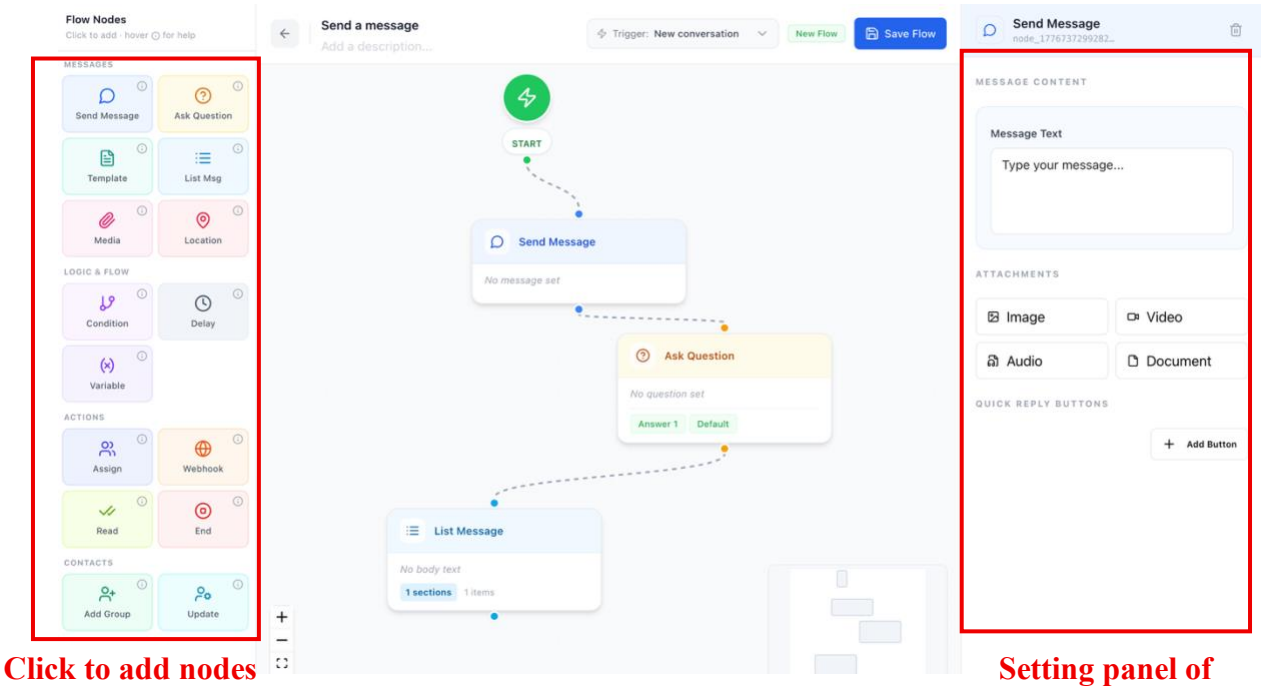


Figure 6.7 - Flow Builder Canvas. Drag nodes from the left palette, connect them on the canvas, and configure each node in the right panel.

6.6.1 Flow Builder Interface Overview

The Flow Builder is made up of four key areas:

- Top Toolbar: Contains Save, Preview, and Deploy buttons. Always save your work before previewing or deploying.
- Left Node Palette: Lists all available node types. Drag any node from here onto the canvas.
- Canvas Area: The main working space where you place and connect nodes. The dotted grid background indicates the canvas area.
- Right Configuration Panel: Opens automatically when you click on a node. Use it to set the content and behaviour of the selected node.

6.6.2 Node Types

BotFizz provides four types of nodes that you can use to build your conversation flow.

Message Node (Blue): Sends a fixed text message to the user. Use this to greet users, provide information, or display a menu of options.

Question Node (Purple): Asks the user a question and waits for their reply. The reply is stored and can be used in later nodes.

Condition Node (Orange): Creates a decision branch in the flow. Based on a condition such as a keyword or a stored reply, the flow follows either a Yes path or a No path.

Action Node (Green): Performs an automatic action such as sending a document, redirecting to a human agent, or calling an external API.

6.6.3 Adding Nodes to the Canvas

29. Find the node type you need in the left palette.
30. Click and hold the node, then drag it onto the canvas.
31. Release the mouse button to place the node. The node will snap to the nearest grid position.
32. Repeat for as many nodes as your conversation requires.

6.6.4 Configuring a Node

33. Click on any node on the canvas to select it.
34. The right Configuration Panel opens and shows the settings for that node type.
35. For a Message Node, type or paste the message text in the Text field. The bot will send this text exactly as written.
36. For a Question Node, type the question text and choose how to store the user's answer (for example, in a variable called user_query).
37. For a Condition Node, enter the keyword or variable to check and set the condition logic (for example, user_query contains pricing).
38. For an Action Node, choose the action type from the dropdown and fill in the required details.
39. Changes are saved automatically as you type in the configuration panel. You will see a small Saved indicator in the panel header.

6.6.5 Connecting Nodes

Nodes must be connected in sequence to form a complete conversation flow. Each node has a small connection dot on its right edge (output) and left edge (input).

40. Hover over the output dot on the right side of a node until you see a crosshair cursor.
41. Click and drag from the output dot to the input dot on the receiving node.
42. Release the mouse button when the connection line snaps to the target node. A visible line now connects the two nodes.
43. Condition Nodes have two output dots labelled Yes and No. Connect each output to the correct next node for that branch.

6.6.6 Saving Your Flow

44. Click the Save button in the top toolbar at any time to manually save your flow.
45. The platform also saves automatically every 30 seconds when the flow has changed.
46. A small Saved notification appears briefly in the toolbar to confirm your flow has been saved.

© University of Gujrat

WARNING: Always click Save before you click Deploy. Unsaved changes will not be included in the deployed version of your chatbot.

6.6.7 Testing Your Flow in the Preview Sandbox

47. Click the Preview button in the top toolbar.
48. A mock chat window opens on the right side of the screen.
49. Type a message as if you were a customer, for example Hello, and press Enter.
50. The bot will respond according to your configured flow. Follow the conversation to test all branches.
51. Type reset at any time in the preview window to restart the simulation from the beginning.
52. When you are satisfied that the flow works correctly, close the preview panel.

6.6.8 Flow Design Best Practices

- Keep each Message Node short. Long walls of text have a lower read rate in messaging apps.
- Always include a Fallback path. If the AI or a Condition Node does not match any expected input, the fallback should guide the user back to a useful option.
- Test every branch. Use the Preview Sandbox to walk through both the Yes and No paths of every Condition Node.
- Name your variables clearly. Instead of var1, use user_name or user_query so your flow remains readable when you return to edit it later.
- Add a closing Message Node. Always end your flow with a friendly closing message so users know the conversation has concluded.

6.7 Configuring AI

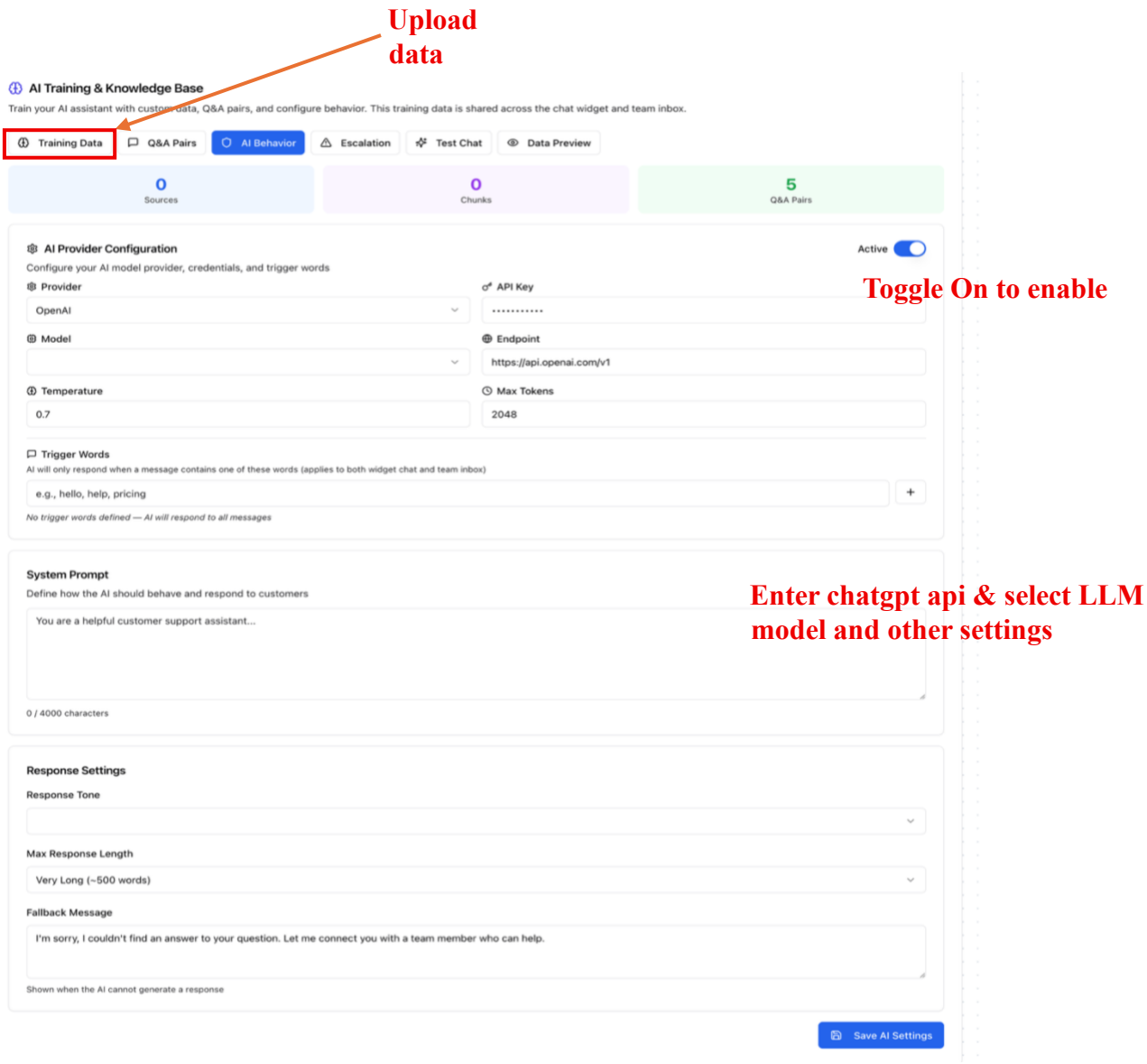


Figure 6.8 - AI Configuration Panel. Enable AI, select your provider, paste your API key, and adjust the confidence threshold.

6.7.1 What AI Does in BotFizz

When AI is enabled for a chatbot, BotFizz forwards incoming user messages to your chosen AI provider (such as OpenAI) before the message reaches the flow logic. The AI analyses the message, identifies the user's intent, and either routes the conversation to the correct flow node or generates a direct response.

If the AI's confidence score for its detected intent is below the threshold you set, the flow falls back to the standard Condition Node logic or a configured fallback message. This prevents the bot from giving incorrect answers when it is not sure.

6.7.2 Enabling AI for a Chatbot

53. Open the chatbot you want to configure and click the AI Settings tab, or navigate to Integrations and select AI Configuration.
54. Click the Enable AI toggle. The toggle turns green when AI is active.
55. From the AI Service Provider dropdown, select your provider. The default option is OpenAI (GPT 4).
56. Paste your API key from your AI provider account into the API Key field.
57. Click the Verify button next to the API Key field. BotFizz will send a test request to confirm the key is valid. A green confirmation message will appear if the key is accepted.
58. Click Save AI Settings.

NOTE: Your API key is encrypted before it is stored. BotFizz staff cannot read your key. The key is only used to send requests to your chosen AI provider on behalf of your chatbot.

6.7.3 Uploading Training Data

You can upload a document containing frequently asked questions and their answers to improve how the AI understands your business context.

59. In the AI Configuration panel, scroll to the Upload Training Data section.
60. Drag and drop your FAQ document onto the upload area, or click Browse to choose a file from your computer.
61. Supported file types are PDF, and plain text TXT files.
62. The platform will process the document and use it as additional context when the AI generates responses.

TIP: Keep your training document focused. A clear list of 20 to 50 specific questions with detailed answers gives better AI responses than a large general purpose document.

6.7.4 Adjusting the Confidence Threshold

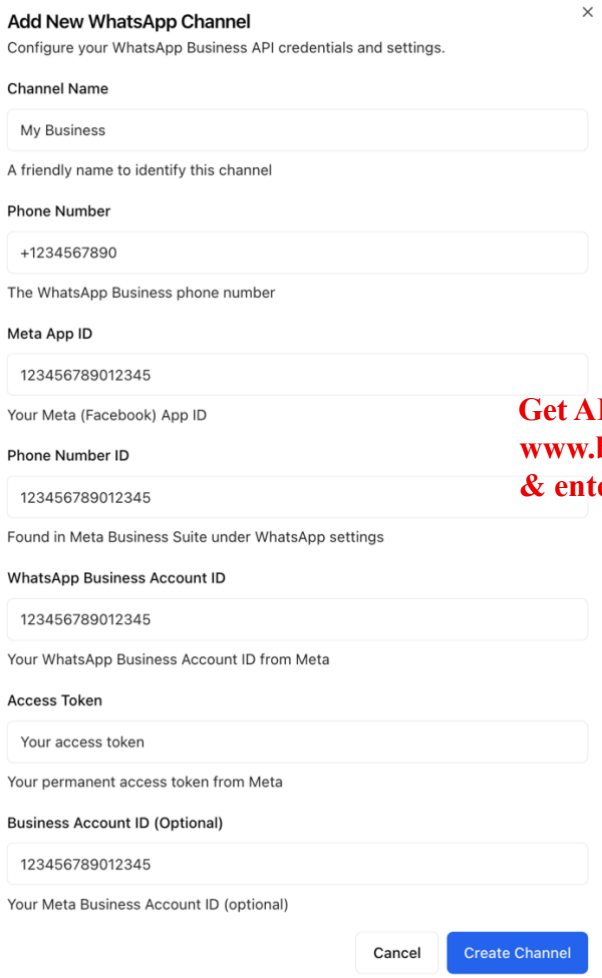
The Confidence Threshold slider controls how certain the AI must be before it acts on a detected intent. The slider ranges from 0 percent to 100 percent. A setting of 75 percent is recommended for most businesses.

- Setting the threshold too low (below 50 percent) causes the AI to respond even when it is unsure, which may produce irrelevant answers.
- Setting the threshold too high (above 90 percent) makes the AI fall back very often, which reduces its usefulness.
- Start at 75 percent, review conversation logs after a week, and adjust as needed.

© University of Gujrat

6.8 Platform Integrations

BotFizz can connect your chatbot to WhatsApp, Instagram, and your own website. Each integration requires credentials from the corresponding platform's developer portal.



Get API from www.business.facebook.com & enter ID & Access Token

For more detailed information check our official website documentation www.botfizz.com

Figure 6.9 - WhatsApp Integration Screen. Paste your Meta credentials and click Connect WhatsApp.

6.8.1 WhatsApp Business Integration

The WhatsApp integration uses the Meta WhatsApp Business API. Before you begin, ensure you have a verified Meta Business account and an approved WhatsApp Business phone number.

Information You Will Need: Phone Number ID, Business Account ID, and a permanent WhatsApp Access Token. All three are available in your Meta Developer Portal under WhatsApp settings.

63. Click Integrations in the left sidebar, then click the WhatsApp card.
64. In the Phone Number ID field, paste the Phone Number ID from your Meta Developer Portal.
65. In the Business Account ID field, paste your Meta Business Account ID.
66. In the WhatsApp Access Token field, paste your permanent access token.
67. Copy the Webhook URL shown on the screen and paste it into the Webhook URL field in your Meta Developer Portal. Set the verify token to the one shown in BotFizz.
68. Click the Connect WhatsApp button.
69. BotFizz will validate your credentials by making a test call to the Meta API. If successful, the status badge changes from Disconnected to Connected, and a WhatsApp QR code appears for sharing.

NOTE: If the connection fails, double check that your Access Token has not expired and that your WhatsApp number has been approved in the Meta Developer Portal.

6.8.2 Instagram Messaging Integration

The Instagram integration uses the Meta Graph API for Instagram Business accounts.

Information You Will Need: Instagram Business Account ID and a Page Access Token from your Meta Developer Portal.

70. Click Integrations in the left sidebar, then click the Instagram card.
71. Paste your Instagram Business Account ID and your Page Access Token into the respective fields.
72. Click Connect Instagram.
73. If validation is successful, the status badge changes to Connected.

6.8.3 Website Widget Integration

The website widget places a chat bubble on your website that visitors can click to start a conversation with your chatbot.

74. Click Integrations in the left sidebar, then click the Website Widget card.
75. Customise the widget colour, button position, and greeting message using the options provided.
76. Click Generate Widget Code.
77. Copy the generated script tag and paste it into the HTML of your website just before the closing body tag.

78. Save and publish your website. The chat bubble will appear for visitors within a few minutes.

TIP: You do not need any developer accounts for the website widget. The script tag can be added to any website builder that allows custom HTML, including WordPress, Shopify, and Wix.

6.8.4 Managing Integration Status

The status badge on each integration card shows the current connection state. Green indicates Connected, red indicates Disconnected, and yellow indicates a warning that requires attention. Hover over a warning badge to see a tooltip explaining the issue.

6.8.5 Disconnecting an Integration

79. Click Integrations in the left sidebar and open the integration you want to disconnect.
80. Scroll to the bottom of the integration screen and click Disconnect.
81. A confirmation dialog will appear. Click Confirm Disconnect.
82. The status badge will change to Disconnected. Your chatbot will stop sending or receiving messages on that platform immediately.

© University of Gujrat

6.9 Deploying Your Chatbot

Deploying your chatbot makes it live so that real users can interact with it. You must have at least one integration connected and your flow saved before you can deploy.

6.9.1 How to Deploy

83. Open the chatbot you want to deploy from the My Chatbots screen.
84. In the Flow Builder toolbar, click the Active Toggle button, or navigate to the Toggle tab.
85. The activeate screen shows all available platforms. Platforms that are connected and ready show a green Active or Connected badge. Platforms that are not connected show a grey Not Connected badge.
86. To Activate to a single platform, click the Deploy to [Platform Name] button on that platform's card.
87. To deploy to all connected platforms at once, click the Active to ai least one Connected Platforms button.
88. The Deployment Log panel at the bottom of the screen shows real time progress. A success message will confirm when the chatbot is live on each platform.

WARNING: Deploying a chatbot with an incomplete flow (for example, a flow with unconnected nodes) may cause the bot to send empty responses. Always test your flow in the Preview Sandbox before deploying.

6.9.2 Sharing Your WhatsApp QR Code

After a successful WhatsApp deployment, a QR code is generated on the deploy screen. Users can scan this QR code with their phone's camera to start a conversation with your chatbot instantly. You can download the QR code image by clicking the Download QR button and share it on your website, social media, or printed materials.

6.9.3 Updating a Live Chatbot

You can edit a live chatbot at any time. Simply open the Flow Builder, make your changes, click Save, and then click Deploy again. The updated flow will go live within seconds and all new conversations will use the updated version. Conversations that are already in progress will complete using the previous version.

6.10 Managing Your Chatbots

The My Chatbots screen displays all chatbot projects you have created. Each project appears as a card showing the chatbot name, status badge, last modified date, and the platforms it is connected to.

6.10.1 Chatbot Card Actions

Each chatbot card has a menu of actions. Click the three dot icon in the top right corner of any card to see the following options.

Action	What It Does
Edit Flow	Opens the Flow Builder so you can change the conversation design.
AI Settings	Opens the AI Configuration panel for this chatbot.
Integrations	Opens the Integrations screen filtered to this chatbot.
Rename	Lets you change the display name of the chatbot.
Duplicate	Creates a full copy of the chatbot including its flow and AI settings.
Deactivate	Pauses the chatbot without deleting it. Users will not get responses until it is reactivated.
Delete	Permanently removes the chatbot and all its data. This action cannot be undone.

6.10.2 Deactivating vs. Deleting

Deactivating a chatbot stops it from responding to users but keeps all of your flow design, AI settings, and conversation history intact. You can reactivate it at any time by opening the card menu and clicking Activate.

Deleting a chatbot permanently removes everything associated with it. Use this option only if you are sure you will never need the chatbot again.

> **WARNING:** Deleted chatbots and their conversation history cannot be recovered. If you are unsure, deactivate the chatbot instead of deleting it.

© University of Gujrat

6.11 Viewing Conversation History

Open Dashboard & Click Conversation then view conversation history will be shown.

6.11.1 Accessing Conversation Logs

To view all conversations your chatbots have had with users, click Analytics in the left sidebar and then select the Conversations tab. Every conversation is listed in reverse chronological order with the most recent at the top.

6.11.2 Filtering Conversations

Use the filter bar at the top of the screen to narrow down the list. You can filter by:

- Platform: Show only WhatsApp, Instagram, or Website conversations.
- Status: Show only Completed, Active, or Escalated conversations.
- Date Range: Limit results to a specific time period.
- Search: Type a keyword or user identifier to find a specific conversation.

Click the Apply Filters button after setting your criteria, or click Clear Filters to reset all filters and show all conversations.

6.11.3 Understanding a Conversation Thread

Click on any row in the conversation list to open the full conversation thread on the right side of the screen. The thread shows all messages in chronological order. Messages sent by the bot are shown on the right in a blue bubble. Messages from the user are shown on the left in a grey bubble. A timestamp appears beneath each message.

© University of Gujrat

6.12 Analytics Dashboard

You can check analytics by clicking the main of the dashboard of your login.

6.12.1 Accessing Analytics

Click Analytics in the left sidebar to go to the Analytics Dashboard. This screen provides a complete picture of how your chatbots are performing.

6.12.2 Key Metrics Explained

Metric	Definition
Total Messages	All messages sent by all chatbots in the selected period.
Delivery Rate	Percentage of messages that were successfully delivered to recipients.
Read Rate	Percentage of delivered messages that the recipient opened.
Reply Rate	Percentage of messages that received a response from the user.
Failure Rate	Percentage of messages that failed to deliver. A healthy failure rate is below 10 percent.
Top Performing Flows	Flows ranked by completion rate, showing which conversations are most successful.
Platform Distribution	Pie chart showing the percentage of conversations on each platform.

6.12.3 Changing the Time Range

Use the time range selector in the top left corner of the chart to change the period displayed. Options are 7 Days, 30 Days, and 3 Months. The metric cards and all charts update instantly when you select a different range.

6.12.4 Exporting Analytics Data

89. Click the Export CSV button in the top right corner of the Analytics Dashboard.
90. A CSV file will be downloaded to your computer containing all metrics for the currently selected time range.
91. Open the CSV in Excel, Google Sheets, or any spreadsheet application to perform further analysis or create custom reports.

© University of Gujrat

6.13 Troubleshooting

6.13.1 Account and Login Issues

Problem	Solution
I did not receive my OTP email.	Check your spam or junk folder. Wait 2 minutes and then click Resend OTP. Ensure you entered the correct email address during registration.
My OTP is invalid.	OTPs expire after 5 minutes. Click Resend OTP to get a fresh code and enter it within the validity window.
I forgot my password.	Click Forgot Password on the Login screen, enter your email address, and follow the instructions in the reset email.
My account is locked.	After five consecutive failed login attempts, your account is temporarily locked for 15 minutes. Wait and try again or use Forgot Password.

6.13.2 Flow Builder Issues

Problem	Solution
My flow does not save.	Check your internet connection. Try clicking Save again. If the problem persists, refresh the page. Unsaved changes will be lost on refresh, so try saving first.
Nodes are overlapping on the canvas.	Click the Auto Layout button in the canvas toolbar to automatically rearrange all nodes neatly.
I cannot connect two nodes.	Each node input can only have one incoming connection. If the target node already has a connection, remove the existing one before adding a new one.
The Preview shows an empty response.	This means a node has no content configured. Click each node in the flow and check that the text fields are filled in.

6.13.3 AI Configuration Issues

Problem	Solution
API key verification fails.	Ensure you copied the full key without leading or trailing spaces. Check that the key is active in your AI provider account and has not reached its usage limit.
AI gives irrelevant responses.	Lower the Confidence Threshold to a value between 60 and 70 percent. Also, upload a more specific training document that focuses on your business topics.
AI training document upload fails.	Ensure the file is in PDF, DOCX, or TXT format and is under 10 MB in size.

© University of Gujrat

6.13.4 Integration Issues

Problem	Solution
WhatsApp shows Disconnected after I entered credentials.	Verify that the webhook URL and verify token in the Meta Developer Portal match exactly what BotFizz shows. Ensure your WhatsApp Business number is approved.
Users are not receiving messages.	Check the Failure Rate metric on the Analytics Dashboard. A high failure rate may indicate that your WhatsApp Access Token has expired. Generate a new token in Meta Developer Portal and reconnect.
Website widget does not appear.	Make sure the embed script tag is placed just before the closing body tag. Clear your browser cache and reload the page.

6.13.5 Analytics Issues

Problem	Solution
Analytics shows no data.	Data appears only after your chatbot has had real conversations. Ensure your chatbot is deployed and Active. Check that the selected time range covers the period when conversations took place.
Export CSV is empty.	Make sure you have conversations within the selected time range. Try changing to a longer time range such as 3 Months before exporting.

6.14 Glossary

Term	Definition
API Key	A private code that allows BotFizz to communicate with a third party service such as OpenAI or the Meta WhatsApp API on your behalf.
Chatbot	An automated program that carries out conversations with users according to a designed conversation flow.
Condition Node	A flow builder node that creates a Yes or No branch based on a condition such as a keyword in the user's message.
Confidence Threshold	A percentage value that determines how certain the AI must be before it uses its result. Messages below the threshold are handled by the fallback logic.
Conversation Flow	The sequence of messages, questions, and conditions that determine how your chatbot responds to users.
CSV	Comma Separated Values. A simple file format that stores table data and can be opened in Excel or Google Sheets.
Deployment	The process of making a chatbot live so that real users can interact with it on one or more platforms.
Fallback	A default response sent by the bot when it cannot match the user's input to any configured flow node or AI intent.
Flow Builder	The drag and drop canvas in BotFizz where you design your chatbot's conversation logic.
Intent	The purpose or goal behind a user's message as understood by the AI. For example, the intent of I want to check my order is order tracking.
JWT	JSON Web Token. A secure, encrypted token that BotFizz uses to identify logged in users without storing passwords in browser cookies.
KPI	Key Performance Indicator. A measurable value that shows how effectively a goal is being achieved. In BotFizz, KPIs include Delivery Rate, Read Rate, and Reply Rate.
Node	A single step or element in the Flow Builder canvas. Examples include Message Node, Question Node, Condition Node, and Action Node.
OTP	One Time Password. A 6 digit code sent to your email to verify your identity. It expires after 5 minutes.
SaaS	Software as a Service. A software delivery model where the application runs in the cloud and is accessed through a web browser without installation.
Webhook	A URL that receives automatic notifications from an external service. BotFizz uses webhooks to receive incoming messages from WhatsApp and Instagram.
Widget	A small, embedded chat bubble placed on a website that users can click to start a conversation with your chatbot.

End of Chapter 6 - User Technical Manual

www.BotFizz.com | Department of Software Engineering | University of Gujrat